

鲲鹏 BoostKit 大数据使能套件

部署指南（HDP）

文档版本 04
发布日期 2021-06-30



版权所有 © 华为技术有限公司 2021。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

目录

1 介绍	1
2 环境要求	3
3 配置部署环境	5
3.1 检查操作系统	5
3.2 设置域名解析	6
3.3 关闭防火墙	7
3.4 关闭 SELinux 和配置 limits 参数	7
3.5 禁用交换分区	8
3.6 设置 umask	8
3.7 设置节点互信	9
3.8 配置 OS 镜像源	10
3.9 安装依赖软件	10
3.10 安装 JDK	11
3.11 搭建 Yum 源服务器	12
3.12 操作系统时钟设置	17
3.13 (可选)创建普通用户	18
3.14 关闭 dnsmasq（仅 openEuler 系统需操作）	19
4 部署 Ambari	20
4.1 安装数据库	20
4.2 部署 Ambari-server	22
5 部署 Hadoop 集群	26
6 部署 Accumulo	44
6.1 部署 Accumulo 服务	44
6.2 删除 Accumulo 服务	47
7 部署 Atlas&Kafka&Solr	50
7.1 部署 Atlas&Kafka&Solr 服务	50
7.2 删除 Atlas&Kafka&Solr 服务	53
8 部署 HBase&Phoenix	56
8.1 部署 HBase&Phoenix 服务	56
8.2 删除 HBase&Phoenix 服务	59

9 部署 Kafka	61
9.1 部署 Kafka 服务	61
9.2 删除 Kafka 服务	64
10 部署 Knox	67
10.1 部署 Knox 服务	67
10.2 删除 Knox 服务	69
11 部署 Oozie	72
11.1 部署 Oozie 服务	72
11.2 删除 Oozie 服务	75
12 部署 Pig	79
12.1 部署 Pig 服务	79
12.2 删除 Pig 服务	81
13 部署 Ranger	83
13.1 部署 Ranger 服务	83
13.2 删除 Ranger 服务	87
14 部署 Spark&Livy	90
14.1 部署 Spark&Livy 服务	90
14.2 删除 Spark&Livy 服务	93
15 部署 Sqoop	95
15.1 部署 Sqoop 服务	95
15.2 删除 Sqoop 服务	97
16 部署 Storm	100
16.1 部署 Storm 服务	100
16.2 删除 Storm 服务	103
17 部署 Zeppelin	105
17.1 部署 Zeppelin 服务	105
17.2 删除 Zeppelin 服务	108
18 卸载 Ambari 集群	110
19 故障排除	111
19.1 无权访问 httpd 文件服务器下级目录	111
19.2 提示未绑定变量	112
19.3 安装 cluster 失败	113
19.4 Ambari 部署时进度部署到 5%时立刻失败	113
19.5 在主机注册时失败	114
19.6 提示 Connection failed: [Errno 111] Connection refused to XXX:16030	115
19.7 ambari-metrics 启动失败: cannot import name _common	115
19.8 ambari-server 启动失败: listening on http port 8080 after 90 seconds	116
19.9 提示需要安装 openeuler-lsb 和 python2-kerberos	116

A 修订记录..... 117

1 介绍

Ambari是Apache Software Foundation中的一个顶级项目，它可以创建、管理、监视Hadoop整个生态圈（例如Hive、HBase、Sqoop、ZooKeeper等）的集群，使得Hadoop以及相关的大数据软件更容易使用。

Ambari是一个分布式架构的软件，由Ambari Server和Ambari Agent两部分组成，用户可通过Ambari Server通知Ambari Agent安装对应的软件，Ambari Agent会定时地发送各个机器每个软件模块的状态给Ambari Server，最终这些状态信息会呈现在Ambari的GUI界面上，方便用户了解到集群的各种状态，并进行相应的维护。

简要介绍

部署Ambari集群前，需准备每台服务器环境：安装OS（本文统一采用CentOS 7.6/openEuler 20.03操作系统）、关闭SELinux权限、关闭防火墙等；部署Ambari集群的整体流程分为2步：

步骤1 配置部署环境。

该步骤包括：更新本地Yum源、安装Ambari Server、安装OpenJDK等。

步骤2 部署Ambari Server。

该步骤包括：Ambari Server Setup、启动Ambari Server服务、安装Ambari Agent、安装组件、启动组件服务等。

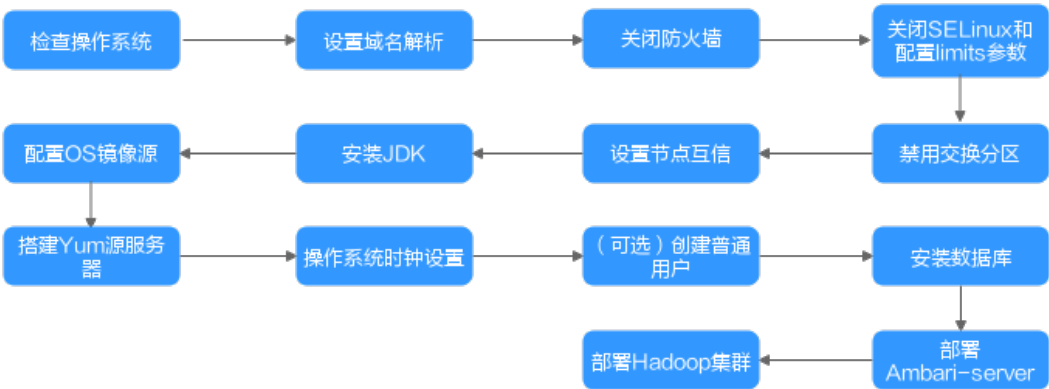
----结束

软件	版本
OpenJDK	1.8.0
Ambari	2.7.3.0-139

部署流程图

部署流程图如[图1-1](#)所示。

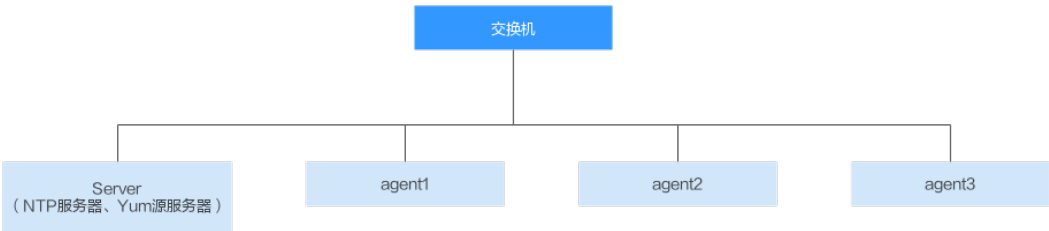
图 1-1 部署流程图



物理组网

物理组网方式如图1-2所示。

图 1-2 物理组网



说明

Yum源服务器建议单独拿一台服务器独立部署。

2 环境要求

硬件要求

 说明

TaiShan服务器部署场景、x86服务器部署场景和混合部署场景下，Ambari的部署方法一样。

表 2-1 TaiShan 服务器部署场景

项目	说明
服务器	TaiShan 200服务器
CPU	华为鲲鹏920处理器
网络	可访问外网
磁盘容量	500GB以上

表 2-2 x86 服务器部署场景

项目	说明
服务器	x86服务器
CPU	x86处理器
网络	可访问外网
磁盘容量	500GB以上

表 2-3 TaiShan 服务器和 x86 服务器混合部署场景

项目	说明
服务器	TaiShan 200服务器和x86服务器

项目	说明
CPU	TaiShan 200服务器：华为鲲鹏920处理器 x86服务器：x86处理器
网络	可访问外网
磁盘容量	500GB以上

操作系统要求

表 2-4 TaiShan 服务器的操作系统要求

项目	版本
CentOS/openEuler	7.6/20.03
Kernel	4.14.0/4.19.90

表 2-5 x86 服务器的操作系统要求

项目	版本
CentOS	7.6
Kernel	3.10.0

3 配置部署环境

- 3.1 检查操作系统
- 3.2 设置域名解析
- 3.3 关闭防火墙
- 3.4 关闭SELinux和配置limits参数
- 3.5 禁用交换分区
- 3.6 设置umask
- 3.7 设置节点互信
- 3.8 配置OS镜像源
- 3.9 安装依赖软件
- 3.10 安装JDK
- 3.11 搭建Yum源服务器
- 3.12 操作系统时钟设置
- 3.13 (可选)创建普通用户
- 3.14 关闭dnsmasq（仅openEuler系统需操作）

3.1 检查操作系统

须知

同一个集群中不可以同时存在鲲鹏CentOS7.6系统和鲲鹏openEuler20.03系统，只能在两者中选择其一！

步骤1 检查操作系统版本。

- CentOS7.6
cat /etc/redhat-release

```
[root@server1 yum.repos.d]# cat /etc/redhat-release  
CentOS Linux release 7.6.1810 (AltArch)
```

- openEuler20.03
cat /etc/openEuler-release

```
[root@server1 ~]# cat /etc/openEuler-release  
openEuler release 20.03 (LTS)
```

说明

对于TaiShan服务器支持的操作系统为CentOS 7.6/openEuler20.03。

步骤2 检查操作系统默认语言。

```
echo $LANG
```

```
[root@server1 yum.repos.d]# echo $LANG  
en_US.UTF-8
```

- 若操作系统默认语言为en_US.UTF-8，则进入[步骤3](#)。
- 若操作系统默认语言非en_US.UTF-8，则需要切换成英文。
 - a. 将默认语言由中文切换为英文。
echo 'export LANG=en_US.UTF-8'>> ~/.bashrc
 - b. 将当前会话关闭后重新登录。

步骤3 设置操作系统时区。

操作系统时区统一设置为（Asia/Shanghai）。

```
cp /usr/share/zoneinfo/Asia/Shanghai /etc/localtime
```

说明

如果安装OS时已经将时区设置为Asia/Shanghai，则[步骤3](#)可忽略。

----结束

3.2 设置域名解析

集群所有主机都必须配置域名解析，才能实现Server对Agent的顺利访问。建议由系统管理员统一配置，配置方法如下：

步骤1 在每台主机上设置相应的主机名。

1. 编辑网络配置文件。
vi /etc/sysconfig/network
设置文件内容为：
NETWORKING=yes
HOSTNAME={主机名}
2. 设置主机名（假设主机名为server1，agent1，agent2，agent3）。
hostnamectl --static set-hostname {主机名}

步骤2 配置“/etc/hosts”。

1. 为集群中每一台主机添加一行包含主机IP地址与完整主机名的信息。
vi /etc/hosts
例如，步骤中设置的主机名为server1，agent1，agent2，agent3：
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
::1 localhost localhost.localdomain localhost6 localhost6.localdomain6
IP1 server1
IP2 agent1

```
IP3 agent2  
IP4 agent3
```

- 可以在一台主机上配置，然后同步“/etc/hosts”到集群中所有节点上。
`scp /etc/hosts root@{目标ip}:/etc/hosts`
- 重启所有节点网络服务。

📖 说明

- CentOS中执行：
`systemctl restart network`
- openeuler中执行：
`systemctl restart NetworkManager`

步骤3 验证。

- 返回主机IP地址。
`hostname -i`
- 返回主机名称。
`hostname -f`

```
[root@server1 yum.repos.d]# hostname -i  
192.168.1.10  
[root@server1 yum.repos.d]# hostname -f  
server1
```

----结束

3.3 关闭防火墙

📖 说明

Ambari部署Hadoop集群时需要保证Ambari Server能够连接到Ambari Agent。

进行安装任务，集群所有主机都必须关闭防火墙。

```
systemctl stop firewalld  
systemctl disable firewalld
```

3.4 关闭 SELinux 和配置 limits 参数

步骤1 更改SELinux为宽容模式。

```
setenforce 0
```

步骤2 编辑“/etc/selinux/config”文件，永久禁止SELinux自动启动。

```
vi /etc/selinux/config
```

设置SELINUX=disabled，如下所示：

```
# This file controls the state of SELinux on the system.  
# SELINUX= can take one of these three values:  
#   enforcing - SELinux security policy is enforced.  
#   permissive - SELinux prints warnings instead of enforcing.  
#   disabled - No SELinux policy is loaded.  
SELINUX=disabled  
# SELINUXTYPE= can take one of three values:  
#   targeted - Targeted processes are protected,  
#   minimum - Modification of targeted policy. Only selected processes are protected.  
#   mls - Multi Level Security protection.  
SELINUXTYPE=targeted
```


步骤3 编辑“/etc/security/limits.conf”文件。

```
vi /etc/security/limits.conf
```

添加如下最后四行的内容：

```
#*      soft  core      0
#*      hard  rss       10000
#@student    hard  nproc    20
#@faculty    soft  nproc    20
#@faculty    hard  nproc    50
#ftp         hard  nproc    0
#@student    -     maxlogins  4

# End of file

* hard  nproc  65535
* soft  nproc  65535
* hard  nofile 65535
* soft  nofile 65535
```

----结束

3.5 禁用交换分区

在群内所有主机禁用交换分区。

```
sysctl vm.swappiness=0
echo vm.swappiness=0 >> /etc/sysctl.conf
```

3.6 设置 umask

为集群所有主机设置umask，设置用户创建目录默认权限。不同的系统采用不同的操作。

CentOS 7.6（鲲鹏/x86）

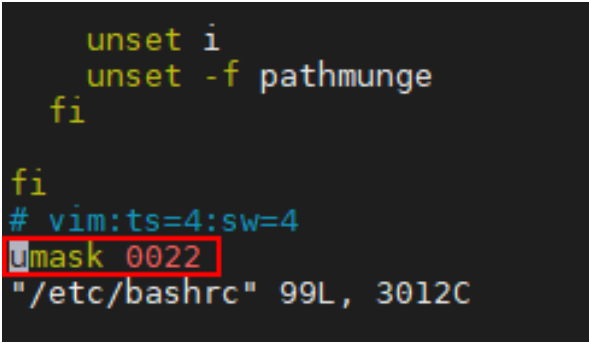
CentOS 7.6操作系统。

```
umask 0022
echo umask 0022 >> /etc/profile
source /etc/profile
```

openEuler 20.03（鲲鹏）

1. openEuler 20.03操作系统。

```
umask 0022
echo umask 0022 >> /etc/profile
echo umask 0022 >> /etc/bashrc
source /etc/profile
```



```
unset i
unset -f pathmunge
fi

fi
# vim:ts=4:sw=4
umask 0022
"/etc/bashrc" 99L, 3012C
```

2. 验证是否返回0022。

```
umask
```

3.7 设置节点互信

Ambari Server与Ambari Agent服务器之间需要配置ssh无密访问，Ambari Server通过SSH公钥授权远程登录其他主机。

配置方法如下。

步骤1 产生ssh密钥。

1. 在集群每台主机产生SSH认证密钥对。

```
ssh-keygen -t rsa
```

2. 弹出对话框直接回车即可。

```
[root@server1 ~]# ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa.
Your public key has been saved in /root/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:FscQ5D3nOcVjXY7T8wdVo+R51tvnmbVXyhi5kNkqV8Y root@server1
The key's randomart image is:
+----[RSA 2048]-----+
|      .+ . .+=|
|      . + o.o*+|
|      o = .+O+=|
|      o O =o+=|
|      S + E .B|
|      . = * oO|
|      . o o o+o|
|      o      |
|      |      |
+-----[SHA256]-----+
```

步骤2 配置ssh免密登录。

● 方法一：

将Ambari Server生成的公钥文件id_rsa.pub拷贝至集群所有主机（包含Ambari Server本身）。

在Ambari Server主机执行：

```
cd ~/.ssh/
ssh-copy-id -i ~/.ssh/id_rsa.pub root@{target_host}
```

 说明

- {target_host}包括hadoop集群的所有主机，可以是主机名或IP地址。
- 该命令完成后会在{target_host}的~/.ssh/目录下生成文件authorized_keys。

● 方法二：

a. 在Ambari Server主机执行。

```
cd ~/.ssh
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

b. 将Ambari Server生成的公钥文件id_rsa.pub拷贝至集群其他Ambari Agent主机target_hosts。

```
scp ~/.ssh/id_rsa.pub root@{target_hosts}:~/
```

c. 在Agent主机分别执行。

```
cat ~/id_rsa.pub >> ~/.ssh/authorized_keys
```

步骤3 所有主机设置文件目录权限。

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

步骤4 Ambari Server主机分别免密码登录检查。

```
ssh {target.host}
```

----结束

3.8 配置 OS 镜像源

集群中每台服务器均需配置本地镜像源。

步骤1 进入存放OS镜像文件的目录，挂载镜像源。

```
mount YOUR_OS.iso /media/ -o loop
```

📖 说明

YOUR_OS.iso用实际的iso包名代替。例如：

- CentOS7.6(鲲鹏)
CentOS-7-aarch64-Everything-1810.iso
- CentOS7.6(x86)
CentOS-7-x86_64-DVD-1810.iso
- ...

步骤2 配置本地源。

- 备份repo。

```
cd /etc/yum.repos.d
mkdir -p /etc/yum.repos.d/bak
mv *.repo bak
```
- 创建并编辑local.repo文件。

```
vi local.repo
```

在local.repo文件中添加如下内容：

```
[local]
name=local repo
baseurl=file:///media
enabled=1
gpgcheck=0
```

步骤3 生效本地源。

```
yum clean all
yum makecache
```

----结束

3.9 安装依赖软件

CentOS7.6（鲲鹏/x86）：

安装"Infrastructure Server"和"Development Tools"。

```
yum groupinstall "Infrastructure Server" --setopt=group_package_types=mandatory,default,optional -y
yum groupinstall "Development Tools" --setopt=group_package_types=mandatory,default,optional -y
```

openEuler20.03（鲲鹏）：

安装"Server"和"Development Tools"。

```
yum groupinstall "Server" -y  
yum groupinstall "Development Tools" -y
```

3.10 安装 JDK

说明

集群内所有节点均需安装JDK，如果需要开启kerberos，还需jce安装包。

下载安装包（根据具体环境选择下载）

- x86环境OpenJDK下载地址：
https://github.com/AdoptOpenJDK/openjdk8-binaries/releases/download/jdk8u222-b10/OpenJDK8U-jdk_x64_linux_hotspot_8u222b10.tar.gz
- 鲲鹏环境OpenJDK下载地址：
https://github.com/AdoptOpenJDK/openjdk8-binaries/releases/download/jdk8u222-b10/OpenJDK8U-jdk_aarch64_linux_hotspot_8u222b10.tar.gz
- 鲲鹏环境毕昇JDK下载地址：
<https://kunpeng.huawei.com/#/developer/devkit/compiler>

说明

- 在[AdoptOpenJDK官网](#)下载最新版的对应平台的OpenJDK，推荐使用OpenJDK 8u272及以上版本或毕昇JDK。
- x86服务器以OpenJDK8U-jdk_x64_linux_hotspot_8u222b10.tar为例，TaiShan服务器以jdk-8u222-linux-aarch64.tar.gz为例。

安装 JDK

步骤1 下载OpenJDK。

```
wget https://github.com/AdoptOpenJDK/openjdk8-binaries/releases/download/jdk8u222-b10/  
OpenJDK8U-jdk_aarch64_linux_hotspot_8u222b10.tar.gz
```

步骤2 解压及修改权限。

```
tar -zxvf OpenJDK8U-jdk_aarch64_linux_hotspot_8u222b10.tar.gz  
chown -R root:root jdk8u222-b10/  
chmod -R 755 jdk8u222-b10/
```

步骤3 将“jdk8u222-b10/”目录移动到“/usr/local”目录下。

```
mv jdk8u222-b10/ /usr/local/
```

步骤4 打开环境变量的配置文件。

```
vi /etc/profile
```

步骤5 添加下面语句至文件末尾处。

```
export JAVA_HOME=/usr/local/jdk8u222-b10  
export PATH=$JAVA_HOME/bin:$PATH
```

步骤6 使环境变量生效。

```
source /etc/profile
```

----结束

说明

JAVA_HOME当前配置为系统默认配置的Java版本，请根据实际情况调整，并确认配置目录可以被所有用户访问到。

3.11 搭建 Yum 源服务器

须知

- 同一个集群中不可以同时存在鲲鹏CentOS 7.6系统和鲲鹏openEuler 20.03系统，只能在两者中选择其一。
- 鲲鹏镜像站中提供的二进制包是基于开源代码直接编译打包而成的，不含漏洞及BUG修复。
- 使用开源软件时需遵守开源软件的许可协议。

准备一台服务器，作为Yum源。搭建Yum源服务器需要做到以下几步：关闭防火墙、关闭selinux权限、安装Apache-Httpd服务和管理rpm包，部分操作参照[3.3 关闭防火墙](#)~[3.5 禁用交换分区](#)。

下载 rpm 包

- 鲲鹏（CentOS 7.6）rpm下载地址：
<https://mirrors.huaweicloud.com/kunpeng/yum/el/7/bigdata/HDP-UTILS-1.1.0.22/repos/HDP-UTILS-1.1.0.22-centos7.tar.gz>
<https://mirrors.huaweicloud.com/kunpeng/yum/el/7/bigdata/HDP-GPL/3.x/updates/3.1.0.0/HDP-GPL-3.1.0.0-centos7-gpl.tar.gz>
<https://mirrors.huaweicloud.com/kunpeng/yum/el/7/bigdata/ambari/2.x/updates/2.7.3.0/ambari-2.7.3.0-centos7.tar.gz>
<https://mirrors.huaweicloud.com/kunpeng/yum/el/7/bigdata/HDP/3.x/updates/3.1.0.0/HDP-3.1.0.0-centos7-rpm.tar.gz>
- 鲲鹏（openEuler 20.03）rpm下载地址：
<https://mirrors.huaweicloud.com/kunpeng/yum/el/7/bigdata/HDP-GPL/3.x/updates/3.1.0.0/HDP-GPL-3.1.0.0-centos7-gpl.tar.gz>
<https://mirrors.huaweicloud.com/kunpeng/yum/oe/20.03-LTS/bigdata/ambari/2.x/updates/2.7.3.0/ambari-2.7.3.0-openEuler20.03.tar.gz>
<https://mirrors.huaweicloud.com/kunpeng/yum/oe/20.03-LTS/bigdata/HDP/3.x/updates/3.1.0.0/HDP-3.1.0.0-openEuler20.03-rpm.tar.gz>

说明

请根据实际使用的操作系统选择其中一个下载，两者不可同时使用。

安装 httpd 组件

步骤1 安装httpd和createrepo。

```
yum install httpd createrepo
```

步骤2 开启httpd服务。

```
service httpd start
```

步骤3 验证。

```
systemctl status httpd.service
```

若状态为running，则安装并启动成功：

```
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; vendor preset: disabled)
   Active: active (running) since Mon 2019-10-14 16:22:09 CST; 18h ago
     Docs: man:httpd(8)
           man:apachectl(8)
  Main PID: 22995 (httpd)
   Status: "Total requests: 34; Current requests/sec: 0; Current traffic: 0 B/sec"
    CGroup: /system.slice/httpd.service
            └─22995 /usr/sbin/httpd -DFOREGROUND
            └─22996 /usr/sbin/httpd -DFOREGROUND
            └─22997 /usr/sbin/httpd -DFOREGROUND
            └─22998 /usr/sbin/httpd -DFOREGROUND
            └─22999 /usr/sbin/httpd -DFOREGROUND
            └─23000 /usr/sbin/httpd -DFOREGROUND
            └─23015 /usr/sbin/httpd -DFOREGROUND
            └─25332 /usr/sbin/httpd -DFOREGROUND
            └─46379 /usr/sbin/httpd -DFOREGROUND

Oct 14 16:22:09 DataNode-1 systemd[1]: Starting The Apache HTTP Server...
Oct 14 16:22:09 DataNode-1 httpd[22995]: AH00558: httpd: Could not reliably determine the server's fully
qualified domain name, using xxx.xxx.xxx.xxx. Set the 'ServerName' d...this message
Oct 14 16:22:09 DataNode-1 systemd[1]: Started The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
```

----结束

管理 rpm 包

说明

以鲲鹏（CentOS 7.6）和x86（CentOS 7.6）场景为例进行说明，仅供参考，其它场景请酌情调整。

- Ambari rpm包：

分别创建如下目录，管理ambari的rpm包。

```
mkdir -p /var/www/html/entry/ambari/2.7.3.0.0/centos7/aarch64
mkdir -p /var/www/html/entry/ambari/2.7.3.0.0/centos7/x86_64
mkdir -p /var/www/html/entry/ambari/2.7.3.0.0/centos7/noarch
```

将之前下载的rpm包进行分类，鲲鹏计算平台的包放到aarch64目录，x86计算平台的包放到x86_64目录，noarch目录下放公共的包。

```
└─ aarch64
   │   └─ ambari-agent-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-infra-manager-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-infra-solr-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-infra-solr-client-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-logsearch-logfeeder-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-logsearch-portal-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-metrics-collector-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-metrics-grafana-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-metrics-hadoop-sink-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-metrics-monitor-2.7.3.0-0.aarch64.rpm
   │   └─ ambari-server-2.7.3.0-0.aarch64.rpm
   └─ noarch
      └─ x86_64
         └─ ambari-agent-2.7.3.0-139.x86_64.rpm
         └─ ambari-infra-manager-2.7.3.0-1.x86_64.rpm
         └─ ambari-infra-solr-2.7.3.0-1.x86_64.rpm
```

```

|— ambari-infra-solr-client-2.7.3.0-1.x86_64.rpm
|— ambari-logsearch-logfeeder-2.7.3.0-1.x86_64.rpm
|— ambari-logsearch-portal-2.7.3.0-1.x86_64.rpm
|— ambari-metrics-collector-2.7.3.0-139.x86_64.rpm
|— ambari-metrics-grafana-2.7.3.0-139.x86_64.rpm
|— ambari-metrics-hadoop-sink-2.7.3.0-139.x86_64.rpm
|— ambari-metrics-monitor-2.7.3.0-139.x86_64.rpm
|— ambari-server-2.7.3.0-139.x86_64.rpm
|— smartsense-hst-1.5.1.2.7.3.0-139.x86_64.rpm

```

- HDP rpm包：

分别创建如下目录，管理hdp的rpm包。

```

mkdir -p /var/www/html/entry/hdp/3.1/centos7/aarch64
mkdir -p /var/www/html/entry/hdp/3.1/centos7/x86_64
mkdir -p /var/www/html/entry/hdp/3.1/centos7/noarch

```

其中Arm的包放到aarch64目录，x86的包放到x86_64目录，noarch目录下放公共的包：

```

|— aarch64
|   |— atlas-metadata_3_1_0_0_78-1.1.0.3.1.0.0-78.aarch64.rpm
|   |— bigtop-jsvc-1.0.15-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-client-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-conf-pseudo-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-hdfs-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-hdfs-datanode-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-hdfs-fuse-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-hdfs-journalnode-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-hdfs-namenode-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-hdfs-secondarynamenode-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-hdfs-zkfc-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-https-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-https-server-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-libhdfs-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-mapreduce-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-mapreduce-historyserver-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-source-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-yarn-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-yarn-nodemanager-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-yarn-proxyserver-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-yarn-registrydns-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-yarn-resourcemanager-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-yarn-timelinereader-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hadoop_3_1_0_0_78-yarn-timelineserver-3.1.1.3.1.0.0-78.aarch64.rpm
|   |— hbase_3_1_0_0_78-2.0.2.3.1.0.0-78.aarch64.rpm
|   |— hive_3_1_0_0_78-3.1.0.3.1.0.0-78.aarch64.rpm
|   |— hive_3_1_0_0_78-jdbc-3.1.0.3.1.0.0-78.aarch64.rpm
|   |— livy2_3_1_0_0_78-0.5.0.3.1.0.0-78.aarch64.rpm
|   |— phoenix_3_1_0_0_78-5.0.0.3.1.0.0-78.aarch64.rpm
|   |— ranger_3_1_0_0_78-hbase-plugin-1.2.0.3.1.0.0-78.aarch64.rpm
|   |— ranger_3_1_0_0_78-hdfs-plugin-1.2.0.3.1.0.0-78.aarch64.rpm
|   |— ranger_3_1_0_0_78-hive-plugin-1.2.0.3.1.0.0-78.aarch64.rpm
|   |— ranger_3_1_0_0_78-yarn-plugin-1.2.0.3.1.0.0-78.aarch64.rpm
|   |— spark2_3_1_0_0_78-2.3.2.3.1.0.0-78.aarch64.rpm
|   |— spark2_3_1_0_0_78-yarn-shuffle-2.3.2.3.1.0.0-78.aarch64.rpm
|   |— tez_3_1_0_0_78-0.9.1.3.1.0.0-78.aarch64.rpm
|
|— noarch
|   |— atlas-metadata_3_1_0_0_78-hbase-plugin-1.1.0.3.1.0.0-78.noarch.rpm
|   |— atlas-metadata_3_1_0_0_78-hive-plugin-1.1.0.3.1.0.0-78.noarch.rpm
|   |— hbase_3_1_0_0_78-doc-2.0.2.3.1.0.0-78.noarch.rpm
|   |— hbase_3_1_0_0_78-master-2.0.2.3.1.0.0-78.noarch.rpm
|   |— hbase_3_1_0_0_78-regionserver-2.0.2.3.1.0.0-78.noarch.rpm
|   |— hbase_3_1_0_0_78-rest-2.0.2.3.1.0.0-78.noarch.rpm
|   |— hbase_3_1_0_0_78-thrift-2.0.2.3.1.0.0-78.noarch.rpm
|   |— hbase_3_1_0_0_78-thrift2-2.0.2.3.1.0.0-78.noarch.rpm
|   |— hdp-select-3.1.0.0-78.el7.noarch.rpm
|   |— hive_3_1_0_0_78-hcatalog-3.1.0.3.1.0.0-78.noarch.rpm
|   |— hive_3_1_0_0_78-hcatalog-server-3.1.0.3.1.0.0-78.noarch.rpm

```

```

|—— hive_3_1_0_0_78-metastore-3.1.0.3.1.0.0-78.noarch.rpm
|—— hive_3_1_0_0_78-server2-3.1.0.3.1.0.0-78.noarch.rpm
|—— hive_3_1_0_0_78-server-3.1.0.3.1.0.0-78.noarch.rpm
|—— hive_3_1_0_0_78-webhcat-3.1.0.3.1.0.0-78.noarch.rpm
|—— hive_3_1_0_0_78-webhcat-server-3.1.0.3.1.0.0-78.noarch.rpm
|—— hive_warehouse_connector_3_1_0_0_78-1.0.0.3.1.0.0-78.noarch.rpm
|—— spark2_3_1_0_0_78-master-2.3.2.3.1.0.0-78.noarch.rpm
|—— spark2_3_1_0_0_78-python-2.3.2.3.1.0.0-78.noarch.rpm
|—— spark2_3_1_0_0_78-worker-2.3.2.3.1.0.0-78.noarch.rpm
|—— zookeeper_3_1_0_0_78-3.4.6.3.1.0.0-78.noarch.rpm
|—— zookeeper_3_1_0_0_78-server-3.4.6.3.1.0.0-78.noarch.rpm
|
|—— x86_64
|—— bigtop-jsvc-1.0.15-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-client-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-conf-pseudo-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-hdfs-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-hdfs-datanode-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-hdfs-fuse-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-hdfs-journalnode-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-hdfs-namenode-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-hdfs-secondarynamenode-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-hdfs-zkfc-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-https-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-https-server-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-libhdfs-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-mapreduce-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-mapreduce-historyserver-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-source-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-yarn-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-yarn-nodemanager-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-yarn-proxyserver-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-yarn-registrydns-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-yarn-resourcemanager-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-yarn-timelinereader-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hadoop_3_1_0_0_78-yarn-timelineserver-3.1.1.3.1.0.0-78.x86_64.rpm
|—— hbase_3_1_0_0_78-2.0.2.3.1.0.0-78.x86_64.rpm
|—— hdp-gpl
|—— hadoopplzo_3_1_0_0_78-0.6.0.3.1.0.0-78.x86_64.rpm
|—— hadoopplzo_3_1_0_0_78-native-0.6.0.3.1.0.0-78.x86_64.rpm
|—— hive_3_1_0_0_78-3.1.0.3.1.0.0-78.x86_64.rpm
|—— hive_3_1_0_0_78-jdbc-3.1.0.3.1.0.0-78.x86_64.rpm
|—— livy2_3_1_0_0_78-0.5.0.3.1.0.0-78.x86_64.rpm
|—— openblas
|—— openblas-0.2.19-4.el7.x86_64.rpm
|—— openblas-devel-0.2.19-4.el7.x86_64.rpm
|—— openblas-openmp-0.2.19-4.el7.x86_64.rpm
|—— openblas-openmp64_0.2.19-4.el7.x86_64.rpm
|—— openblas-openmp64-0.2.19-4.el7.x86_64.rpm
|—— openblas-Rblas-0.2.19-4.el7.x86_64.rpm
|—— openblas-serial64_0.2.19-4.el7.x86_64.rpm
|—— openblas-serial64-0.2.19-4.el7.x86_64.rpm
|—— openblas-static-0.2.19-4.el7.x86_64.rpm
|—— openblas-threads-0.2.19-4.el7.x86_64.rpm
|—— openblas-threads64_0.2.19-4.el7.x86_64.rpm
|—— openblas-threads64-0.2.19-4.el7.x86_64.rpm
|—— phoenix_3_1_0_0_78-5.0.0.3.1.0.0-78.x86_64.rpm
|—— ranger
|—— ranger_3_1_0_0_78-admin-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-atlas-plugin-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-hbase-plugin-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-hdfs-plugin-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-hive-plugin-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-kafka-plugin-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-kms-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-knox-plugin-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-solr-plugin-1.2.0.3.1.0.0-78.x86_64.rpm
|—— ranger_3_1_0_0_78-storm-plugin-1.2.0.3.1.0.0-78.x86_64.rpm

```



```
| | | ranger_3_1_0_0_78-tagsync-1.2.0.3.1.0.0-78.x86_64.rpm  
| | | ranger_3_1_0_0_78-usersync-1.2.0.3.1.0.0-78.x86_64.rpm  
| | | ranger_3_1_0_0_78-yarn-plugin-1.2.0.3.1.0.0-78.x86_64.rpm  
| | | ranger_3_1_0_0_78-hive-plugin-1.2.0.3.1.0.0-78.x86_64.rpm  
| | | snappy  
| | | | snappy-1.1.0-3.el7.x86_64.rpm  
| | | | snappy-devel-1.1.0-3.el7.x86_64.rpm  
| | | spark2_3_1_0_0_78-2.3.2.3.1.0.0-78.x86_64.rpm  
| | | spark2_3_1_0_0_78-yarn-shuffle-2.3.2.3.1.0.0-78.x86_64.rpm  
| | | tez_3_1_0_0_78-0.9.1.3.1.0.0-78.x86_64.rpm
```

创建 Yum 源仓库

步骤1 使用createrepo创建Yum源仓库。

```
cd /var/www/html/entry/ambari/2.7.3.0.0/centos7 && createrepo .  
cd /var/www/html/entry/hdp/3.1/centos7 && createrepo .
```

须知

- createrepo后需要加路径，上述命令最后的“.”，表示当前路径，不可缺少，否则会报错。
- 创建完成后将在上述两个目录分别生成repodata文件夹。

```
[root@Management-1 centos7]# ll  
total 12  
drwxr-xr-x 2 root root 4096 Oct 15 16:34 aarch64  
drwxr-xr-x 2 root root 6 Oct 15 16:34 noarch  
drwxr-xr-x 2 root root 4096 Oct 15 19:01 repodata  
drwxr-xr-x 2 root root 4096 Oct 15 16:37 x86_64
```

步骤2 配置repo文件。

说明

该步骤需在集群中每台服务器上执行。

1. 创建并编辑ambari.repo文件。

```
cd /etc/yum.repos.d  
vi ambari.repo
```

在ambari.repo文件中添加如下内容：

```
[ambari]  
name=ambari repo  
baseurl=http://{Yum源服务器ip}/entry/ambari/2.7.3.0.0/centos7  
enabled=1  
gpgcheck=0
```

2. 创建并编辑hdp.repo文件。

```
vi hdp.repo
```

在hdp.repo文件中添加如下内容：

```
[hdp]  
name=hdp repo  
baseurl=http://{Yum源服务器ip}/entry/hdp/3.1/centos7  
enabled=1  
gpgcheck=0
```

说明

baseurl可以在浏览器中验证是否能够正常访问。

步骤3 生效源地址。

```
yum clean all  
yum makecache
```

须知

- 文档中有将rpm包分别管理，如Taishan的包放到aarch64目录，noarch目录下放公共的包，在实际部署时，将RPM包统一放在“/var/www/html/entry/ambari/2.7.3.0.0/centos7/aarch64”目录。
- 在“/etc/yum.repos.d”中创建ambari.repo时，需要将url路径写到上一层目录，即：“baseurl=http://ip地址/entry/ambari/2.7.3.0.0/centos7/”。
- 注意baseurl字符均为小写，否则生效Yum源时会报错。

----结束

3.12 操作系统时钟设置

集群中所有节点主机的时间必须设置同步才能访问Ambari的Web UI界面。

步骤1 安装NTP服务。

1. 检查是否安装NTP服务。

```
rpm -qa|grep ntp
```

```
[hdfs@server1 ~]$ rpm -qa | grep ntp  
fontpackages-filesystem-1.44-8.el7.noarch  
ntpdate-4.2.6p5-28.el7.centos.aarch64  
ntp-4.2.6p5-28.el7.centos.aarch64
```

2. 安装 NTP 服务。

```
yum install ntp -y
```

步骤2 检测时间同步是否开机自启。

在集群内所有节点检查是否设置时钟同步开机自启。

```
systemctl is-enabled ntpd
```

```
[hdfs@server1 ~]$ systemctl is-enabled ntpd  
enabled
```

- 若已设置，则进入[步骤3](#)。
- 若未设置，则设置开机自启。

```
systemctl enable ntpd
```

步骤3 （可选）配置 NTP server。

1. 在集群中的其中一台服务器配置NTP server。

```
vi /etc/ntp.conf
```

📖 说明

有外部时钟源则跳过此操作。

2. 参照下面内容进行设置。

```
# Hosts on local network are less restricted.  
restrict IP address1 mask IP address2 nomodify notrap  
#允许内网其他机器同步时间，IP address1为当前网关，IP address2为子网掩码
```

```
# Use public servers from the pool.ntp.org project.
# Please consider joining the pool (http://www.pool.ntp.org/join.html).
#server 0.centos.pool.ntp.org iburst      #注释掉原时钟服务器
#server 1.centos.pool.ntp.org iburst
#server 2.centos.pool.ntp.org iburst
#server 3.centos.pool.ntp.org iburst

server IP address      #设置本机为时钟服务器
fudge IP address stratum 10
```

3. 设置时间为当前时间。

例：

```
date -s '20191010 13:49'
```

4. 重启ntp服务。

```
systemctl restart ntpd.service
```

步骤4 配置ntp client。

1. 在其他服务器中配置。

```
vi /etc/ntp.conf
```

添加如下内容：

```
# Hosts on local network are less restricted.
restrict IP address1 mask IP address2 nomodify notrap

# Use public servers from the pool.ntp.org project.
# Please consider joining the pool (http://www.pool.ntp.org/join.html).
#server 0.centos.pool.ntp.org iburst
#server 1.centos.pool.ntp.org iburst
#server 2.centos.pool.ntp.org iburst
#server 3.centos.pool.ntp.org iburst
server IP address prefer      #设置为ntpservers的ip
fudge IP address stratum 10
```

2. 重启ntp服务。

```
systemctl restart ntpd.service
```

步骤5 5分钟后验证ntp时钟同步。

```
ntpstat
```

ntp server状态如下：

```
synchronised to local net at stratum 6
time correct to within 12 ms
polling server every 64 s
```

ntp client状态如下：

```
synchronised to NTP server (IP address) at stratum 7
time correct to within 41 ms
polling server every 1024 s
```

----结束

3.13 (可选)创建普通用户

- 步骤1 主机server1创建普通用户，用于安装ambari-server及日常维护。

```
groupadd bigdata
useradd -g bigdata -d /home/bigdata bigdata
```

- 步骤2 修改bigdata密码。

```
echo "bigdata@123" | passwd --stdin bigdata
```

步骤3 给bigdata用户设置root权限。

```
cp /etc/sudoers /etc/sudoers_bak  
echo "bigdata ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers
```

----结束

3.14 关闭 dnsmasq（仅 openEuler 系统需操作）

说明

在openEuler系统中dnsmasq默认启动并占用53端口，与YARN REGISTRY DNS组件使用的端口冲突，故需要在安装yarn之前将该端口释放出来，或者修改YARN REGISTRY DNS组件使用的端口。此处以关闭dnsmasq为例说明。

步骤1 获取dnsmasq进程ID。

```
lsof -i:53
```

```
[root@server1 home]# lsof -i:53  
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME  
dnsmasq 6945 dnsmasq 5u IPv4 157830 0t0 UDP server1:domain  
dnsmasq 6945 dnsmasq 6u IPv4 157831 0t0 TCP server1:domain (LISTEN)
```

步骤2 关闭dnsmasq。

```
kill -9 DNS_PID
```

说明

命令中DNS_PID需替换为步骤1中查询出来的PID，以步骤1查询结果为例：

```
kill -9 6945
```

步骤3 禁用dnsmasq。

1. 检查dnsmasq服务是否已经安装。

```
systemctl status dnsmasq.service
```

从返回结果看，如果如下图所示，则当前dnsmasq为启用状态，则执行步骤3.2。

```
[root@agent3 ~]# systemctl status dnsmasq.service  
● dnsmasq.service - DNS caching server.  
   Loaded: loaded (/usr/lib/systemd/system/dnsmasq.service; enabled; vendor preset: disabled)  
   Active: inactive (dead)
```

如果提示如下图所示，为disabled状态，则当前dnsmasq已经被禁用。

```
[root@agent3 ~]# systemctl status dnsmasq.service  
● dnsmasq.service - DNS caching server.  
   Loaded: loaded (/usr/lib/systemd/system/dnsmasq.service; disabled; vendor preset: disabled)  
   Active: inactive (dead)
```

2. 禁用dnsmasq。

```
systemctl disable dnsmasq.service
```

----结束

4 部署 Ambari

4.1 安装数据库

4.2 部署Ambari-server

4.1 安装数据库

步骤1 安装数据库服务。

📖 说明

在集群中server服务器上执行即可。

1. 安装mariadb及mysql相关服务。

```
yum install mariadb mariadb-server mysql-connector-java.noarch -y
```

⚠️ 警告

1. 对于openEuler系统暂未提供mysql-connector-java.noarch安装包，故需如下安装：

```
yum install mariadb mariadb-server -y
```

2. 下载mysql-connector-java.jar，并放置到“/usr/share/java”目录下（jar包版本可酌情调整）：

```
mkdir -p /usr/share/java  
wget https://repo1.maven.org/maven2/mysql/mysql-connector-java/5.1.49/mysql-connector-java-5.1.49.jar -O /usr/share/java/mysql-connector-java.jar
```

2. 启动并设置mariadb为开机启动。

```
systemctl start mariadb  
systemctl enable mariadb
```

3. 启动状态查询。

```
systemctl status mariadb.service
```

状态为running则启动成功：

```
[root@server1 etc]# systemctl status mariadb.service  
● mariadb.service - MariaDB database server  
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; vendor preset: disabled)  
   Active: active (running) since Wed 2019-10-16 17:21:14 CST; 4 days ago  
     Main PID: 6191 (mysqld_safe)  
    CGroup: /system.slice/mariadb.service  
            └─6191 /bin/sh /usr/bin/mysqld_safe --basedir=/usr
```

```
6353 /usr/libexec/mysqld --basedir=/usr --datadir=/var/lib/mysql --plugin-dir=/usr/lib64/  
mysql/plugin --log-error=/var/log/mariadb/mariadb.log --pid-file=/var/run/mariadb/ma...
```

Warning: Journal has been rotated since unit was started. Log output is incomplete or unavailable.



警告

openEuler中查看mariadb状态时可能会提示“Plugin 'gssapi' init function returned error”，此时可修改“/etc/my.cnf.d/auth_gssapi.cnf”文件关闭gssapi认证，之后重启mysql即可。

```
[root@agent1 java]# systemctl status mariadb.service
● mariadb.service - MariaDB 10.3.9 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; vendor preset: disabled)
   Active: active (running) since Thu 2020-09-17 14:19:54 CST; 7s ago
     Docs: man:mysqld(8)
           https://mariadb.com/kb/en/library/systemd/
   Main PID: 98204 (mysqld)
     Status: "Taking your SQL requests now..."
       Tasks: 30
      Memory: 95.2M
      Corrupt: /system.slice/mariadb.service
             -98204 /usr/libexec/mysqld

Sep 17 14:19:54 agent1 mysqld[98204]: 2020-09-17 14:19:54 0 [Warning] mysqld: GSSAPI plugin-- default principal '-mariadb/agent1@-red.found-in-boydb
Sep 17 14:19:54 agent1 mysqld[98204]: 2020-09-17 14:19:54 0 [ERROR] mysqld: Server GSSAPI error (major 851968, minor 2529639093) : gss_acquire_cred failed -Unspecified GSS failure. Minor code may prov
Sep 17 14:19:54 agent1 mysqld[98204]: 2020-09-17 14:19:54 0 [ERROR] Plugin 'gssapi' init function returned error.
Sep 17 14:19:54 agent1 mysqld[98204]: 2020-09-17 14:19:54 0 [Note] Server socket created on IP: '*'
Sep 17 14:19:54 agent1 mysqld[98204]: 2020-09-17 14:19:54 0 [Note] Reading of all Master_info entries succeeded
Sep 17 14:19:54 agent1 mysqld[98204]: 2020-09-17 14:19:54 0 [Note] Added new Master_info: '*' to hash table
Sep 17 14:19:54 agent1 mysqld[98204]: 2020-09-17 14:19:54 0 [Note] /usr/libexec/mysqld: ready for connections.
Sep 17 14:19:54 agent1 mysqld[98204]: Version: '10.3.9-MariaDB'  socket: '/var/lib/mysql/mysql.sock'  port: 3306  MariaDB Server
Sep 17 14:19:54 agent1 systemd[1]: Started MariaDB 10.3.9 database server.

ESC00
● mariadb.service - MariaDB 10.3.9 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; vendor preset: disabled)
   Active: active (running) since Thu 2020-09-17 14:19:54 CST; 7s ago
     Docs: man:mysqld(8)
           https://mariadb.com/kb/en/library/systemd/
   Main PID: 98204 (mysqld)
     Status: "Taking your SQL requests now..."
       Tasks: 30
```

操作如下：

1. 修改“/etc/my.cnf.d/auth_gssapi.cnf”文件。

```
vim /etc/my.cnf.d/auth_gssapi.cnf
```

2. 重启mysql。

```
systemctl restart mariadb.service
```

4. 登录并修改默认密码。

```
mysql -uroot -p
```

- a. 默认没有密码，按回车即可登录。

```
[root@server1 yum.repos.d]# mysql -uroot -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 2
Server version: 5.5.60-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

- b. 设置密码。

```
set password = password('密码');
flush privileges;

MariaDB [(none)]> set password = password('root');
Query OK, 0 rows affected (0.00 sec)

MariaDB [(none)]> flush privileges;
Query OK, 0 rows affected (0.00 sec)
```

须知

flush privileges必须执行，否则设置不生效。

5. 验证密码。

退出后重新登录，查看密码是否修改成功。

```
MariaDB [(none)]> exit;
Bye
[root@server1 yum.repos.d]# mysql -uroot -p
Enter password:root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 5.5.60-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

步骤2 创建Ambari数据库并配置权限。

```
MariaDB [(none)]> create database ambari;
Query OK, 1 row affected (0.00 sec)

MariaDB [(none)]> use ambari;
Database changed
MariaDB [ambari]> CREATE USER 'ambari'@'%' IDENTIFIED BY 'ambari123';
Query OK, 0 rows affected (0.00 sec)

MariaDB [ambari]> GRANT ALL PRIVILEGES on *.* To 'ambari'@'%' IDENTIFIED BY 'ambari123';
Query OK, 0 rows affected (0.00 sec)

MariaDB [ambari]> CREATE USER 'ambari'@'server1' IDENTIFIED BY 'ambari123';
Query OK, 0 rows affected (0.01 sec)

MariaDB [ambari]> GRANT ALL PRIVILEGES on *.* To 'ambari'@'server1' IDENTIFIED BY 'ambari123';
Query OK, 0 rows affected (0.00 sec)

MariaDB [ambari]> CREATE USER 'ambari'@'agent1' IDENTIFIED BY 'ambari123';
Query OK, 0 rows affected (0.00 sec)

MariaDB [ambari]> GRANT ALL PRIVILEGES on *.* To 'ambari'@'agent1' IDENTIFIED BY 'ambari123';
Query OK, 0 rows affected (0.00 sec)

MariaDB [ambari]> CREATE USER 'ambari'@'agent2' IDENTIFIED BY 'ambari123';
Query OK, 0 rows affected (0.00 sec)

MariaDB [ambari]> GRANT ALL PRIVILEGES on *.* To 'ambari'@'agent2' IDENTIFIED BY 'ambari123';
Query OK, 0 rows affected (0.00 sec)

MariaDB [ambari]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)
```

 说明

根据实际情况进行修改，需要对集群中每一台服务器创建用户。

----结束

4.2 部署 Ambari-server

操作可选用bigdata用户执行。

```
su bigdata
```

下载安装 Ambari-server

- root用户执行：

```
yum install ambari-server.aarch64 -y
```

- bigdata用户执行：
`sudo yum install ambari-server.aarch64 -y`

初始化 Ambari 数据库连接配置

步骤1 在集群中所有服务器上切换回root用户添加环境变量。

```
vim /etc/profile
```

添加内容如下：

```
export buildNumber=2.7.3.0.0
```

步骤2 生效环境变量。

```
source /etc/profile
```

步骤3 初始化Ambari数据库连接配置。

```
ambari-server setup --jdbc-db=mysql --jdbc-driver=/usr/share/java/mysql-connector-java.jar
```

bigdata用户执行：

```
sudo ambari-server setup --jdbc-db=mysql --jdbc-driver=/usr/share/java/mysql-connector-java.jar
Using python /usr/bin/python
Setup ambari-server
Copying /usr/share/java/mysql-connector-java.jar to /var/lib/ambari-server/resources/mysql-connector-
java.jar
If you are updating existing jdbc driver jar for mysql with mysql-connector-java.jar. Please remove the old
driver jar, from all hosts. Restarting services that need the driver, will automatically copy the new jar to the
hosts.
JDBC driver was successfully initialized.
Ambari Server 'setup' completed successfully.
```

----结束

Ambari-server 安装配置

步骤1 安装Ambari-server。

```
ambari-server setup
```

bigdata用户执行：

```
sudo ambari-server setup
Using python /usr/bin/python
Setup ambari-server
Checking SELinux...
SELinux status is 'enabled'
SELinux mode is 'permissive'
WARNING: SELinux is set to 'permissive' mode and temporarily disabled.
OK to continue [y/n] (y)? y
Customize user account for ambari-server daemon [y/n] (n)? n
Enter user account for ambari-server daemon (root):bigdata
Adjusting ambari-server permissions and ownership...
Checking firewall status...
Checking JDK...
[1] Oracle JDK 1.8 + Java Cryptography Extension (JCE) Policy Files 8
[2] Custom JDK
=====
Enter choice (1): 2
WARNING: JDK must be installed on all hosts and JAVA_HOME must be valid on all hosts.
WARNING: JCE Policy files are required for configuring Kerberos security. If you plan to use Kerberos,please
make sure JCE Unlimited Strength Jurisdiction Policy Files are valid on all hosts.
Path to JAVA_HOME: /usr/local/jdk8u222-b10
Validating JDK on Ambari Server...done.
Check JDK version for Ambari Server...
```



```
JDK version found: 8
Minimum JDK version is 8 for Ambari. Skipping to setup different JDK for Ambari Server.
```

📖 说明

- 若使用普通用户安装Ambari-server，参数“Customize user account for ambari-server daemon [y/n] (n)?”选择“y”，参数“Enter user account for ambari-server daemon (root)”则输入建立的普通用户。
- 参数“Path to JAVA_HOME”根据安装JDK的实际路径填写。

步骤2 选择Custom JDK，路径以上面步骤中实际的路径为准。

```
Checking GPL software agreement...
GPL License for LZO: https://www.gnu.org/licenses/old-licenses/gpl-2.0.en.html
Enable Ambari Server to download and install GPL Licensed LZO packages [y/n] (n)? y
Completing setup...
Configuring database...
Enter advanced database configuration [y/n] (n)? y
Configuring database...
=====
Choose one of the following options:
[1] - PostgreSQL (Embedded)
[2] - Oracle
[3] - MySQL / MariaDB
[4] - PostgreSQL
[5] - Microsoft SQL Server (Tech Preview)
[6] - SQL Anywhere
[7] - BDB
=====
Enter choice (1):3
Hostname (localhost): server1
Port (3306):
Database name (ambari):
Username (ambari):
Enter Database Password (bigdata):ambari123
Re-enter password:
Configuring ambari database...
```

📖 说明

参数Enter Database Password（bigdata）输入[4.1 安装数据库](#)中设置的密码。

步骤3 根据实际情况输入后回车。

```
Should ambari use existing default jdbc /usr/share/java/mysql-connector-java.jar [y/n] (y)? y
Configuring remote database connection properties...
WARNING: Before starting Ambari Server, you must run the following DDL directly from the database shell
to create the schema: /var/lib/ambari-server/resources/Ambari-DDL-MySQL-CREATE.sql
Proceed with configuring remote database connection properties [y/n] (y)? y
Extracting system views...
ambari-admin-2.7.3.0.0.jar

Ambari repo file doesn't contain latest json url, skipping repoinfos modification
Adjusting ambari-server permissions and ownership...
Ambari Server 'setup' completed successfully.
```

----结束

初始化 Ambari 数据库

步骤1 登录mysql。

```
mysql -uroot -p
MariaDB [(none)]> use ambari;
Database changed
MariaDB [ambari]> source /var/lib/ambari-server/resources/Ambari-DDL-MySQL-CREATE.sql;
```

步骤2 验证脚本是否初始化成功，出现以下table列表：

```
MariaDB [ambari]> show tables;
+-----+
| Tables_in_ambari |
+-----+
| ClusterHostMapping |
| QRTZ_BLOB_TRIGGERS |
| QRTZ_CAENDARS      |
| QRTZ_CRON_TRIGGERS |
| QRTZ_FIRED_TRIGGERS |
| QRTZ_JOB_DETAILS   |
| QRTZ_LOCKS         |
| QRTZ_PAUSED_TRIGGER_GRPS |
| QRTZ_SCHEDULER_STATE |
| QRTZ_SIMPLE_TRIGGERS |
| QRTZ_SIMPROP_TRIGGERS |
| QRTZ_TRIGGERS      |
| adminpermission    |
+-----+
MariaDB [(none)]> exit;
Bye
```

----结束

启动 Ambari 服务

- 启动Ambari服务。

ambari-server start

bigdata用户执行：

sudo ambari-server start

```
[root@server1 yum.repos.d]# ambari-server start
Using python /usr/bin/python
Starting ambari-server
Ambari Server running with administrator privileges.
Organizing resource files at /var/lib/ambari-server/resources...
Ambari database consistency check started...
Server PID at: /var/run/ambari-server/ambari-server.pid
Server out at: /var/log/ambari-server/ambari-server.out
Server log at: /var/log/ambari-server/ambari-server.log
Waiting for server start.....
Server started listening on 8080

DB configs consistency check: no errors and warnings were found.
Ambari Server 'start' completed successfully.
```

（可选）Ambari 相关命令

- 查看Ambari server状态。

ambari-server status

```
[root@Namenode1 centos7]# ambari-server status
Using python /usr/bin/python
Ambari-server status
Ambari Server running
Found Ambari Server PID: 18130 at: /var/run/ambari-server/ambari-server.pid
```

- 停止Ambari server服务。

ambari-server stop

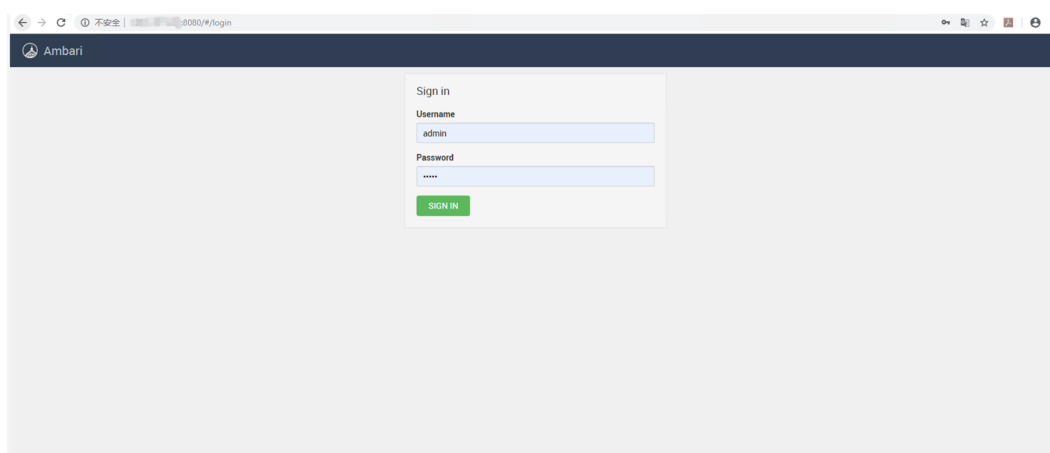
5 部署 Hadoop 集群

登录 Ambari 界面

启动ambari-server后，通过客户端浏览器登录Ambari的界面。

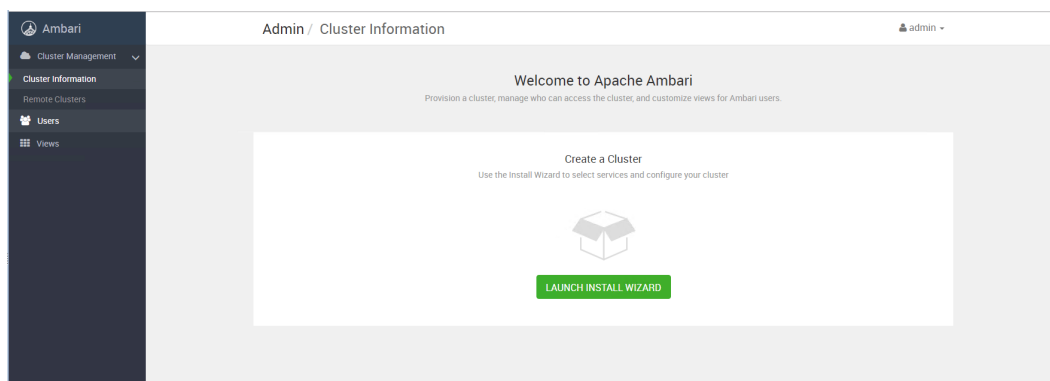
浏览器网址：http://<your.ambari.server>:8080。

默认登录用户名/密码为：admin/admin。



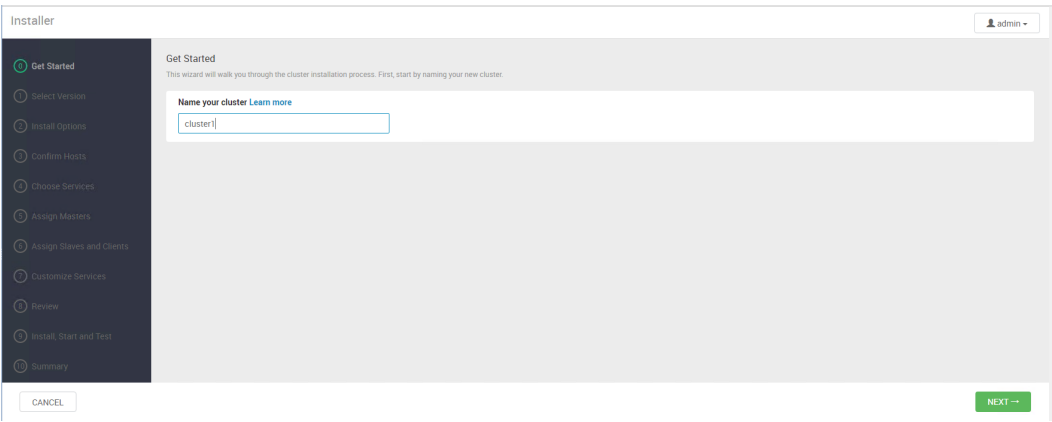
启动 Ambari 部署向导

步骤1 单击“Launch Install Wizard”搭建集群。



步骤2 输入集群的名字，不能使用空格和其他特殊字符。

例如：“cluster1”。

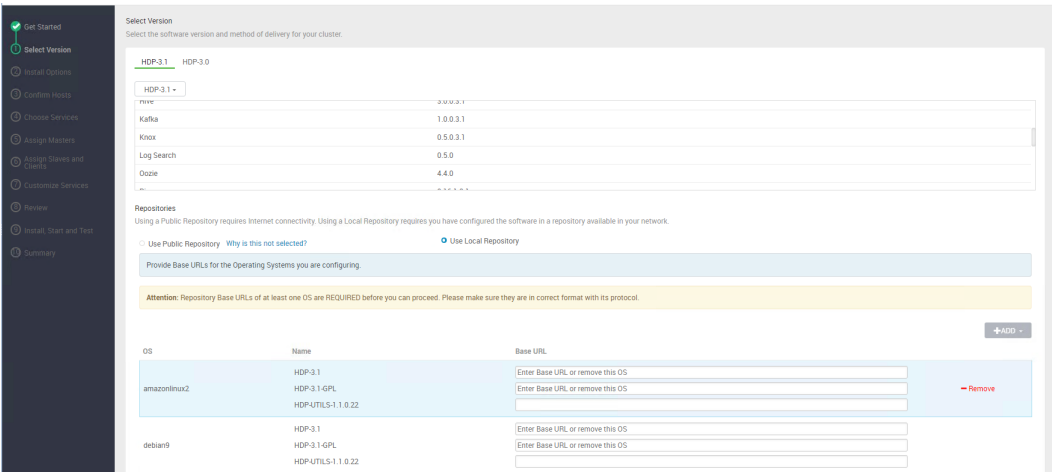


步骤3 单击“下一步”，进入选择版本。

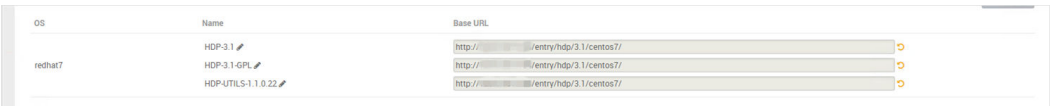
----结束

选择版本

步骤1 将除了redhat7之外的os选项通过右边的remove删除。



步骤2 在redhat7对应的Base URL中填入hdp.repo文件中的地址，这里为http://{Yum源服务器ip}/entry/hdp/3.1/centos7。

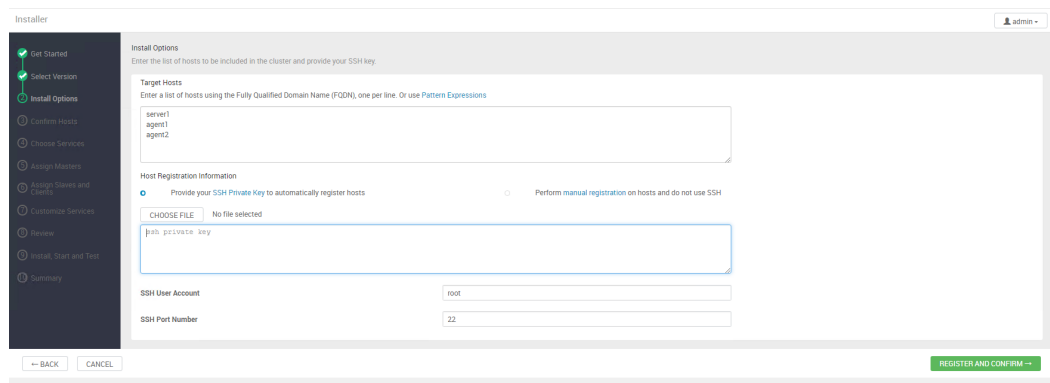


步骤3 单击“下一步”，进入安装选项。

----结束

安装选项

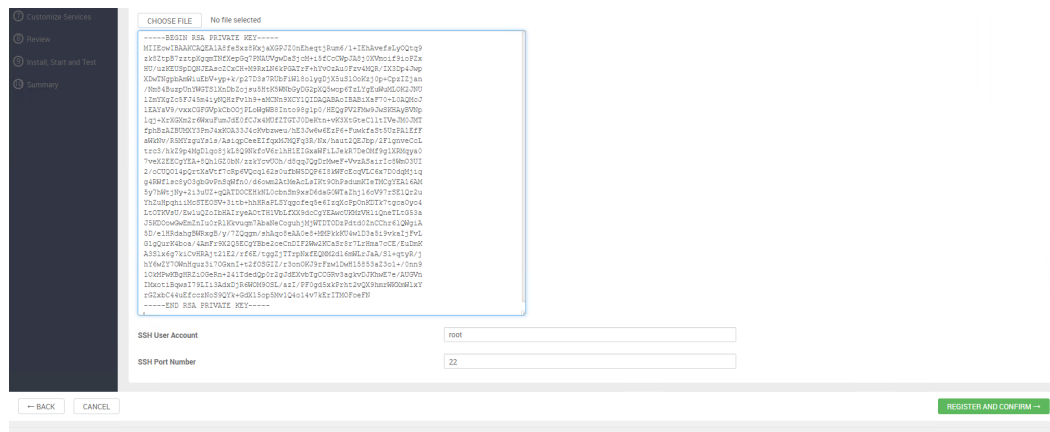
步骤1 在目标主机中填入集群中所有服务器的主机名。



步骤2 填入server主机的ssh密钥。

通过命令获取。

```
cat ~/.ssh/id_rsa
```



警告

对于openEuler系统需要在末尾增加一个空行，即"-----END OPENSSSH PRIVATE KEY-----"行后增加空行。

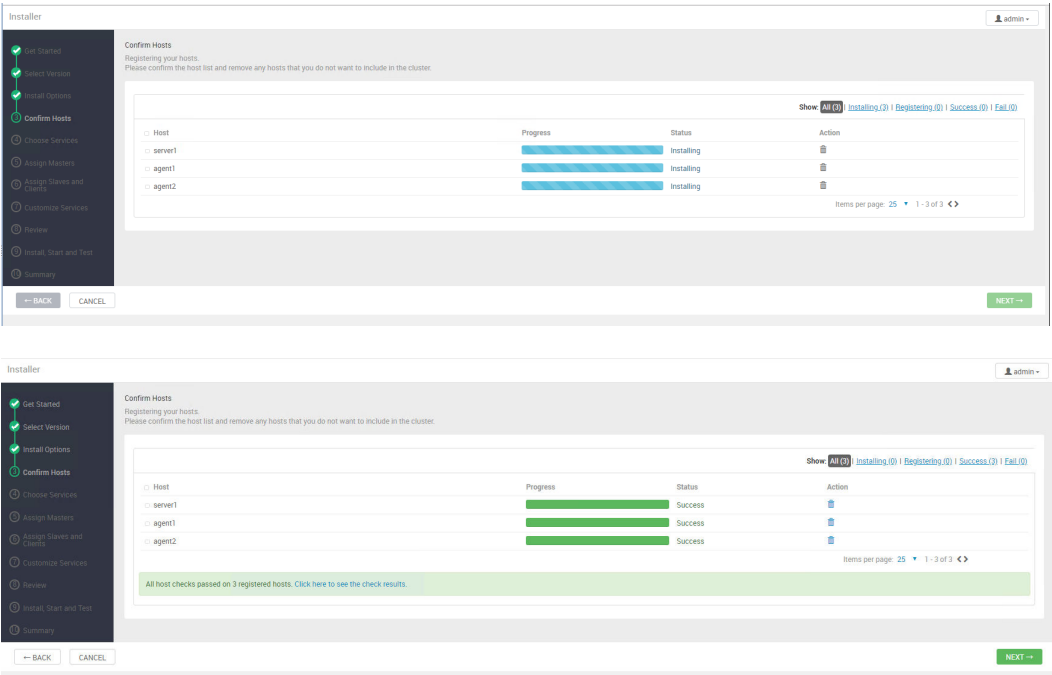
```
...  
-----END OPENSSSH PRIVATE KEY-----
```

步骤3 单击“REGISTER AND CONFIRM”，进入确认主机。

----结束

确认主机

步骤1 安装ambari-agent，并检测主机，保证检查结果没问题。



步骤2 单击“下一步”，进入部署基础服务。
----结束

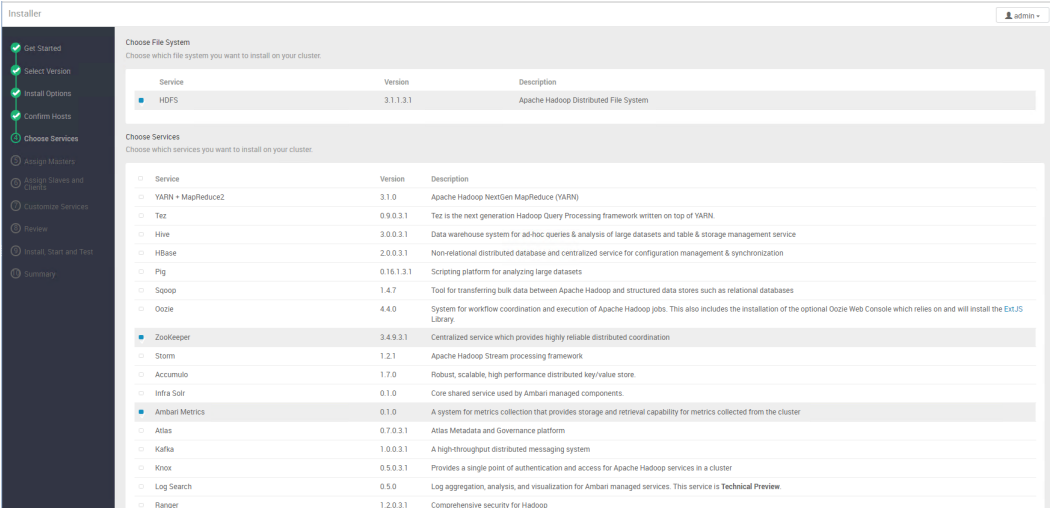
部署基础服务

说明

没有特别说明直接单击下一步即可。

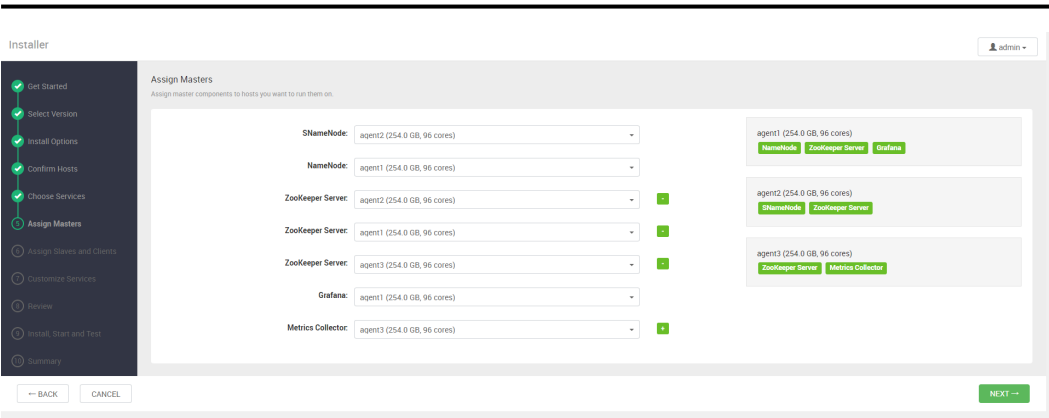
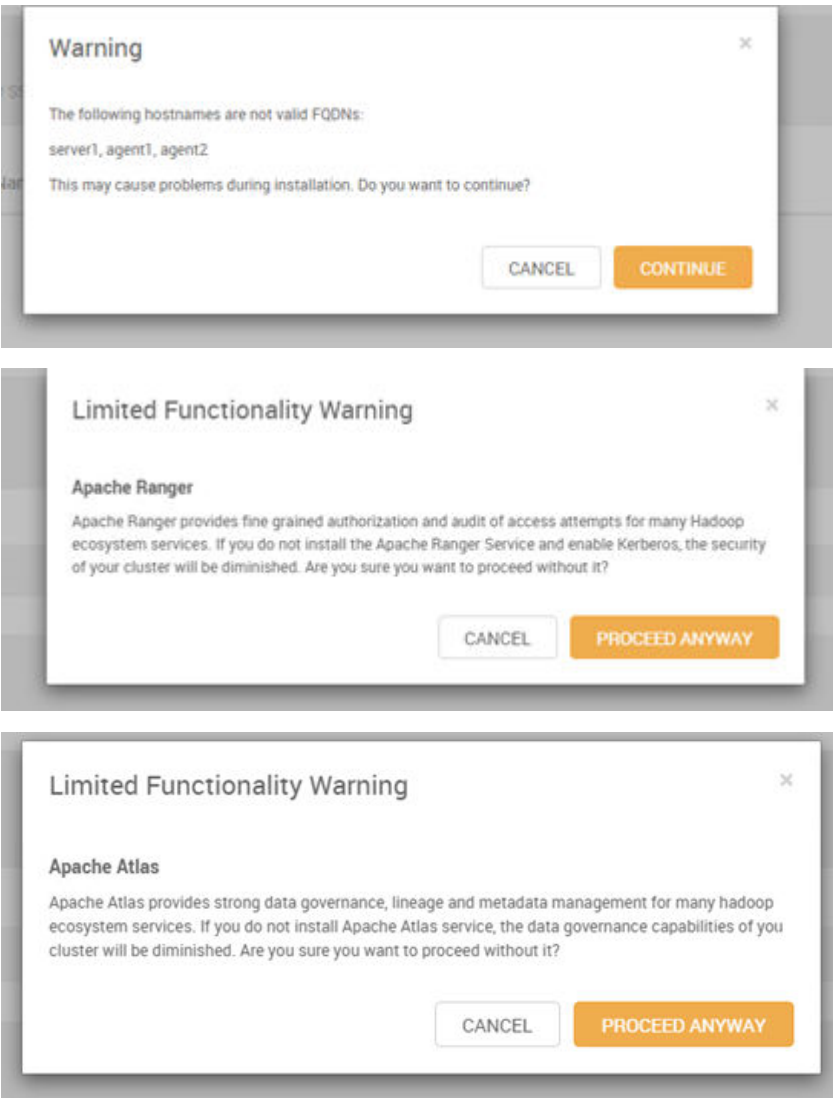
步骤1 选择服务，单击“下一步”。

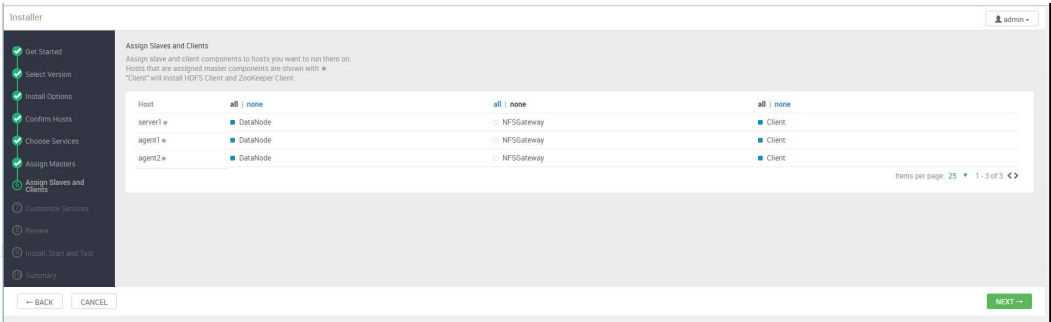
步骤2 先安装HDFS、ZooKeeper、Ambari Metrics。



须知

有可能会出现以下几种情况，都选择continue或proceed anyway。

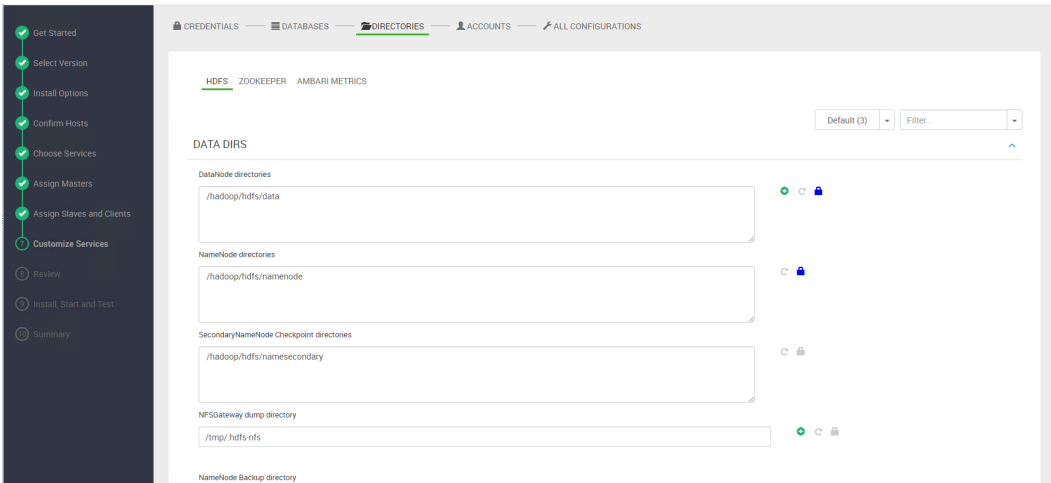
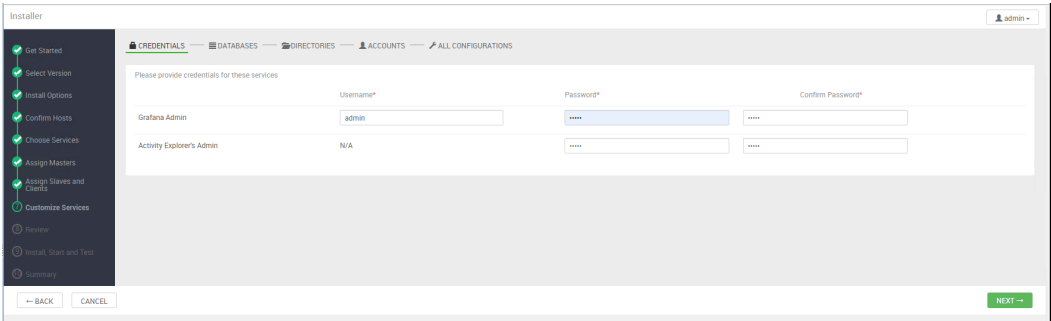


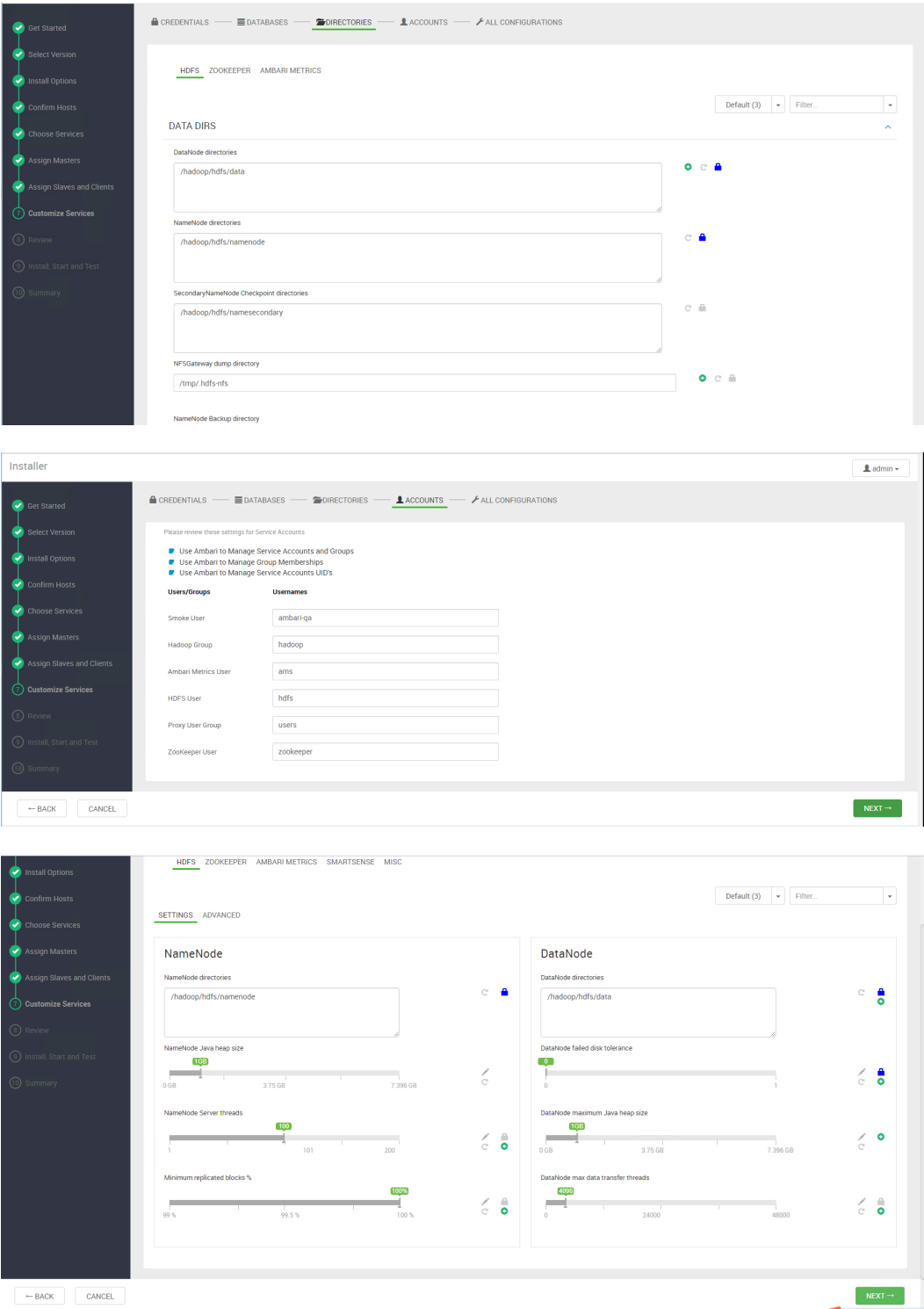


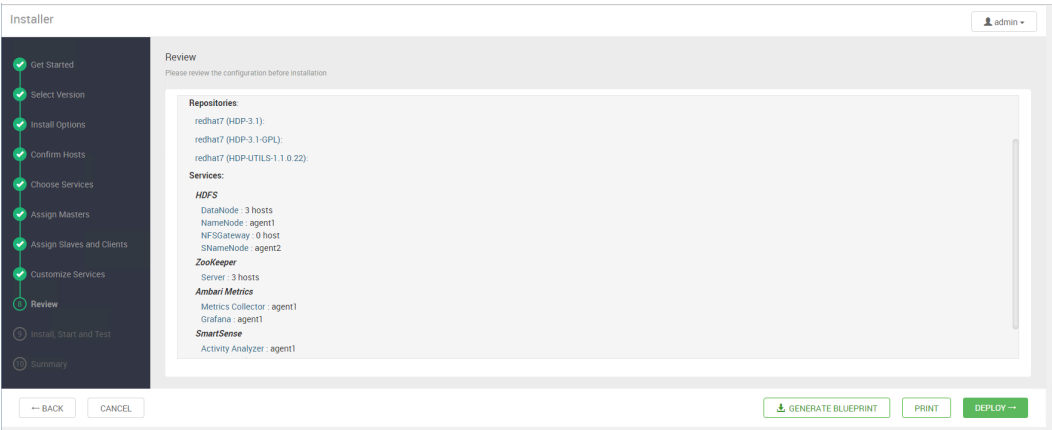
须知

- 1. Datanode directories只能配置数据盘对应的目录，配置系统盘目录容易将系统盘写满，导致服务挂起。
- 2. NameNode directories配置的目录数量控制在2-3个，可以配置为系统盘较大的目录。
- 3. 格式化数据盘和挂盘命令参考如下：
`parted /dev/sdb mklabel gpt`
`parted /dev/sdb mkpart logical 0% 100%`
`mkfs.xfs -f /dev/sdb1`
`mkdir -p /hadoop/data1`
`mount /dev/sdb1 /hadoop/data1`

步骤3 输入集群的密码，此处都为默认密码。

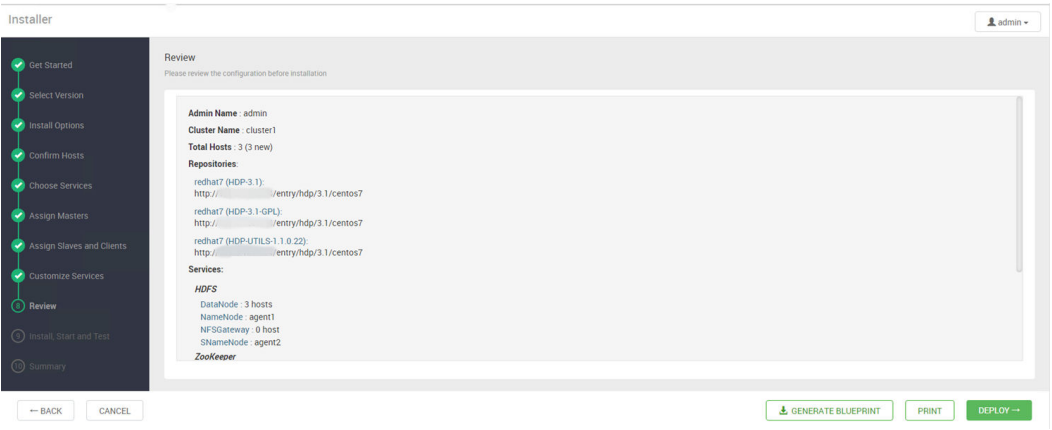




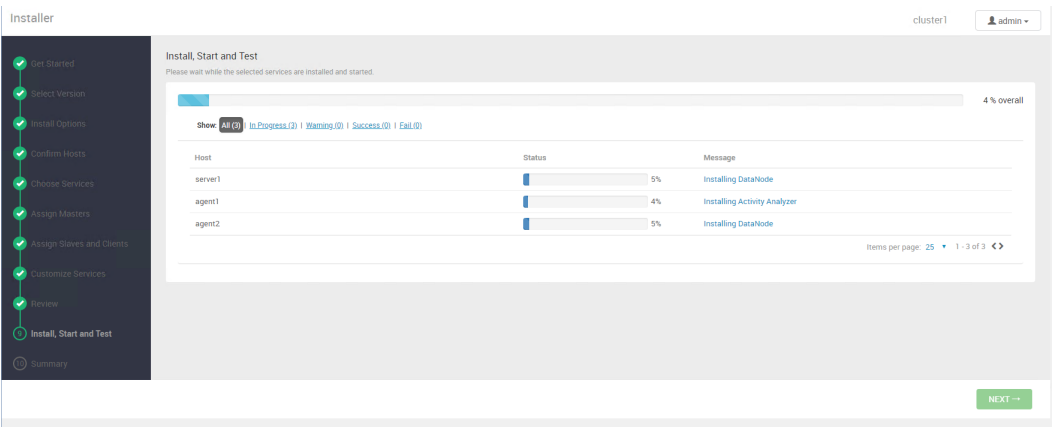


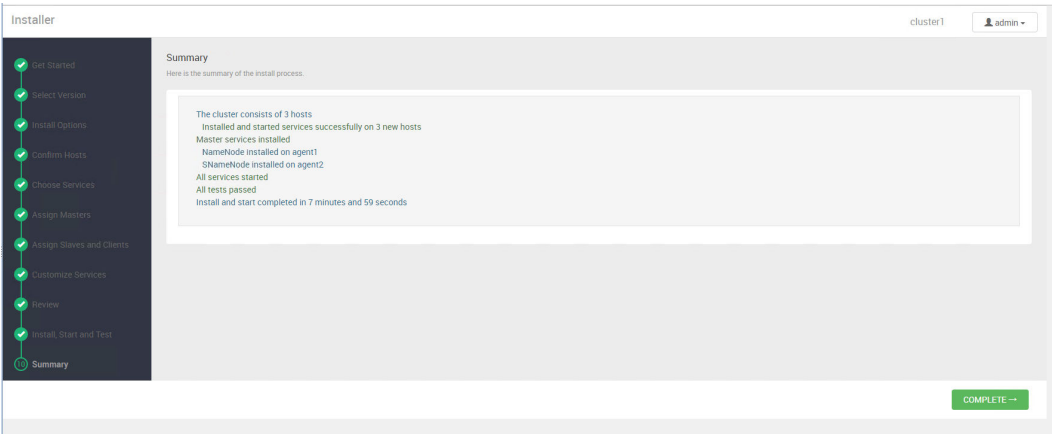
须知

当前页面时需要刷新一下浏览器页面，使当前界面中Repositories中有url地址。



步骤4 等待安装完成。



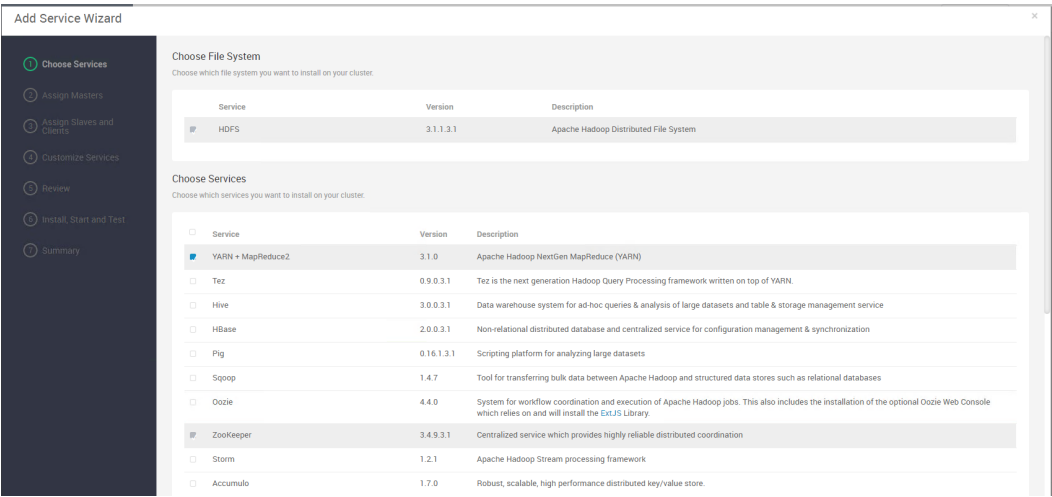
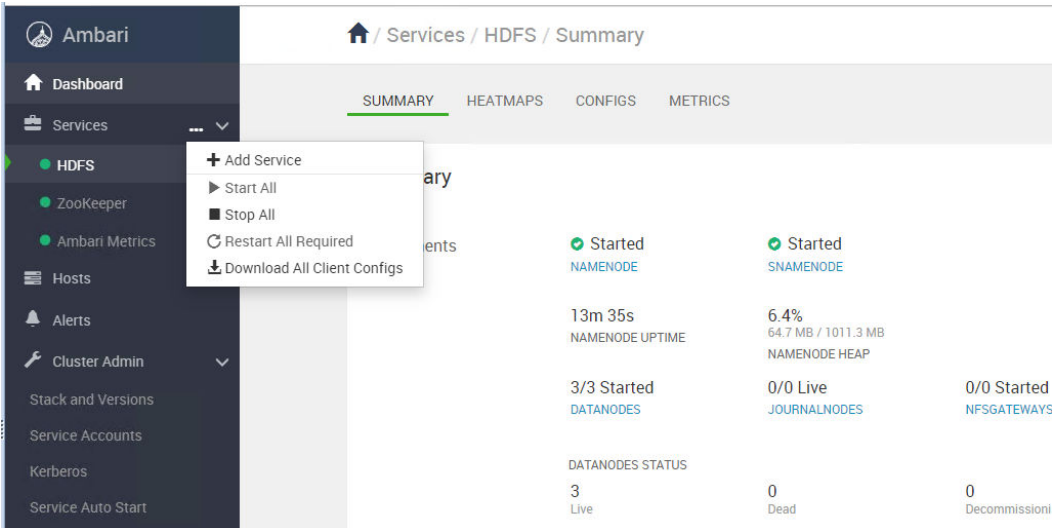


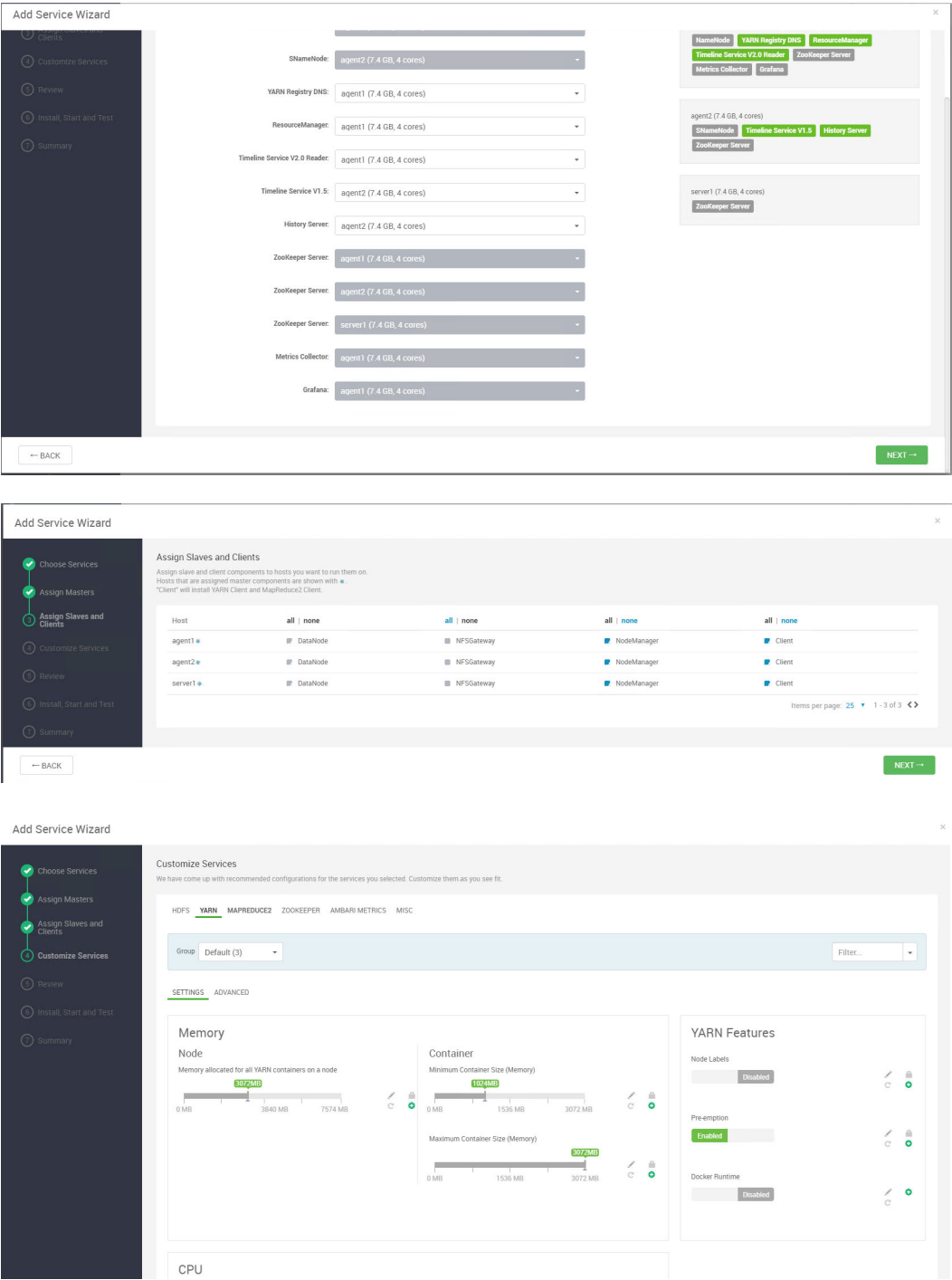
----结束

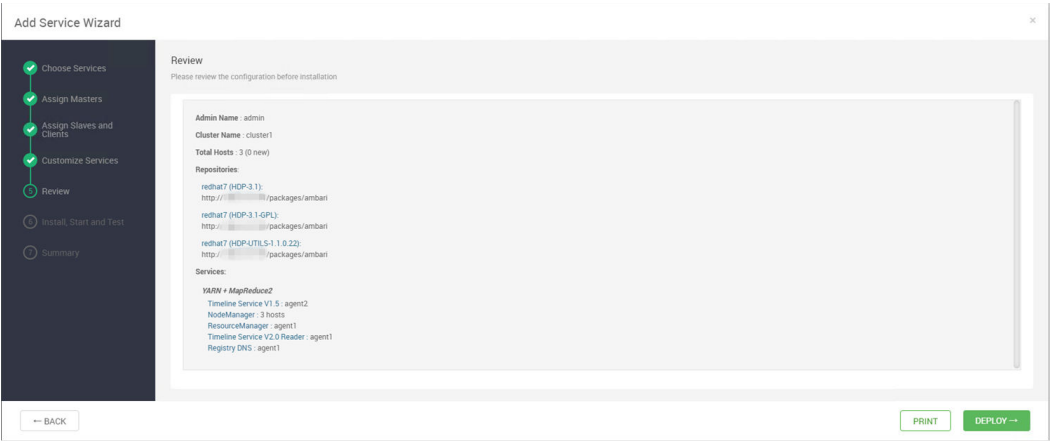
添加服务

添加Yarn+MapReduce2服务：

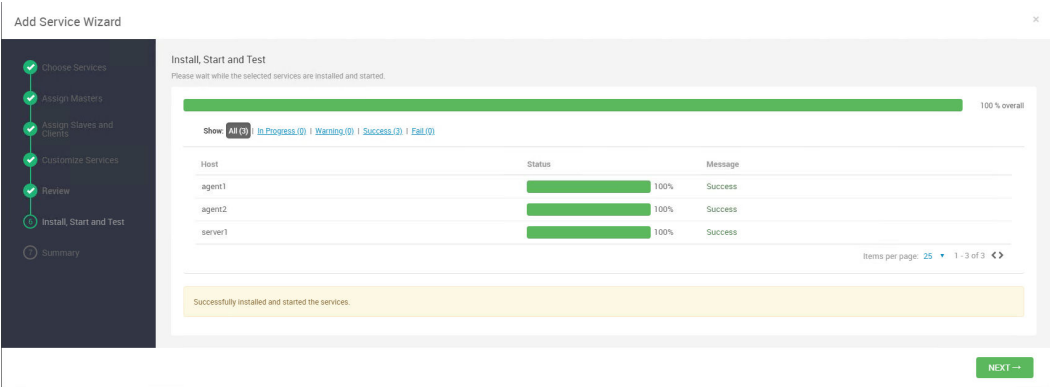
步骤1 单击Add Service，选择yarn+mapreduce2。





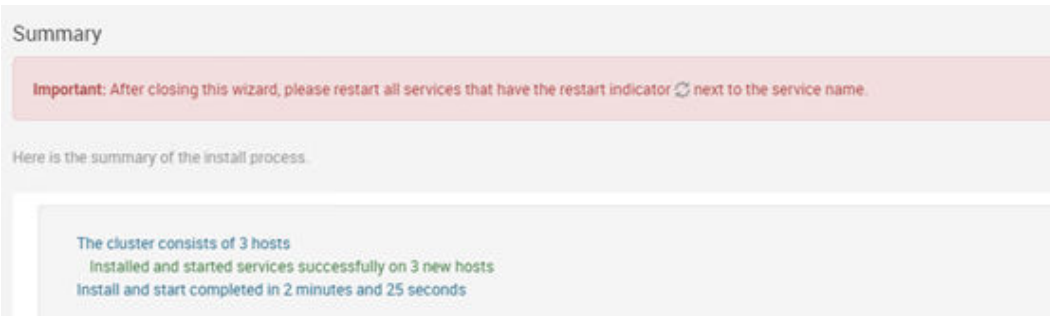


步骤2 单击“下一步”，等待安装完成。



须知

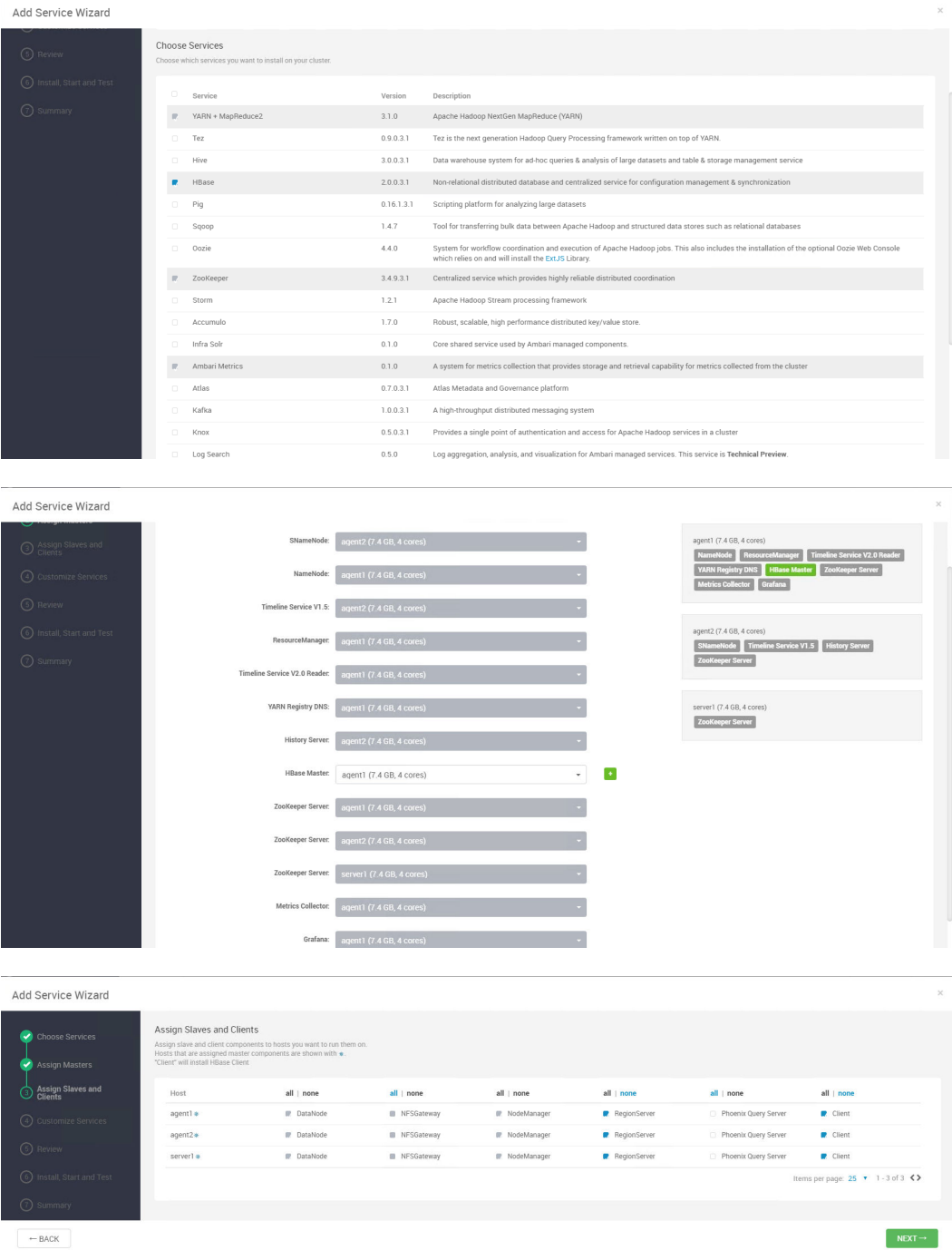
安装完，可能会有以下告警，请重启带重启标志的组件。

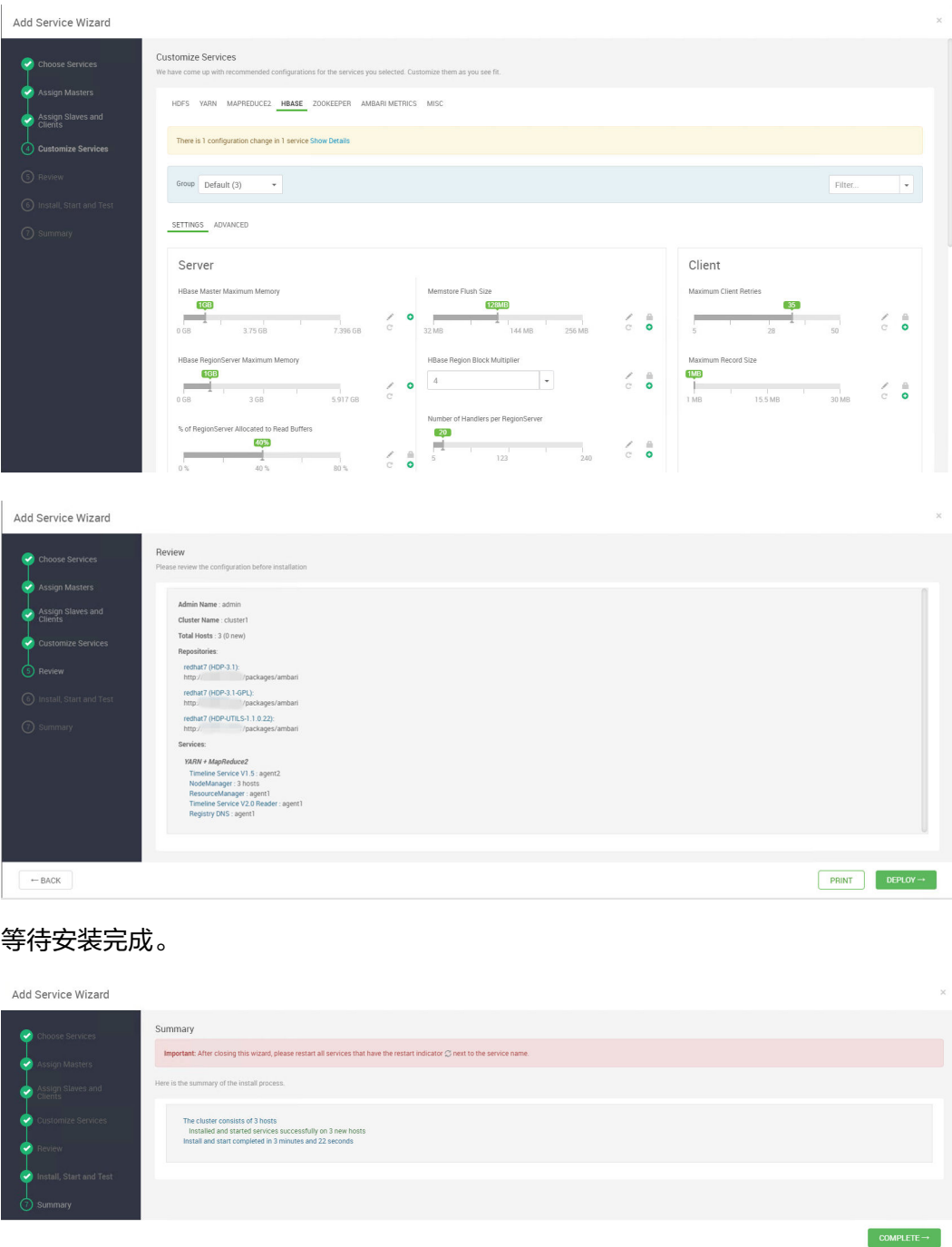


----结束

添加 hbase 服务

单击add service，选择HBase。

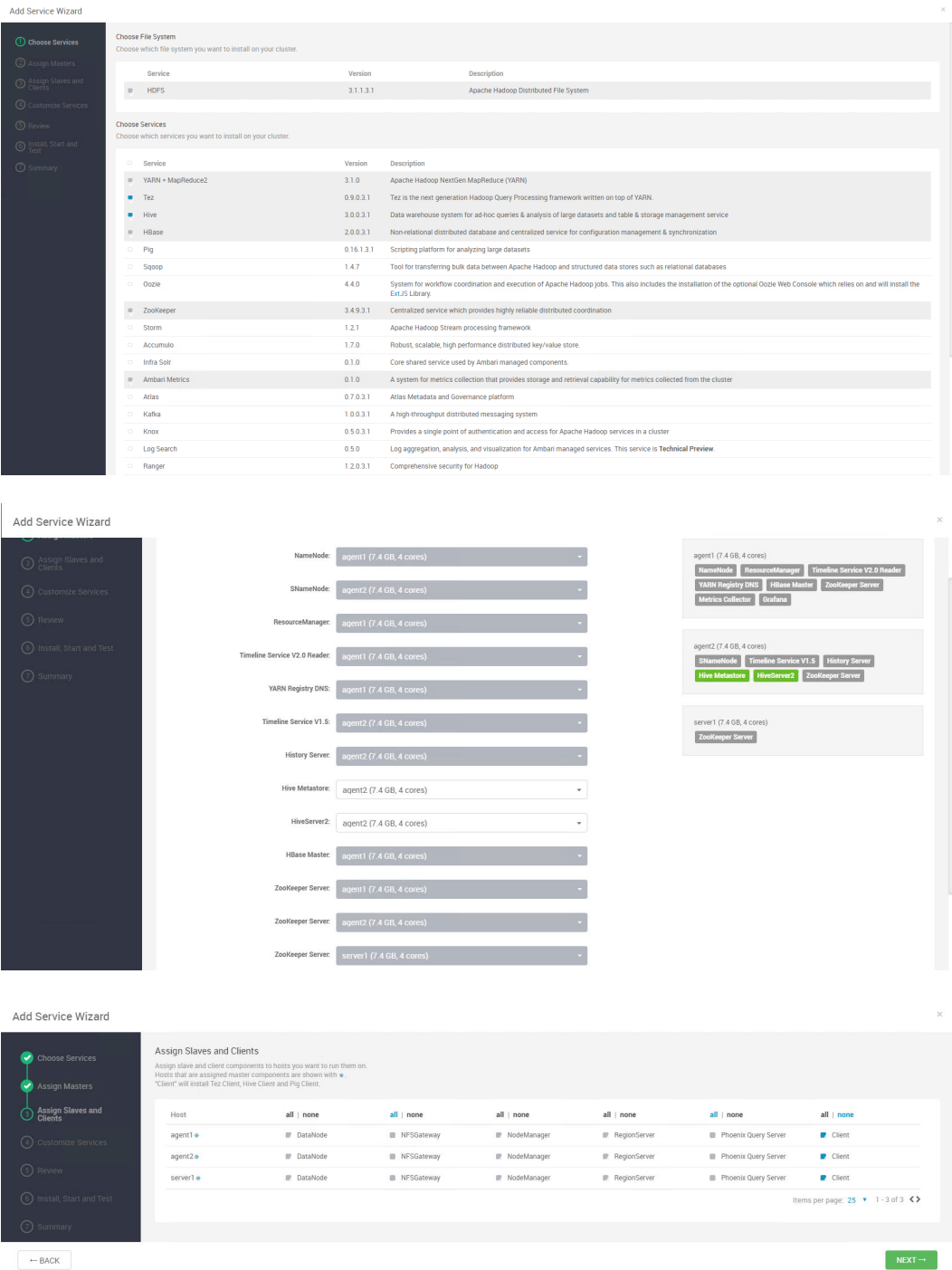




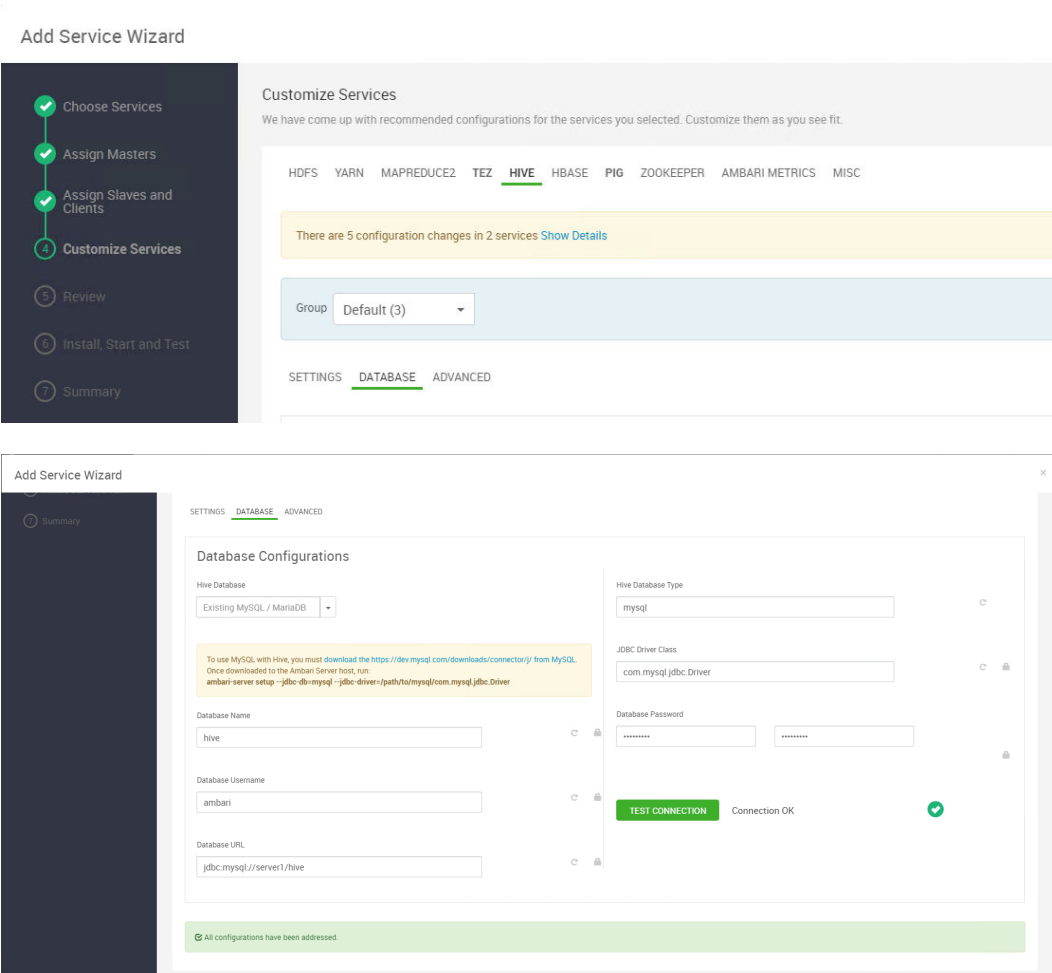
等待安装完成。

添加 hive 服务

步骤1 单击add service，选择Hive和Tez。



步骤2 设置Hive的Database。



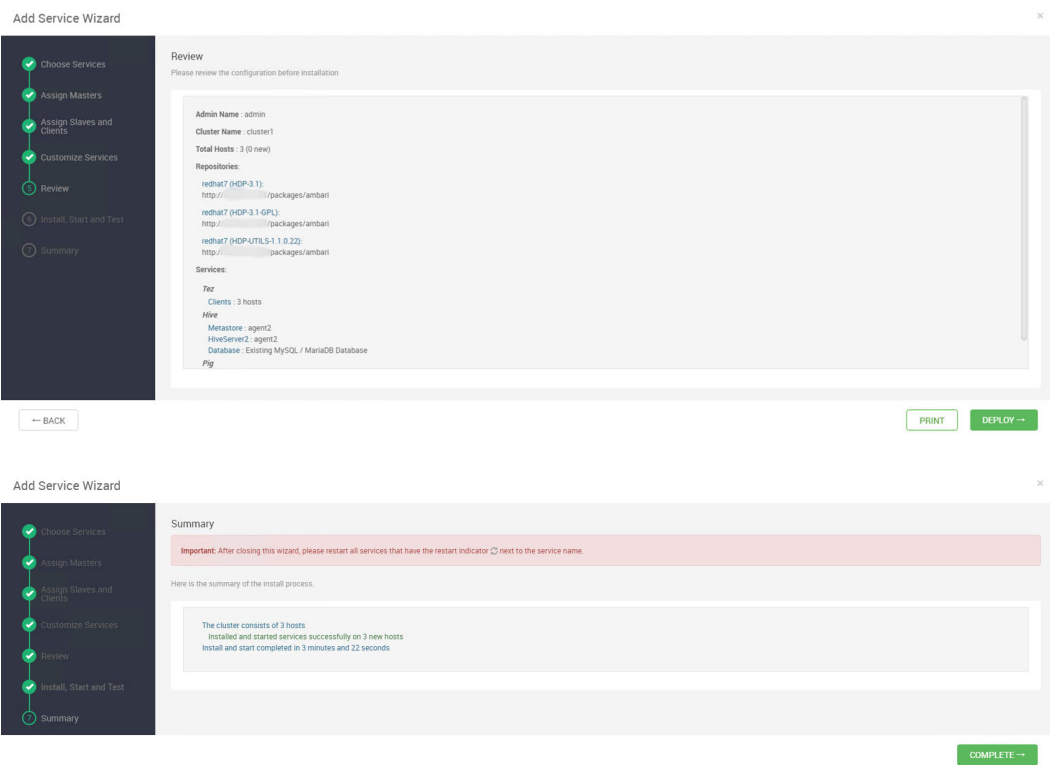
说明

添加Hive服务时需要在MySQL所在的server中新建一个名称为Hive的Database。

- 1. 登录MySQL。
`mysql -u root -p`
- 2. 创建名称为Hive的Database。
`create database hive;`
- 3. 退出MySQL。
`exit;`

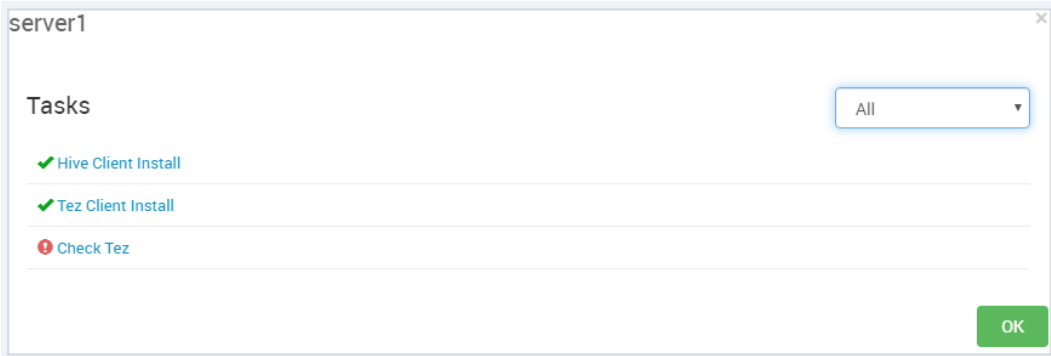
Database的下拉框选择Existing MySQL/MariaDB，Username为4.1 安装数据库 中创建的 Ambari，密码为ambari123，Database URL的主机部分需改成mysql server主机的hostname，图中server主机是server1，根据实际情况填写。如：`jdbc:mysql://server1/hive`。

步骤3 Test connection成功后可单击“下一步”，等待安装完成。



说明

Check Tez失败可忽略。



----结束

添加 spark 服务

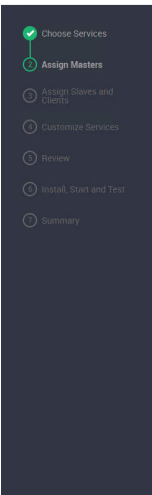
单击“add service”，选择spark2。

Add Service Wizard



☐	Pig	0.16.1.3.1	Scripting platform for analyzing large datasets
☐	Sqoop	1.4.7	Tool for transferring bulk data between Apache Hadoop and structured data stores such as relational databases
☐	Oozie	4.4.0	System for workflow coordination and execution of Apache Hadoop jobs. This also includes the installation of the optional Oozie Web Console which relies on and will install the ExtJS Library.
☒	ZooKeeper	3.4.9.3.1	Centralized service which provides highly reliable distributed coordination
☐	Storm	1.2.1	Apache Hadoop Stream processing framework
☐	Accumulo	1.7.0	Robust, scalable, high performance distributed key/value store.
☐	Infra Solr	0.1.0	Core shared service used by Ambari managed components.
☒	Ambari Metrics	0.1.0	A system for metrics collection that provides storage and retrieval capability for metrics collected from the cluster
☐	Atlas	0.7.0.3.1	Atlas Metadata and Governance platform
☐	Kafka	1.0.0.3.1	A high-throughput distributed messaging system
☐	Knox	0.5.0.3.1	Provides a single point of authentication and access for Apache Hadoop services in a cluster
☒	Log Search	0.5.0	Log aggregation, analysis, and visualization for Ambari managed services. This service is Technical Preview .
☐	Ranger	1.2.0.3.1	Comprehensive security for Hadoop
☐	Ranger KMS	1.2.0.3.1	Key Management Server
☐	SmartSense	1.5.1.2.7.3.0-139	SmartSense - Hortonworks SmartSense Tool (HST) helps quickly gather configuration, metrics, logs from common HDP services that aids to quickly troubleshoot support cases and receive cluster-specific recommendations.
☒	Spark2	2.3.0	Apache Spark 2.3 is a fast and general engine for large-scale data processing.
☐	Zeppelin Notebook	0.8.0	A web-based notebook that enables interactive data analytics. It enables you to make beautiful data-driven, interactive and collaborative documents with SQL, Scala and more.
☐	Druid	0.12.1	A fast column-oriented distributed data store.
☐	Superset	0.23.0	Superset is a data exploration platform designed to be visual, intuitive and interactive. This service is Technical Preview .

Add Service Wizard



Assign Masters

Assign master components to hosts you want to run them on.

DisplayName:	agent2 (7.4 GB, 4 cores)
NameNode:	agent1 (7.4 GB, 4 cores)
Timeline Service V1.5:	agent2 (7.4 GB, 4 cores)
ResourceManager:	agent1 (7.4 GB, 4 cores)
Timeline Service V2.0 Reader:	agent1 (7.4 GB, 4 cores)
YARN Registry DNS:	agent1 (7.4 GB, 4 cores)
History Server:	agent2 (7.4 GB, 4 cores)
Hive Metastore:	agent2 (7.4 GB, 4 cores)
HiveServer2:	agent2 (7.4 GB, 4 cores)
HBase Master:	agent1 (7.4 GB, 4 cores)
ZooKeeper Server:	agent1 (7.4 GB, 4 cores)
ZooKeeper Server:	agent2 (7.4 GB, 4 cores)

agent1 (7.4 GB, 4 cores)

NameNode ResourceManager Timeline Service V2.0 Reader

YARN Registry DNS HBase Master ZooKeeper Server

Metrics Collector Grafana Spark2 History Server

agent2 (7.4 GB, 4 cores)

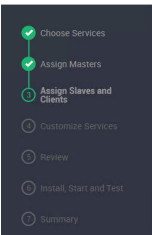
DisplayName Timeline Service V1.5 History Server

Hive Metastore HiveServer2 ZooKeeper Server

server1 (7.4 GB, 4 cores)

ZooKeeper Server

Add Service Wizard



Assign Slaves and Clients

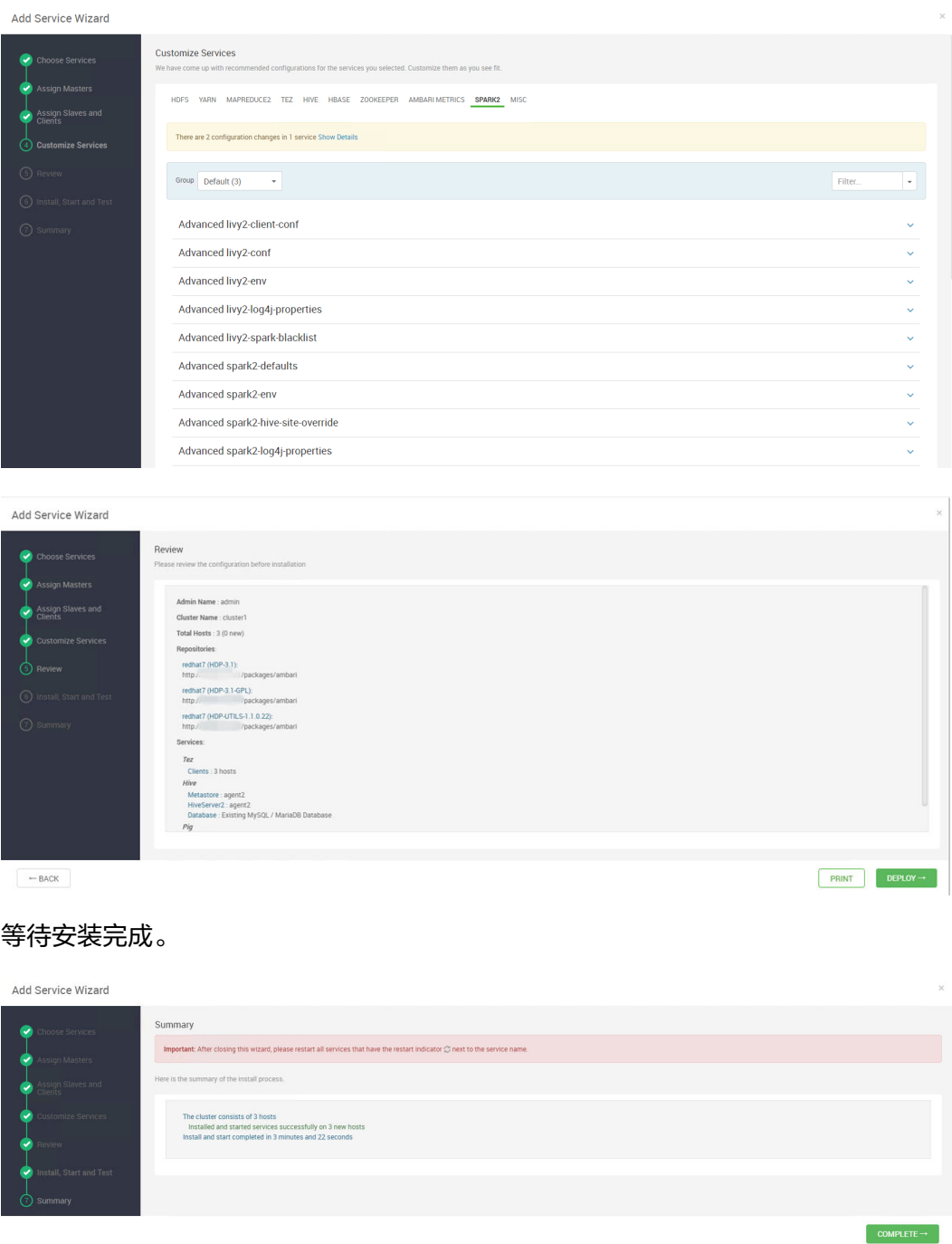
Assign slave and client components to hosts you want to run them on.
Hosts that are assigned master components are shown with *.
Client will install Spark2 Client.

Host	☐ all ☐ none	☐ all ☐ none	☐ all ☐ none	☐ all ☐ none	☐ all ☐ none	☐ all ☐ none	☐ all ☐ none	☐ all ☐ none
agent1 *	☒ DataNode	☒ NFSGateway	☒ NodeManager	☒ RegionServer	☒ Phoenix Query Server	☐ Livy for Spark2 Server	☒ Spark2 Thrift Server	☒ Client
agent2 *	☒ DataNode	☒ NFSGateway	☒ NodeManager	☒ RegionServer	☒ Phoenix Query Server	☐ Livy for Spark2 Server	☐ Spark2 Thrift Server	☒ Client
server1 *	☒ DataNode	☒ NFSGateway	☒ NodeManager	☒ RegionServer	☒ Phoenix Query Server	☐ Livy for Spark2 Server	☐ Spark2 Thrift Server	☒ Client

Items per page: 25 • 1 - 3 of 3 < >

← BACK

NEXT →



等待安装完成。

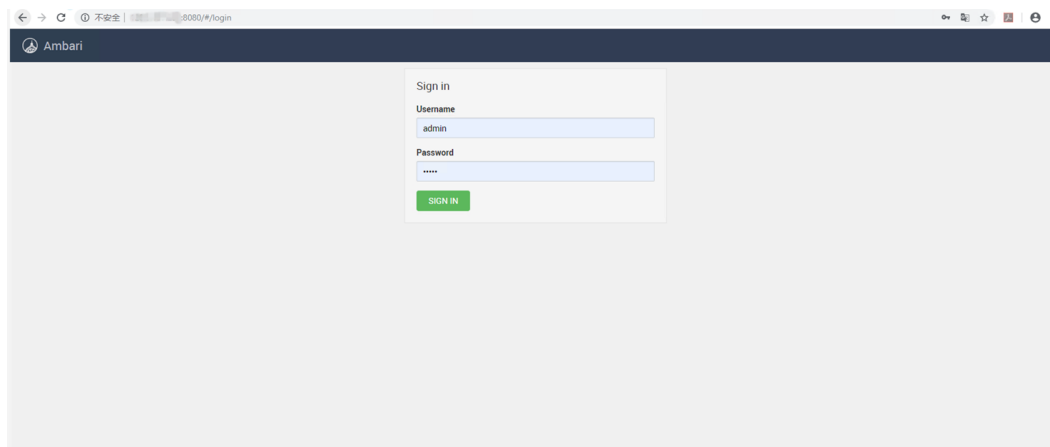
6 部署 Accumulo

6.1 部署 Accumulo 服务

6.2 删除 Accumulo 服务

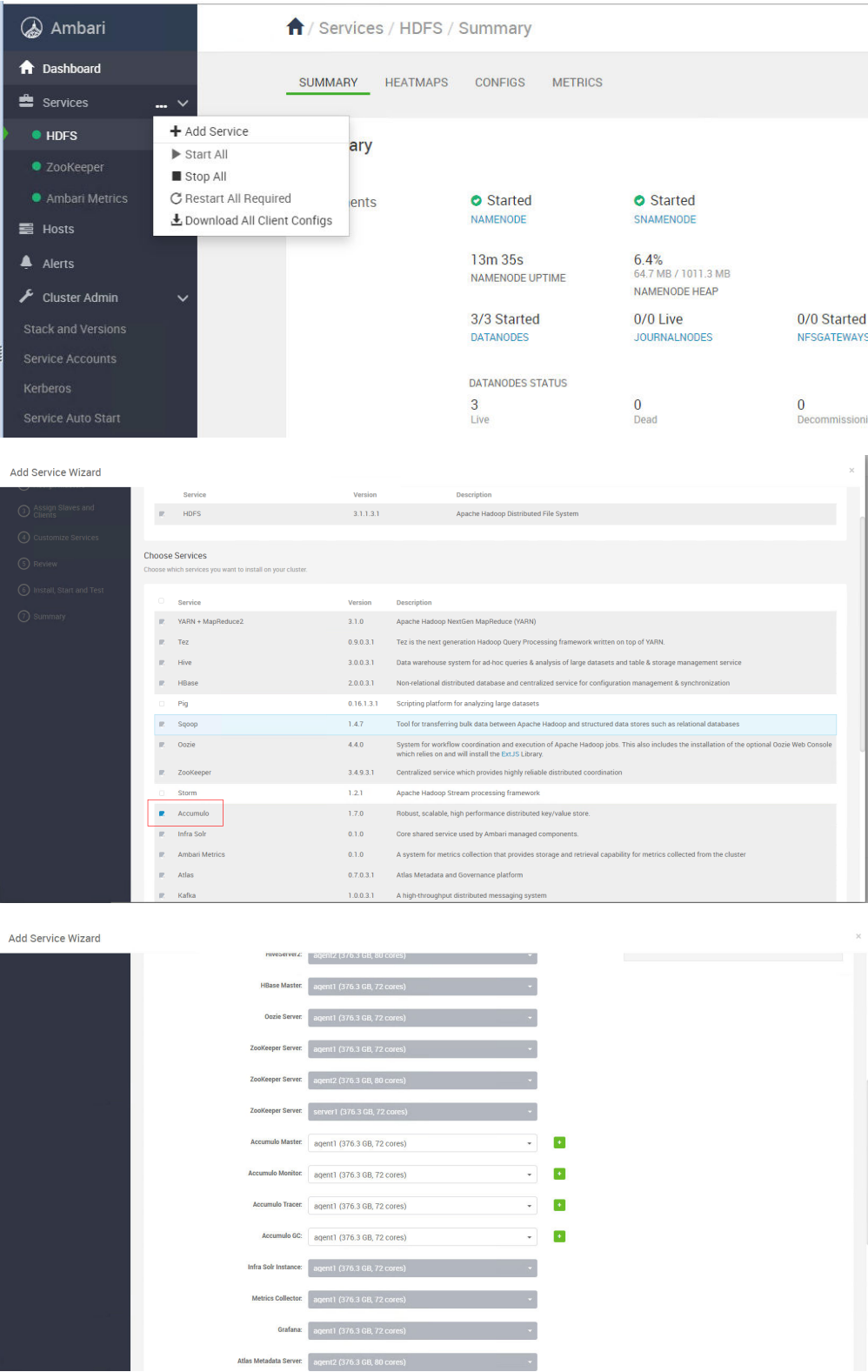
6.1 部署 Accumulo 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

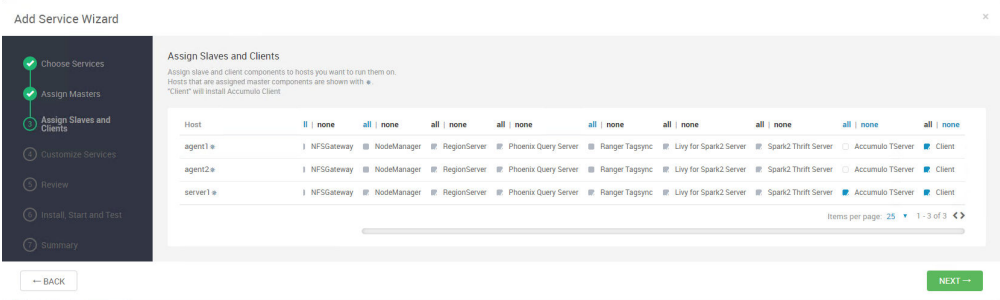


步骤2 添加Accumulo服务。

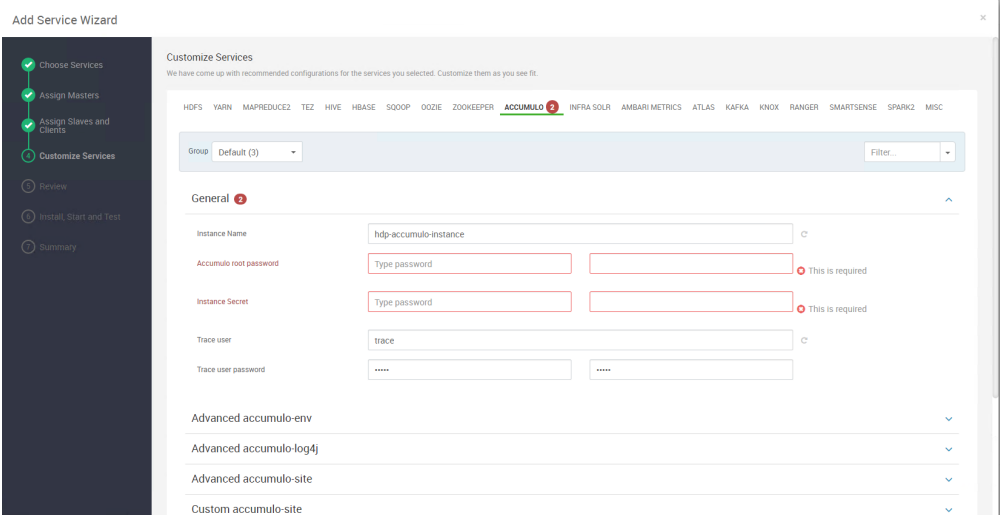
1. 单击Add Service，选择Accumulo。



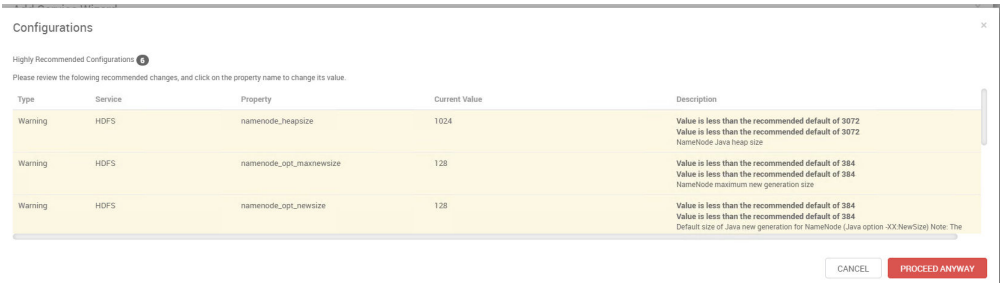
2. 推荐按照下图勾选。



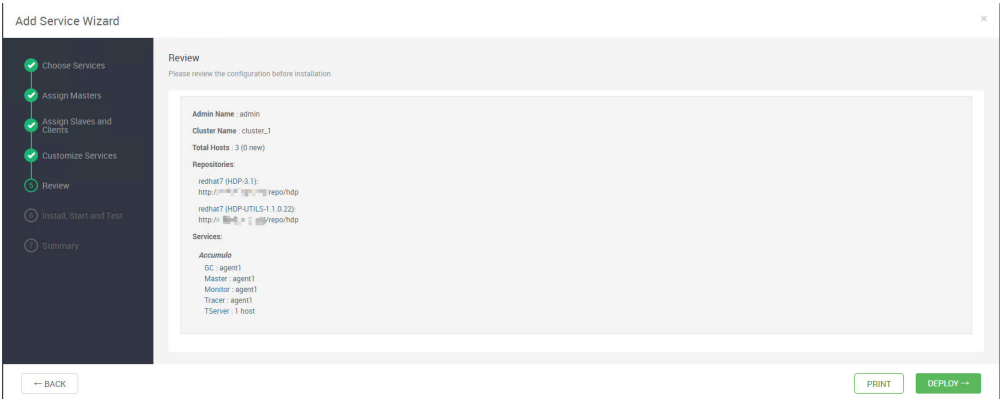
3. 设置密码。



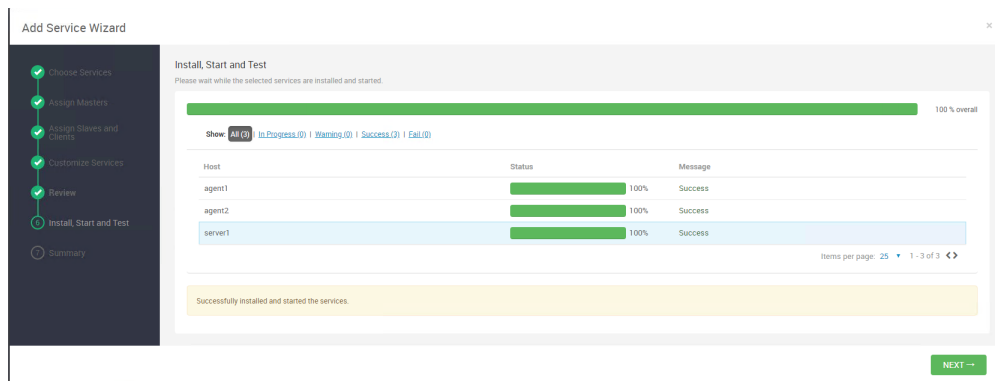
4. 如有弹框，跳过即可，不影响部署，也可按照弹框提示对参数进行修改。



5. 然后单击DEPLOY。



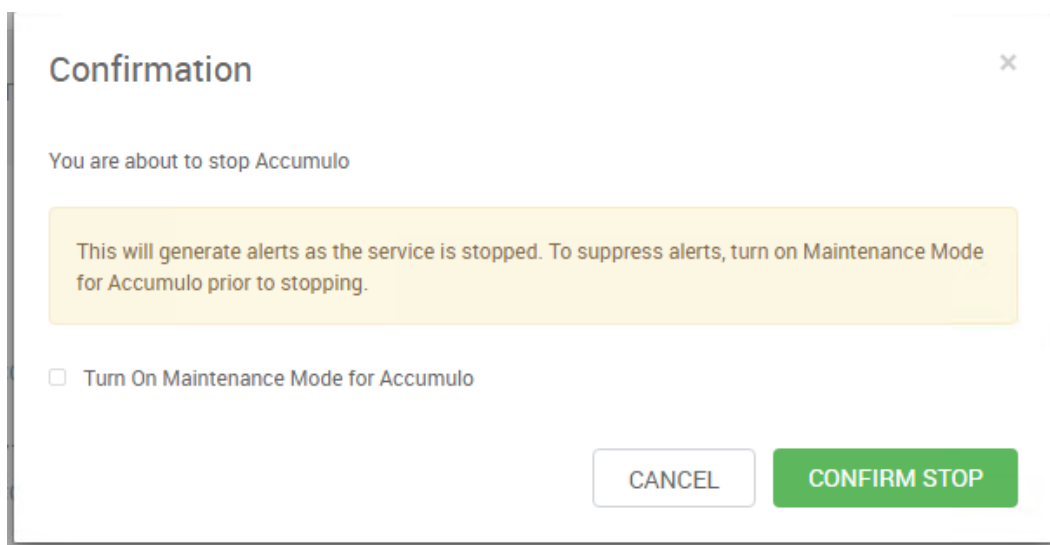
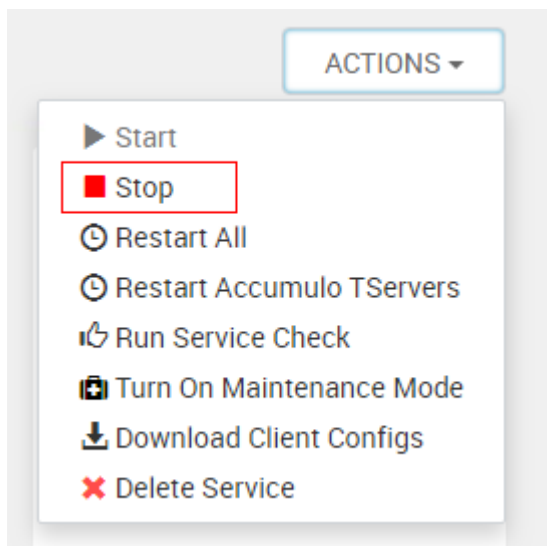
6. 单击NEXT，等待安装完成。



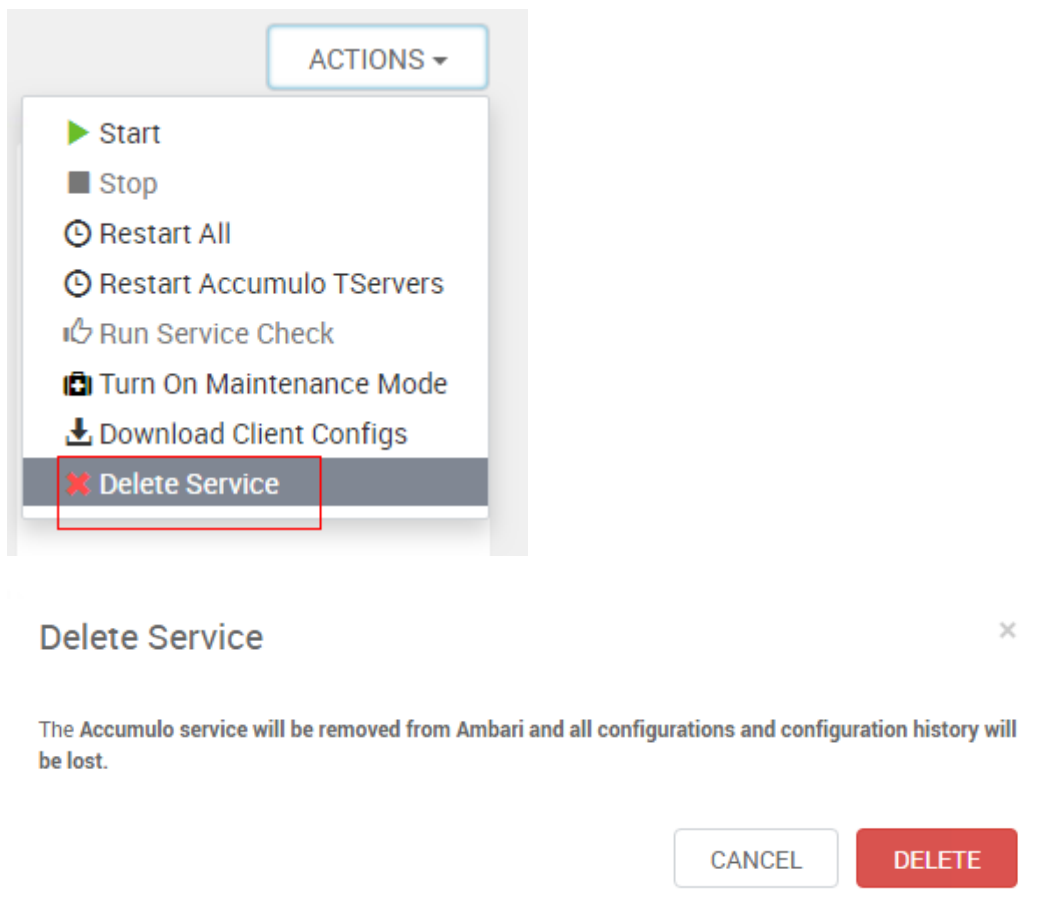
----结束

6.2 删除 Accumulo 服务

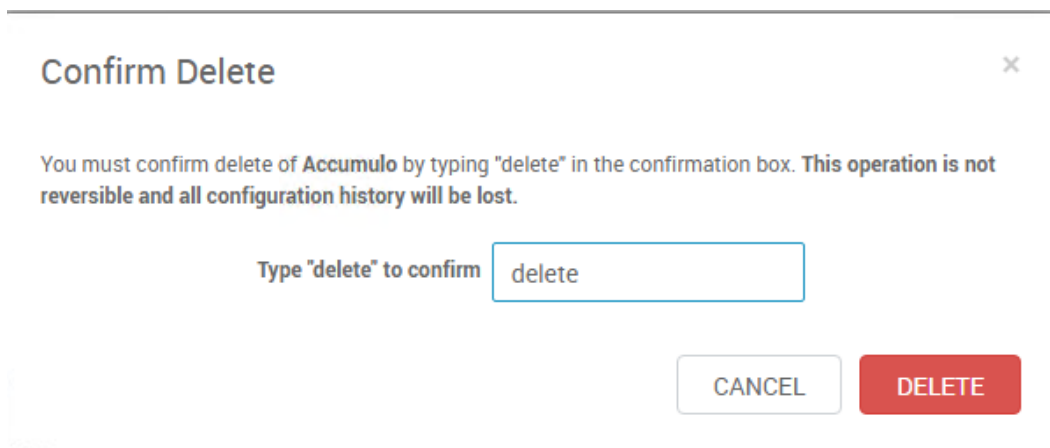
步骤1 在Accumulo界面单击ACTIONS，选择Stop，先停止服务。



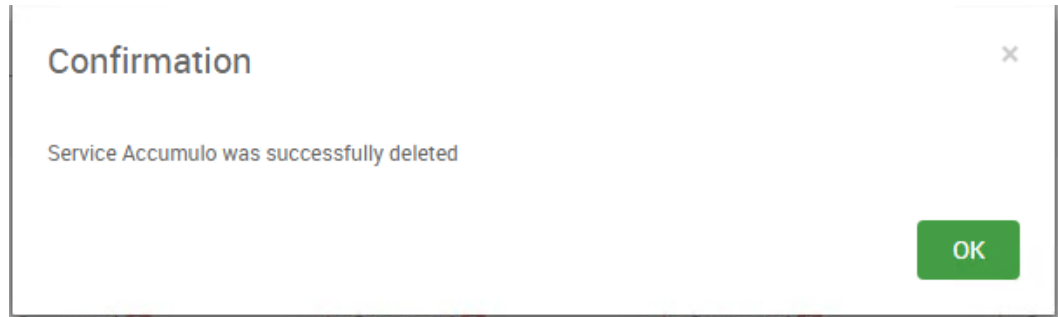
步骤2 停止服务后选择Delete Service，并单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

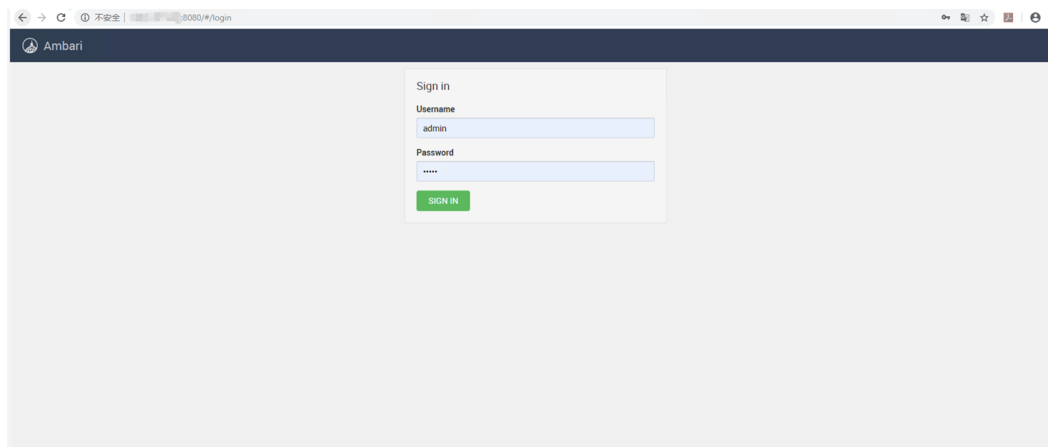
7 部署 Atlas&Kafka&Solr

7.1 部署Atlas&Kafka&Solr服务

7.2 删除Atlas&Kafka&Solr服务

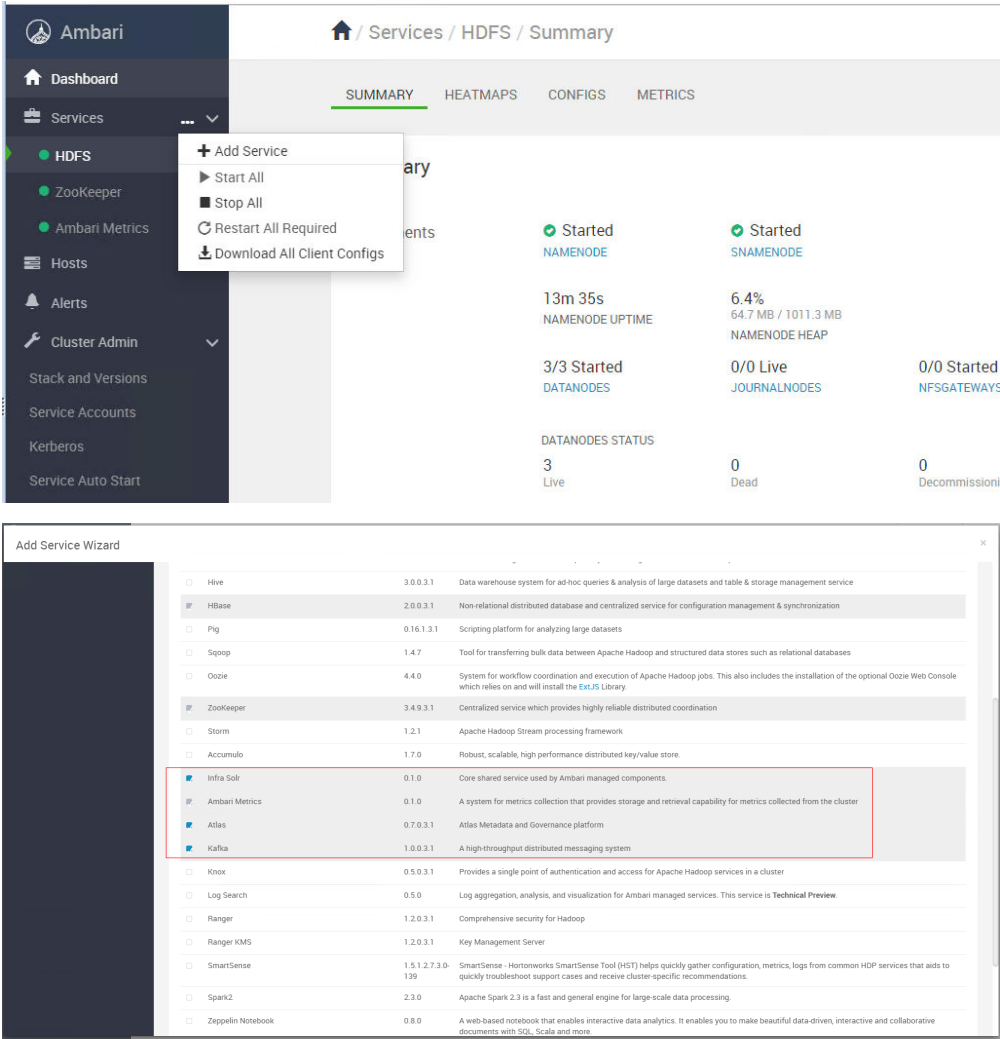
7.1 部署 Atlas&Kafka&Solr 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

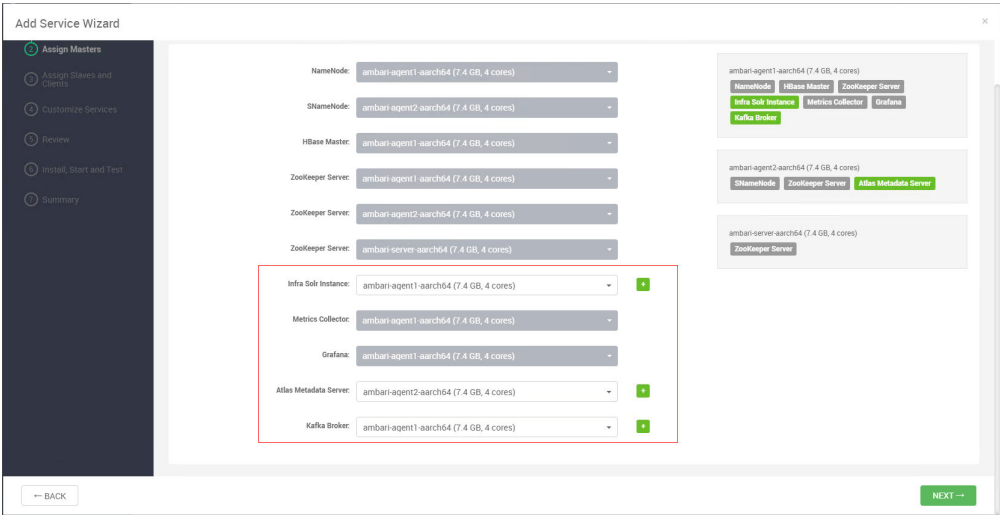


步骤2 添加Atlas和Kafk以及Solr服务。

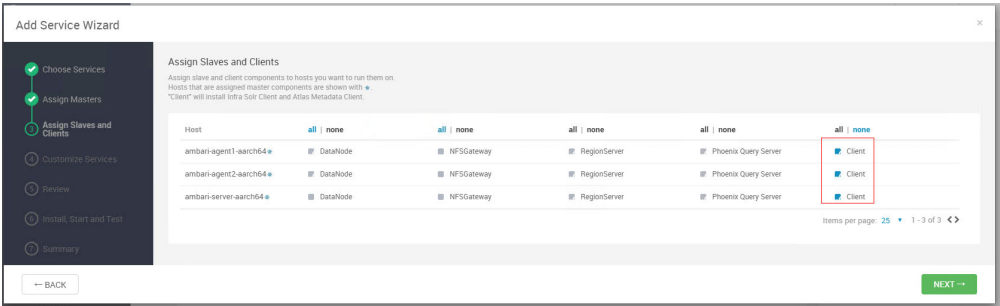
1. 单击Add Service，选择Atlas和Kafka以及Solr。



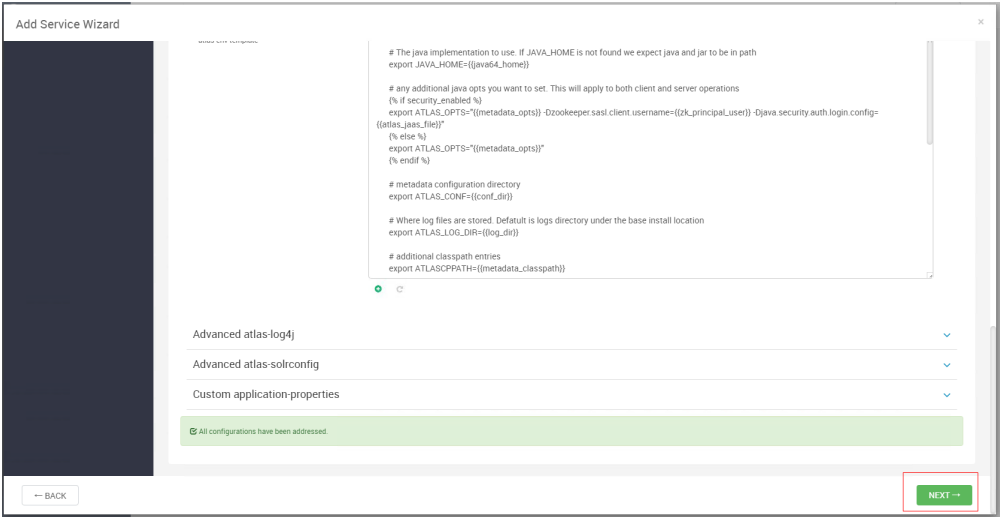
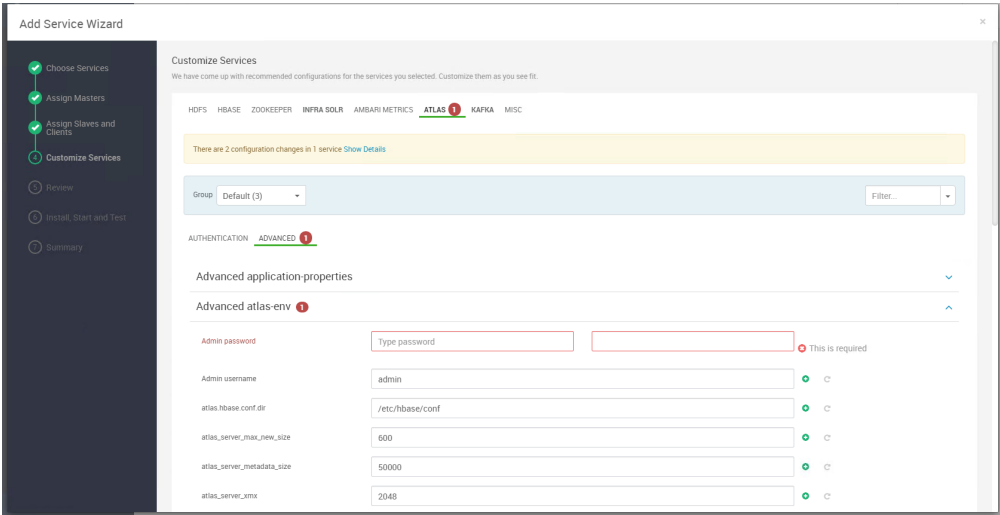
2. 红框处没有特殊需求按照默认选项即可。



3. 推荐按照下图勾选client。



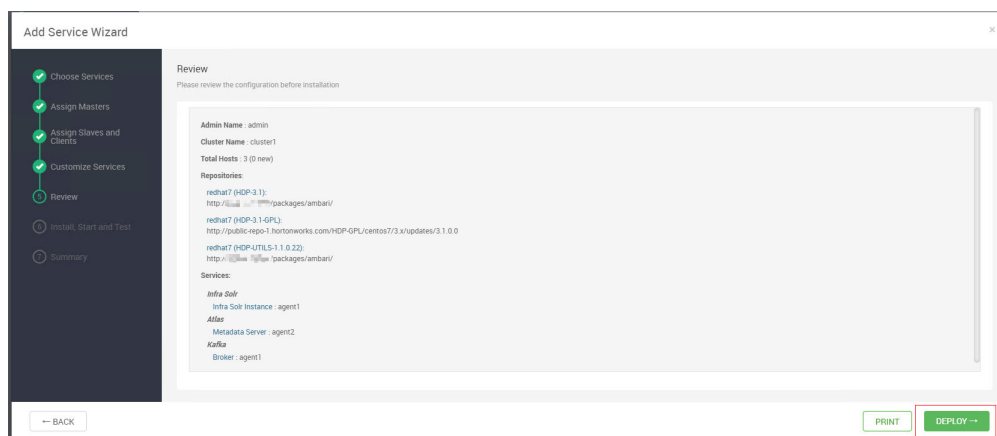
4. 设置Admin password。



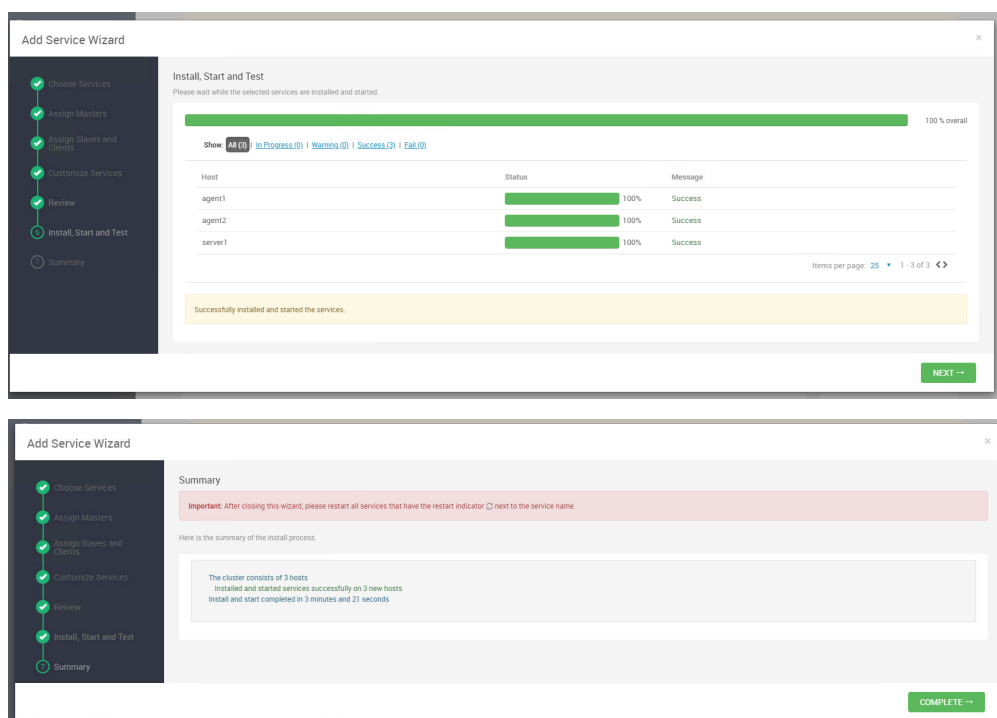
5. 如有弹框，跳过即可，不影响部署，也可按照弹框提示对参数进行修改。



6. 然后单击DEPLOY。



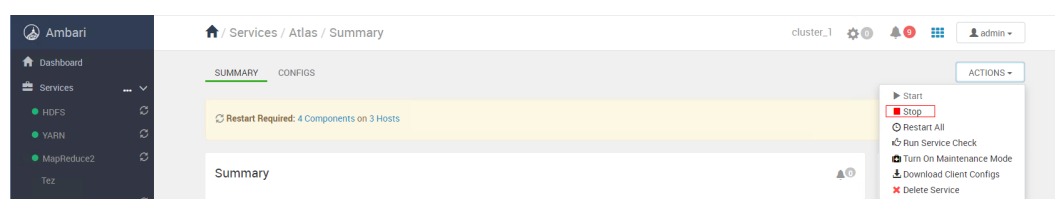
7. 单击NEXT，等待安装完成。

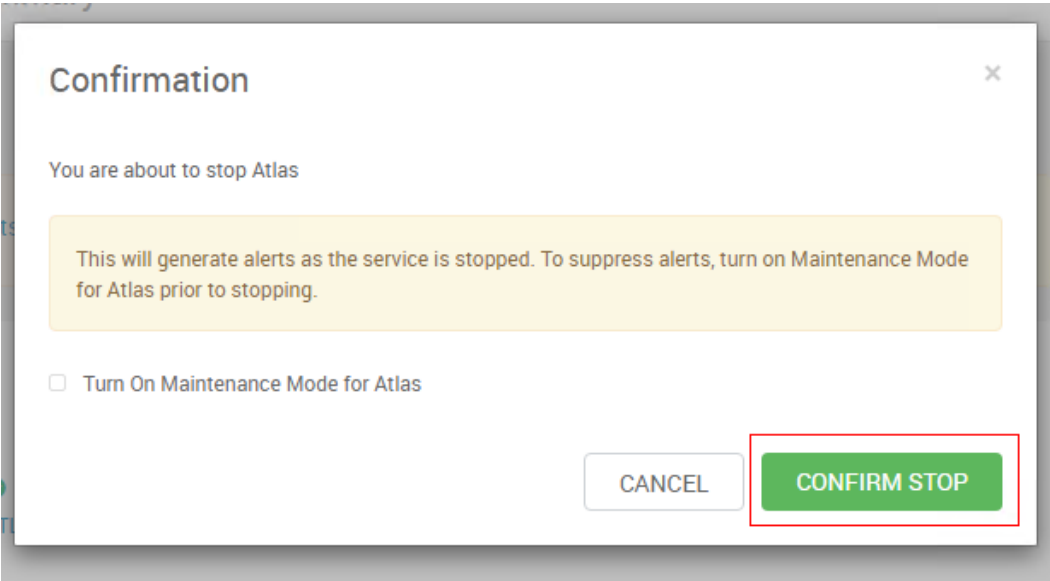


----结束

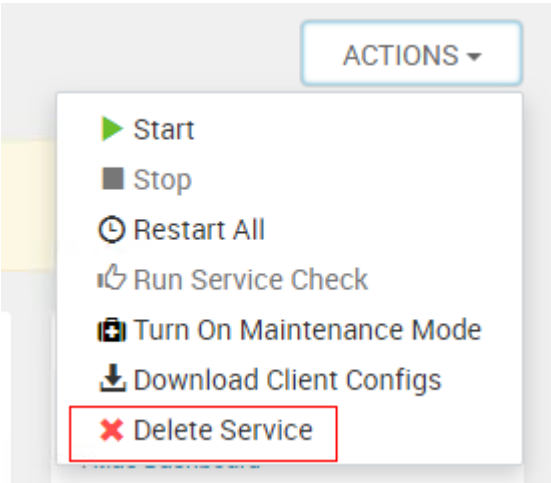
7.2 删除 Atlas&Kafka&Solr 服务

步骤1 在Atlas界面单击ACTIONS，选择Stop，先停止服务。

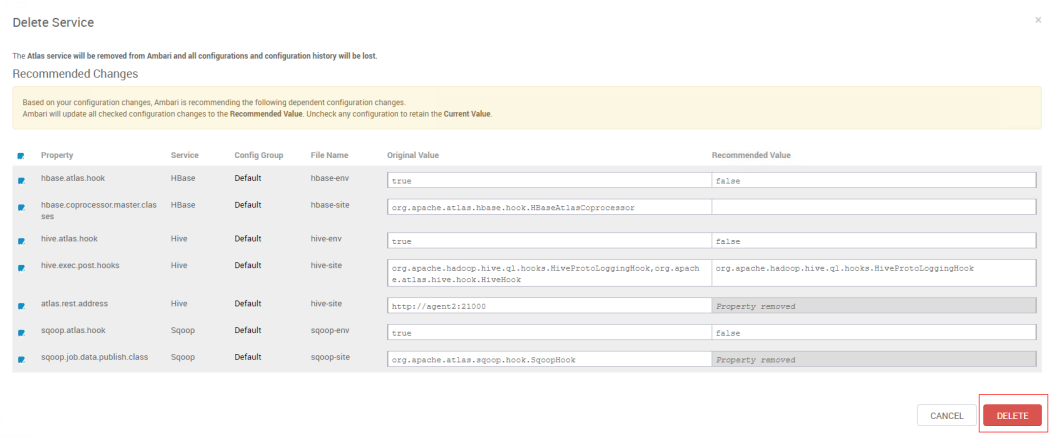




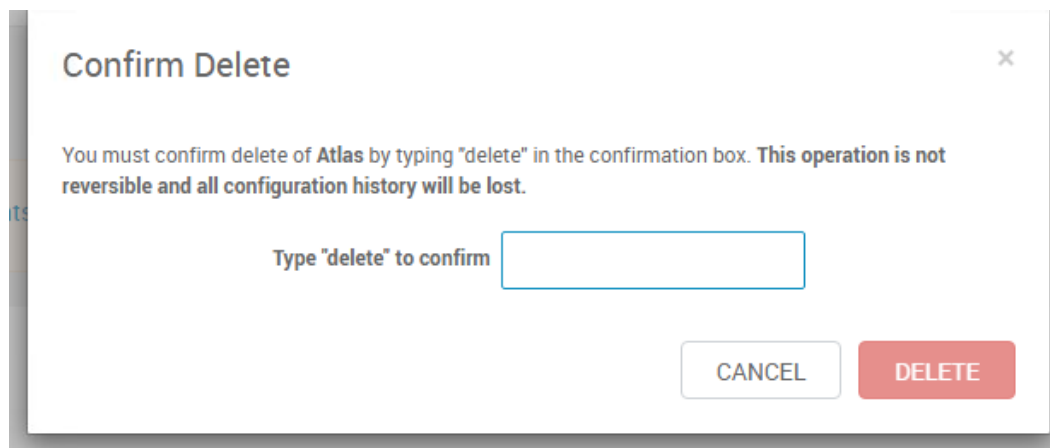
步骤2 停止服务后选择Delete Service。



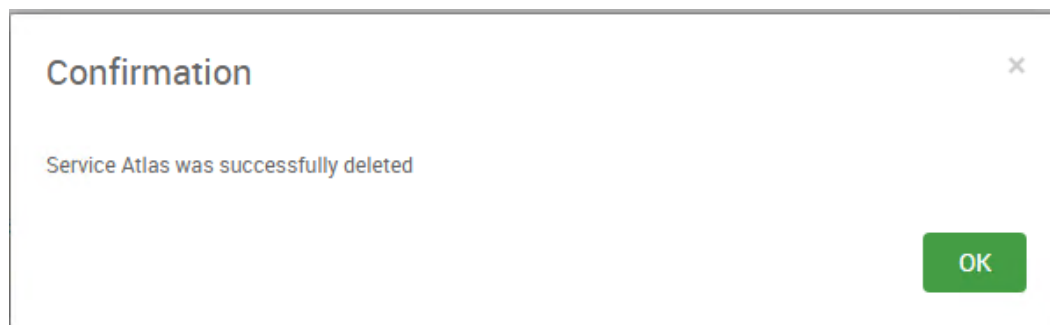
步骤3 勾选所有服务，并单击DELETE。



步骤4 在对话框中输入delete，并单击DELETE。



步骤5 删除完成后单击OK。



----结束

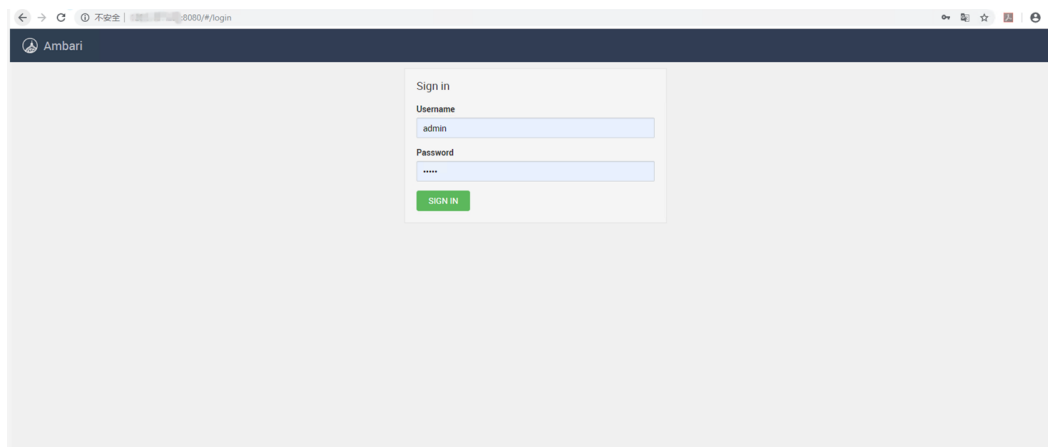
8 部署 HBase&Phoenix

8.1 部署HBase&Phoenix服务

8.2 删除HBase&Phoenix服务

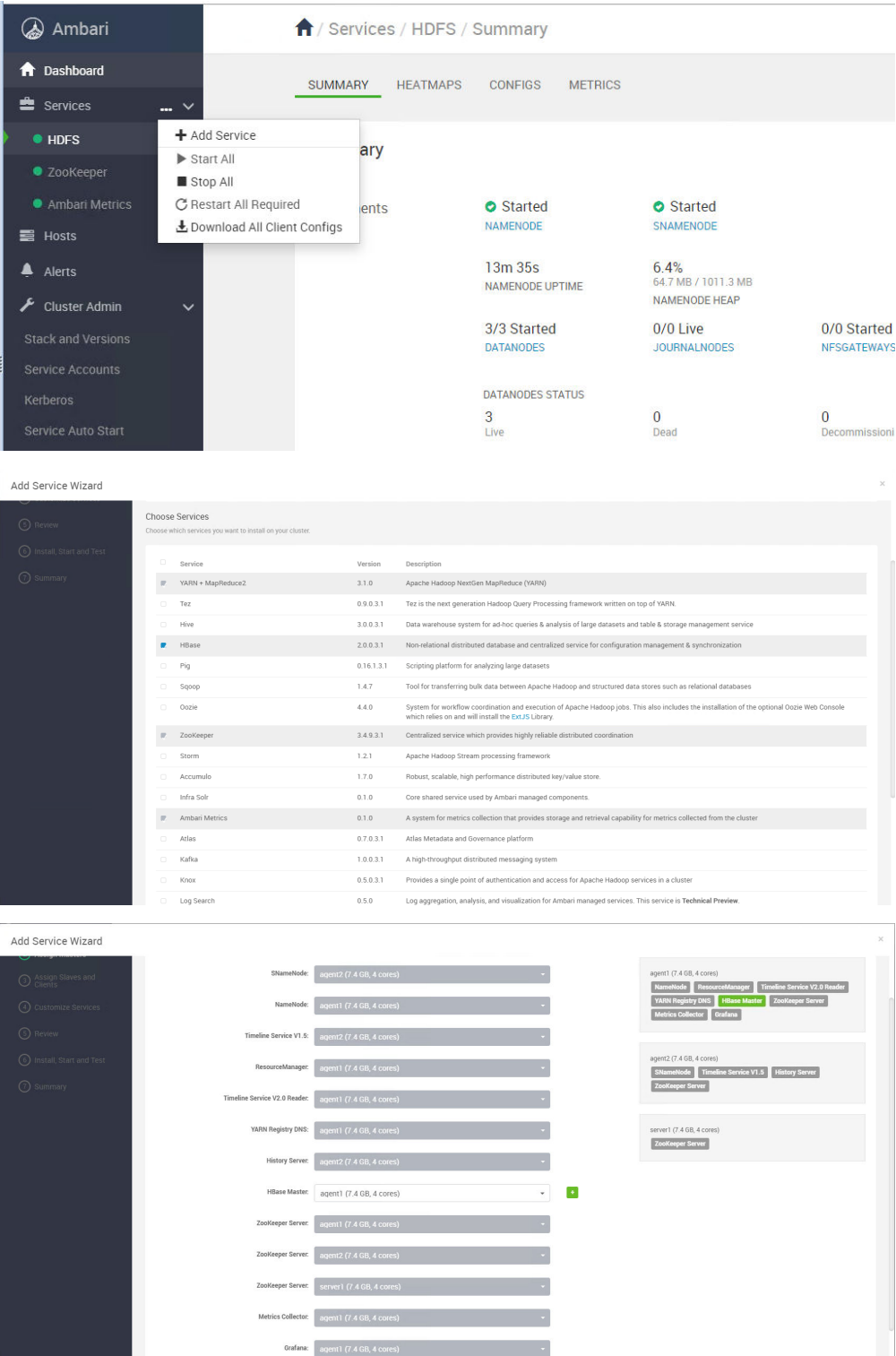
8.1 部署 HBase&Phoenix 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

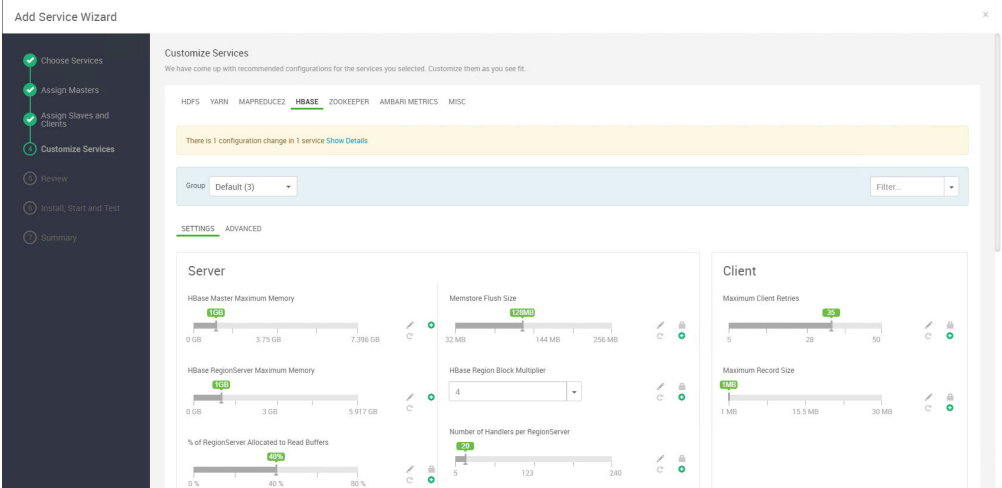
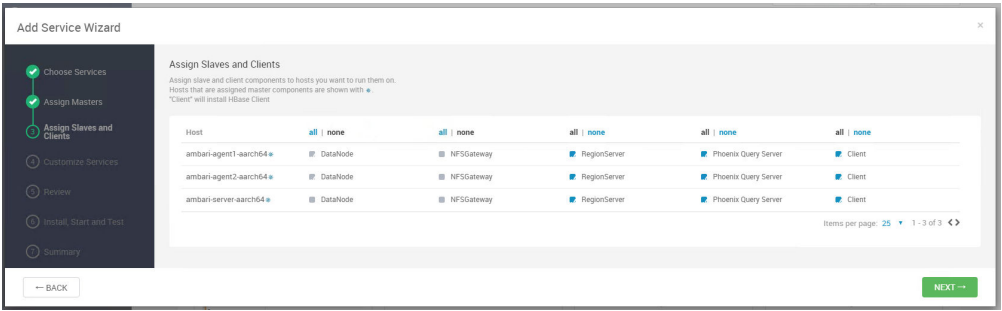


步骤2 添加HBase和Phoenix服务。

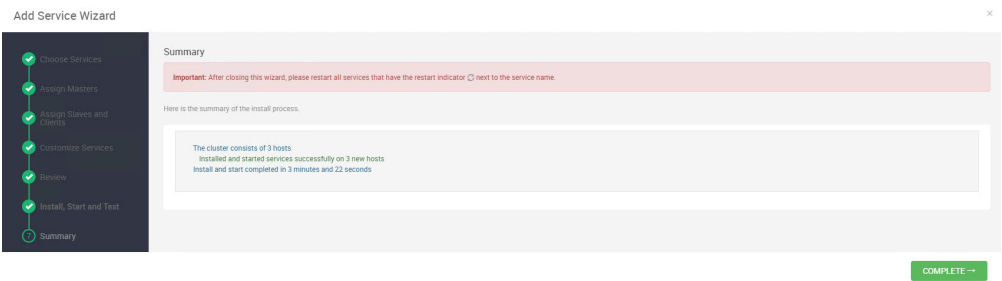
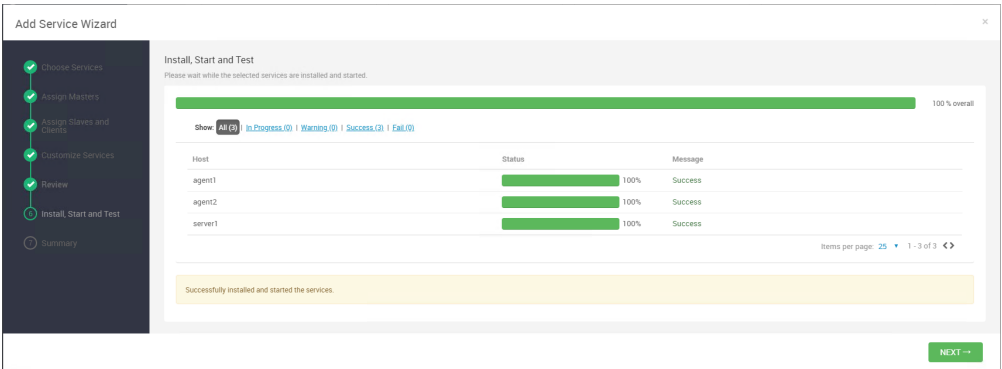
1. 单击Add Service，选择HBase。



2. 勾选Phoenix Query Server即可安装Phoenix服务。



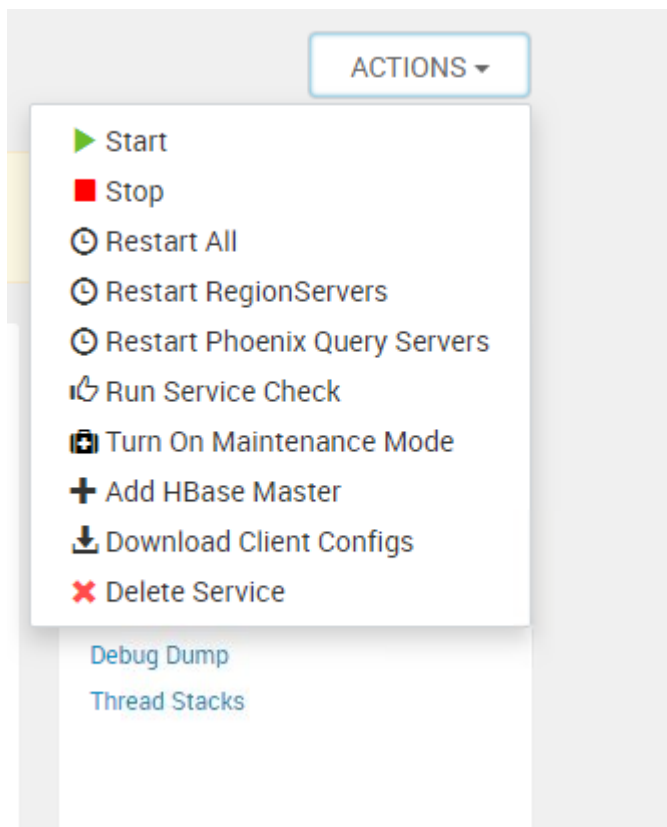
3. 单击NEXT，等待安装完成。



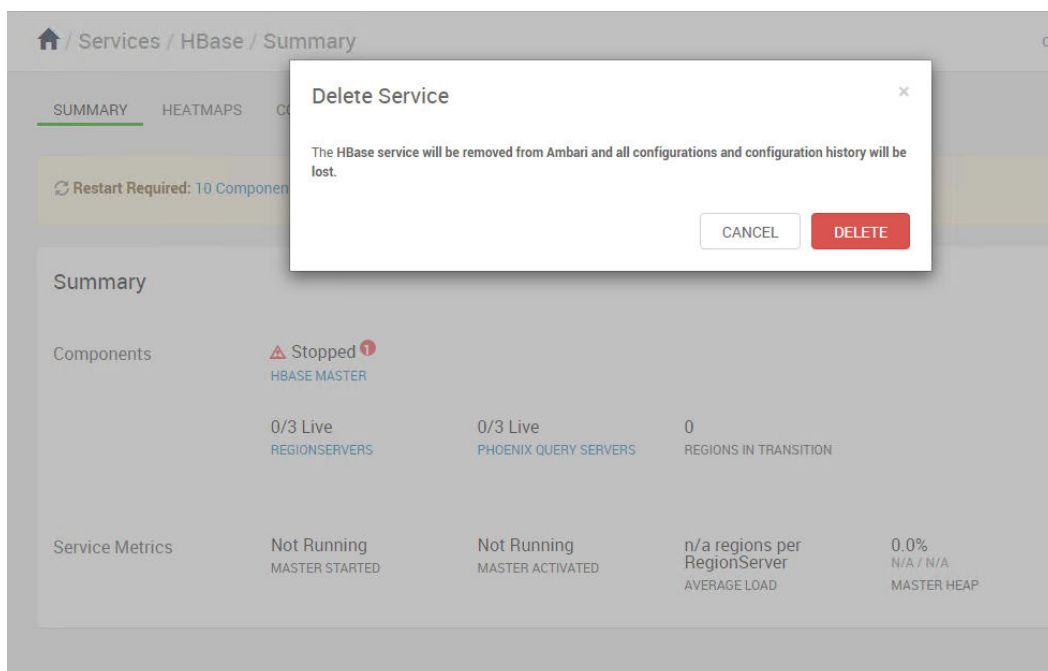
----结束

8.2 删除 HBase&Phoenix 服务

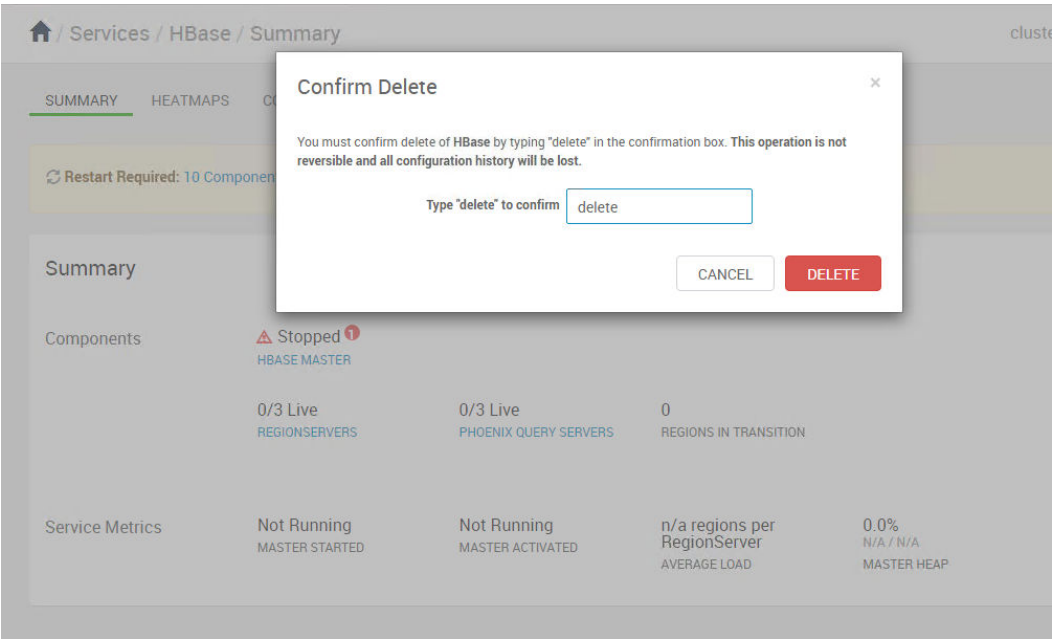
步骤1 在HBase界面单击ACTIONS，选择Stop，先停止服务。



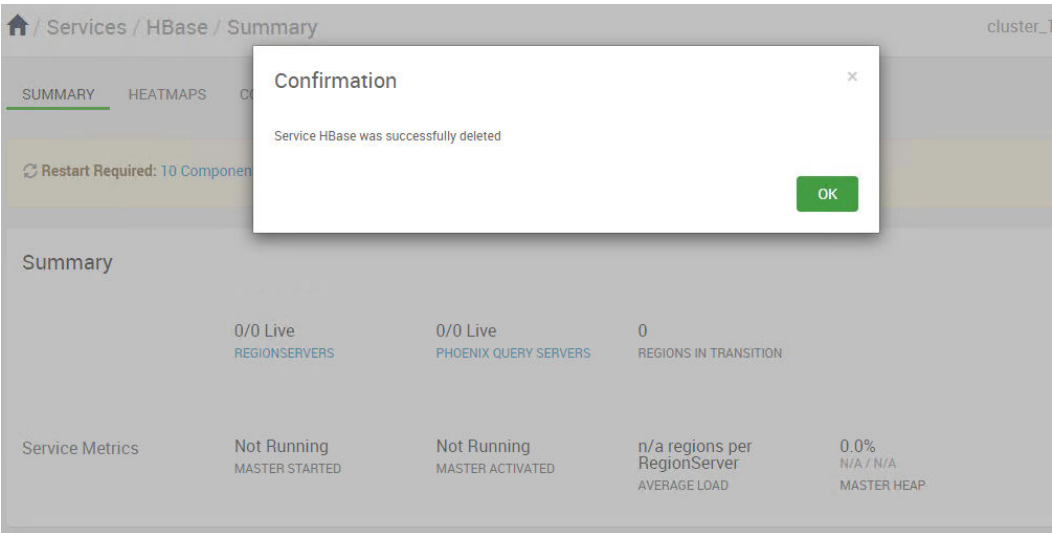
步骤2 停止服务后选择Delete Service，并单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

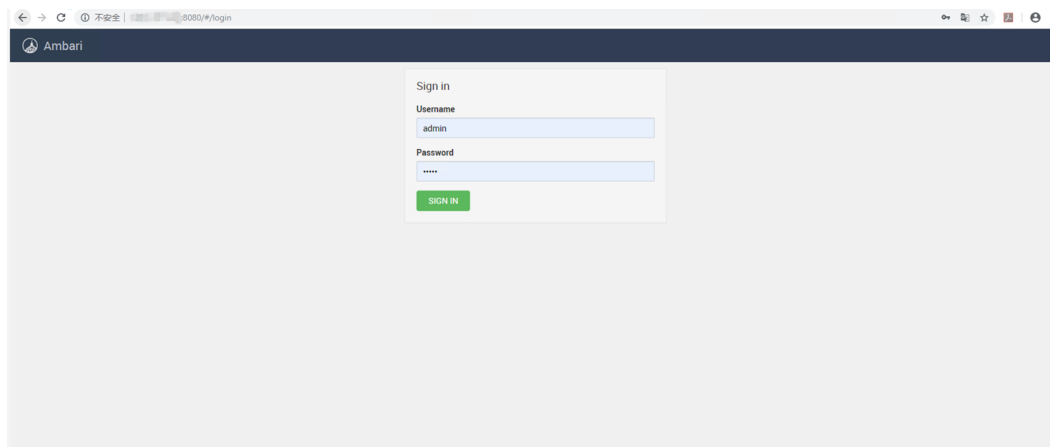
9 部署 Kafka

9.1 部署Kafka服务

9.2 删除Kafka服务

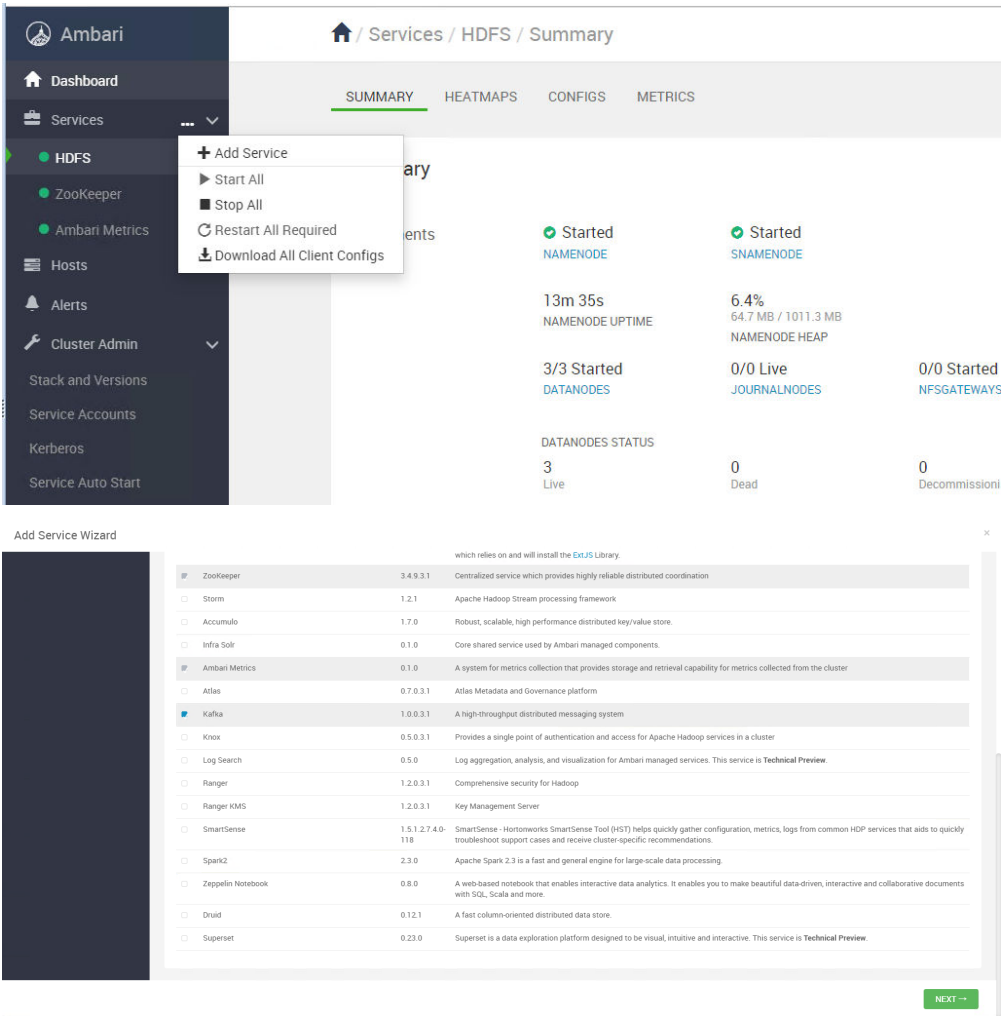
9.1 部署 Kafka 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

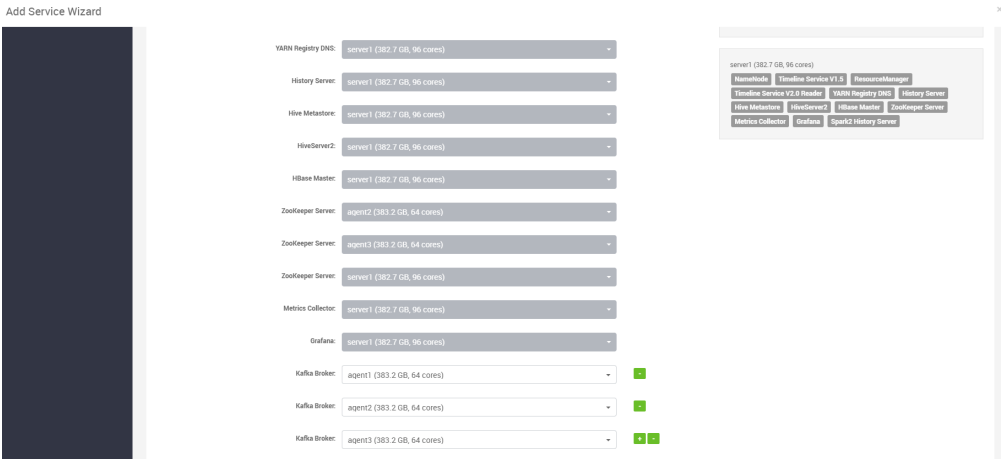


步骤2 添加Kafka服务。

1. 单击Add Service，选择Kafka。



根据需求，选择安装Kafka的节点，单击绿色的+号来新增Kafka节点。



2. 修改kafka参数。

参数1：

只需要在默认值后面添加/kafka即可。

zookeeper.connection = agent2:2181,agent3:2181,server1:2181/kafka

参数2：

kafka数据目录根据实际的配置来。

Log.directories = /data/data1/kafka-logs,/data/data2/kafka-logs,/data/data3/kafka-logs,/data/data4/kafka-logs,/data/data5/kafka-logs,/data/data6/kafka-logs,/data/data7/kafka-logs,/data/data8/kafka-logs,/data/data9/kafka-logs,/data/data10/kafka-logs,/data/data11/kafka-logs,/data/data12/kafka-logs

Add Service Wizard

Customize Services

We have come up with recommended configurations for the services you selected. Customize them as you see fit.

HDFS YARN MAPREDUCE2 TEZ HIVE HBASE ZOOKEEPER AMBARI METRICS **KAFKA** SPARK2 MISC

Group: Default (4) Filter: [v]

Kafka Broker

Kafka Broker hosts: agent1 and 2 others

zookeeper.connect: agent2-2181,agent3-2181,server1-2181/kafka

log.roll.hours: 168

log.retention.hours: 168

Log directories: /data/data1/kafka-logs,/data/data2/kafka-logs,/data/data3/kafka-logs,/data/data4/kafka-logs,/data/data5/kafka-logs,/data/data6/kafka-logs,/data/data7/kafka-logs,/data/data8/kafka-logs,/data/data9/kafka-logs,/data/data10/kafka-logs,/data/data11/kafka-logs,/data/data12/kafka-logs

listeners: PLAINTEXT://localhost:6667

Add Service Wizard

log.roll.hours: 168

log.retention.hours: 168

Log directories: /data/data1/kafka-logs,/data/data2/kafka-logs,/data/data3/kafka-logs,/data/data4/kafka-logs,/data/data5/kafka-logs,/data/data6/kafka-logs,/data/data7/kafka-logs,/data/data8/kafka-logs,/data/data9/kafka-logs,/data/data10/kafka-logs,/data/data11/kafka-logs,/data/data12/kafka-logs

listeners: PLAINTEXT://localhost:6667

Advanced kafka-broker

Advanced kafka-env

Advanced kafka-log4j

Advanced kafka_client_jaas_conf

Advanced kafka_jaas_conf

Custom kafka-broker

Custom kafka-log4j

All configurations have been addressed.

BACK NEXT

Add Service Wizard

Configurations

Highly Recommended Configuration

Please review the following recommended changes, and click on the property name to change its value.

Type	Service	Property	Current Value	Description
Warning	HDFS	namenode_heapsize	1024	Value is less than the recommended default of 3072 Value is less than the recommended default of 3072 NameNode Java heap size
Warning	HDFS	namenode_opt_maxnewsize	128	Value is less than the recommended default of 384 Value is less than the recommended default of 384 NameNode maximum new generation size
Warning	HDFS	namenode_opt_newsize	128	Value is less than the recommended default of 384

CANCEL PROCEED ANYWAY

3. 单击DEPLOY。

Add Service Wizard

Review

Please review the configuration before installation

Admin Name: admin

Cluster Name: cluster1

Total Hosts: 3 (0 new)

Repos/hosts:

redhat7 (HDP-3.1): http://repo/

redhat7 (HDP-3.1-GPL): http://repo/

redhat7 (HDP-UTLS-1.1.0.22): http://repo/

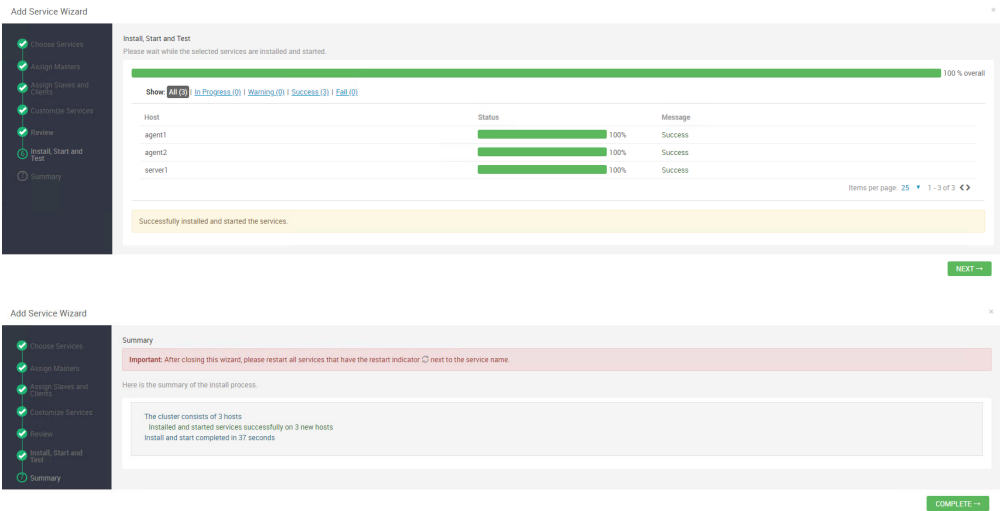
Services:

Kafka

Broker: agent1

BACK PRINT DEPLOY

4. 单击NEXT，等待安装完成。

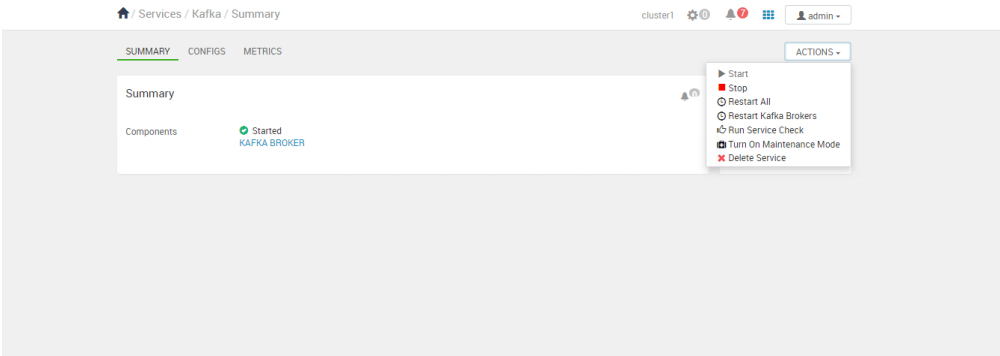


----结束

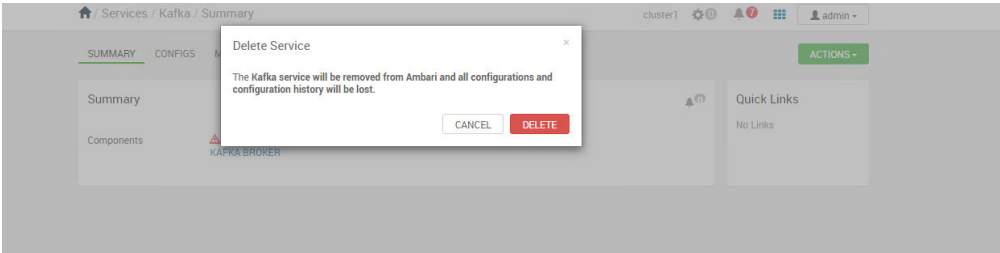
9.2 删除 Kafka 服务

步骤1 在Ambari界面上删除Kafka服务。

1. 在Kafka界面单击ACTIONS，选择Stop，先停止服务。



2. 停止服务后，单击ACTIONS，选择Delete Service，并单击DELETE。



3. 在对话框中输入"delete"，并单击DELETE。

2. 删除 “/Kafka” 目录。

```
rmr /kafka
```

步骤4 确认目录是否删除。

```
ls /
```

```
[zk: localhost:2181(CONNECTED) 5] ls /  
[registry, zookeeper, hbase-unsecure, hiveserver2, hiveserver2-leader, rmstore, kafka, atsv2-hbase-unsecure, ambari-metrics-cluster]  
[zk: localhost:2181(CONNECTED) 6] rmr /kafka  
[zk: localhost:2181(CONNECTED) 7] ls /  
[registry, zookeeper, hbase-unsecure, hiveserver2, hiveserver2-leader, rmstore, atsv2-hbase-unsecure, ambari-metrics-cluster]
```

----结束

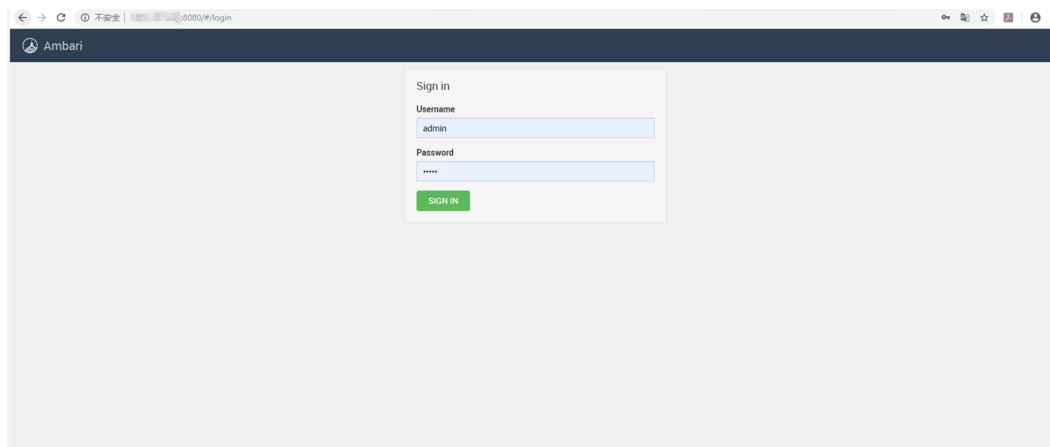
10 部署 Knox

10.1 部署Knox服务

10.2 删除Knox服务

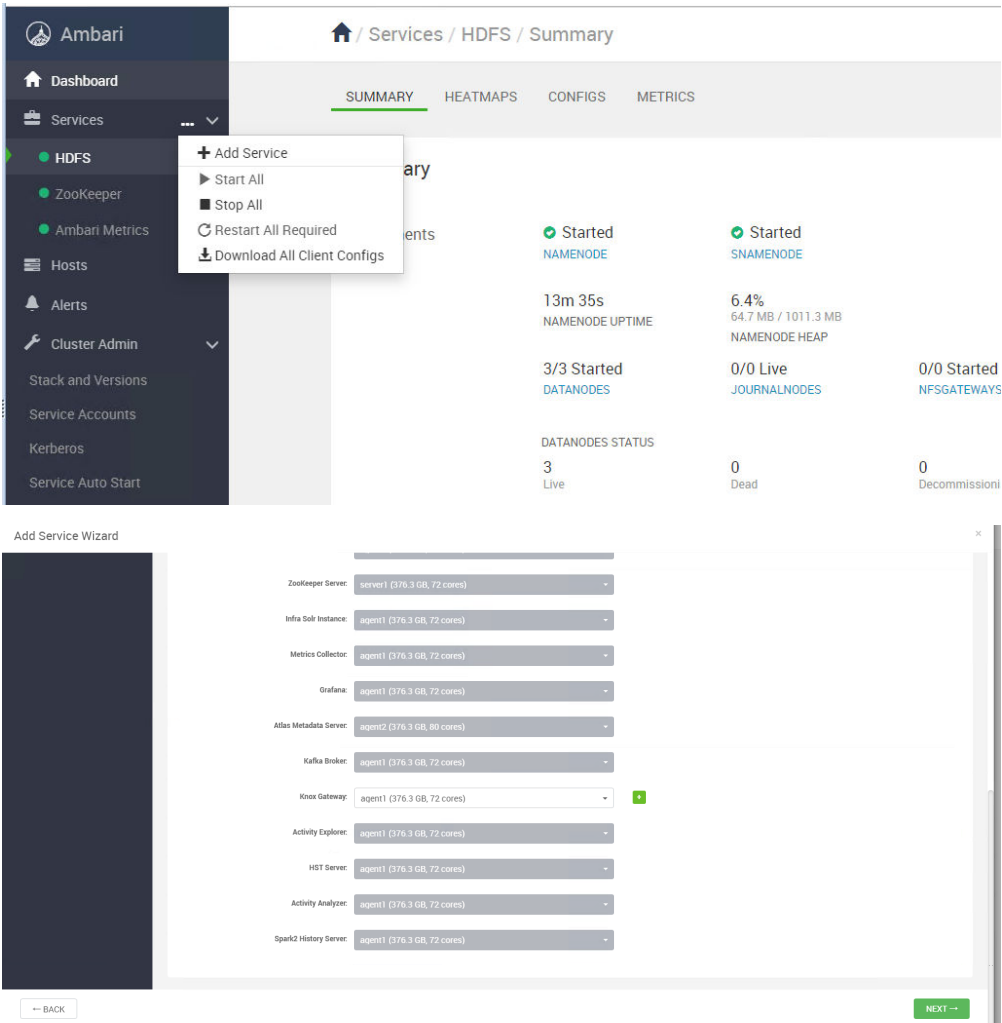
10.1 部署 Knox 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

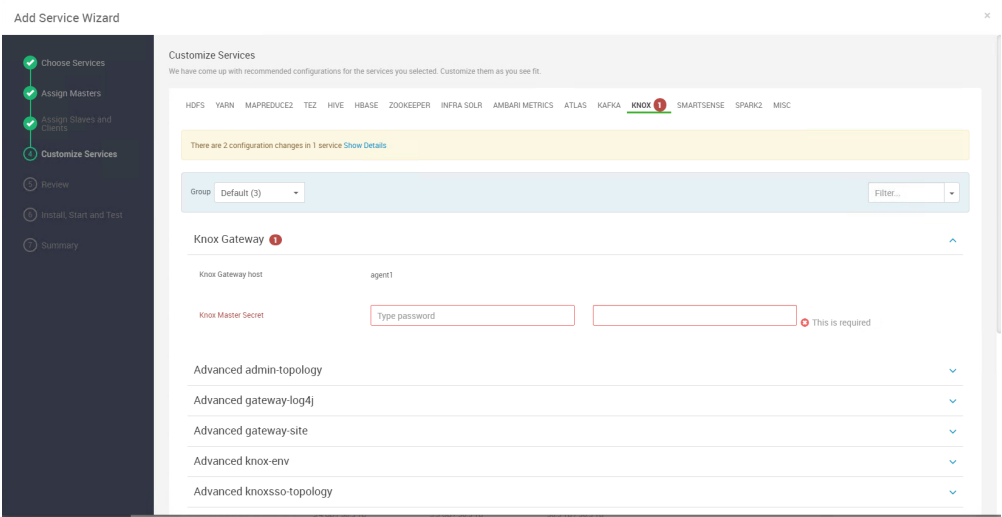


步骤2 添加Knox服务。

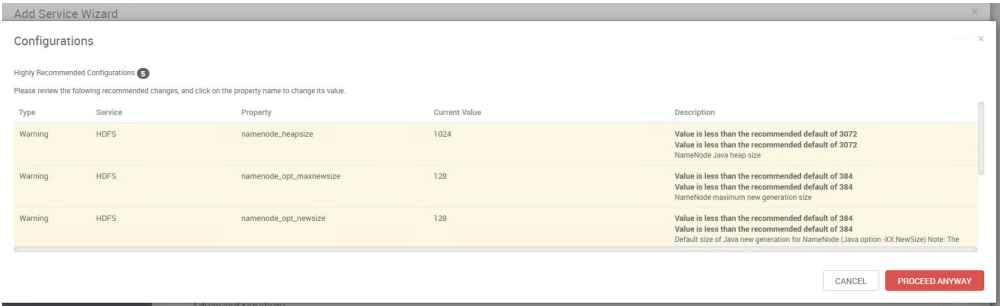
1. 单击Add Service，选择Knox。



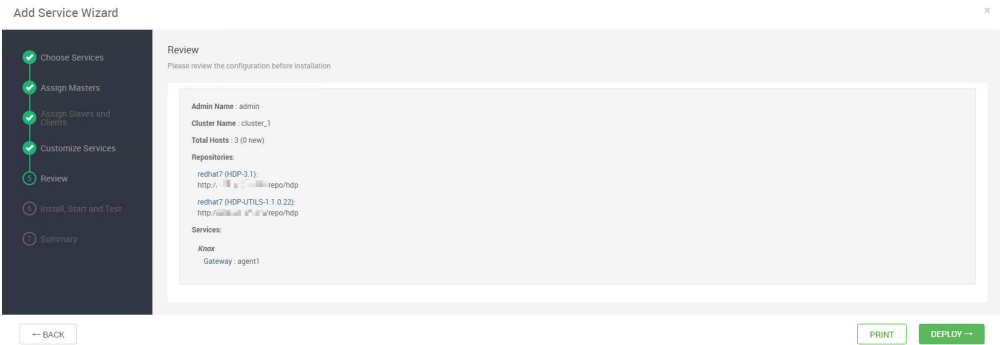
2. 设置Knox Master Secret密码。



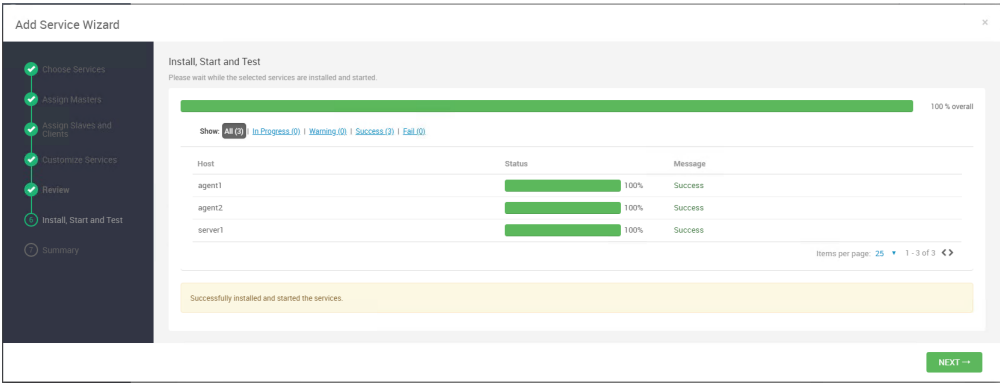
3. 如有弹框，跳过即可，不影响部署，也可按照弹框提示对参数进行修改。



4. 单击DEPLOY。



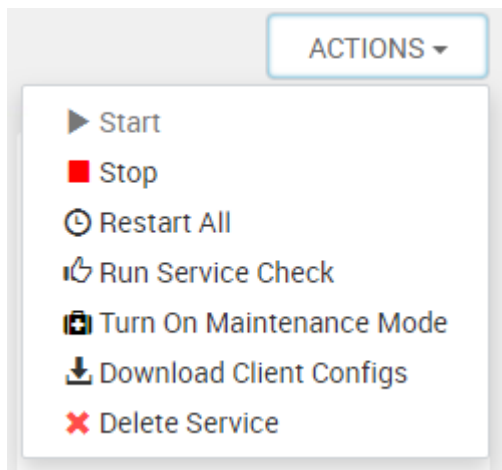
5. 单击NEXT，等待安装完成。



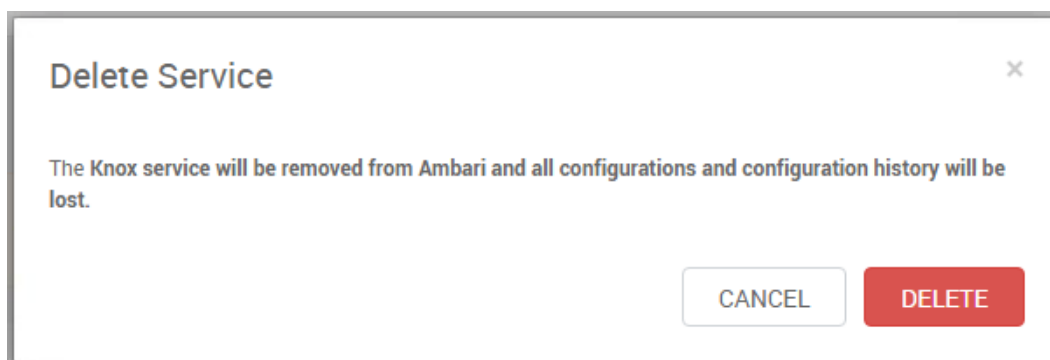
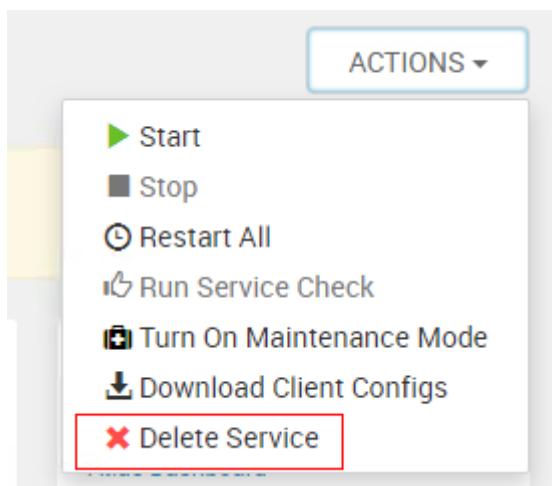
-----结束

10.2 删除 Knox 服务

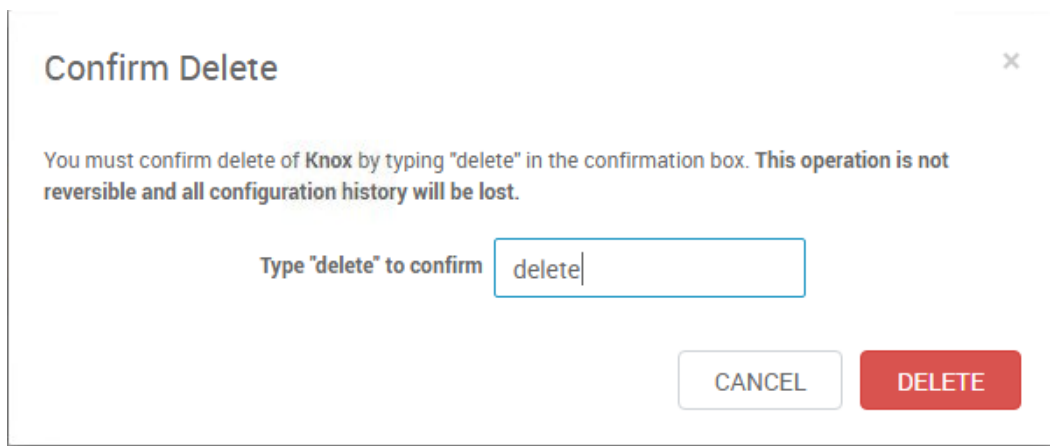
步骤1 在Knox界面单击ACTIONS，选择Stop，先停止服务。



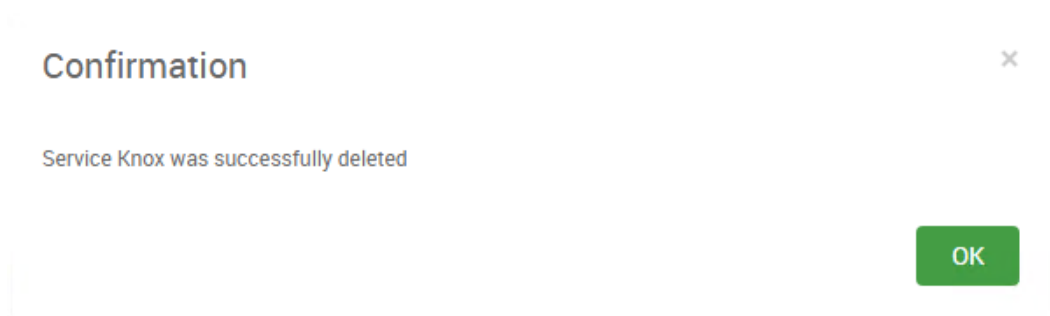
步骤2 停止服务后选择Delete Service，并单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

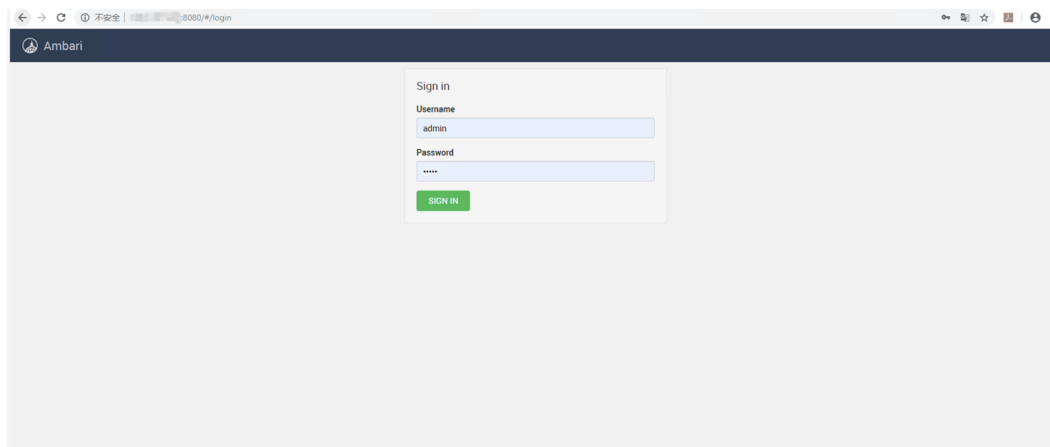
11 部署 Oozie

11.1 部署Oozie服务

11.2 删除Oozie服务

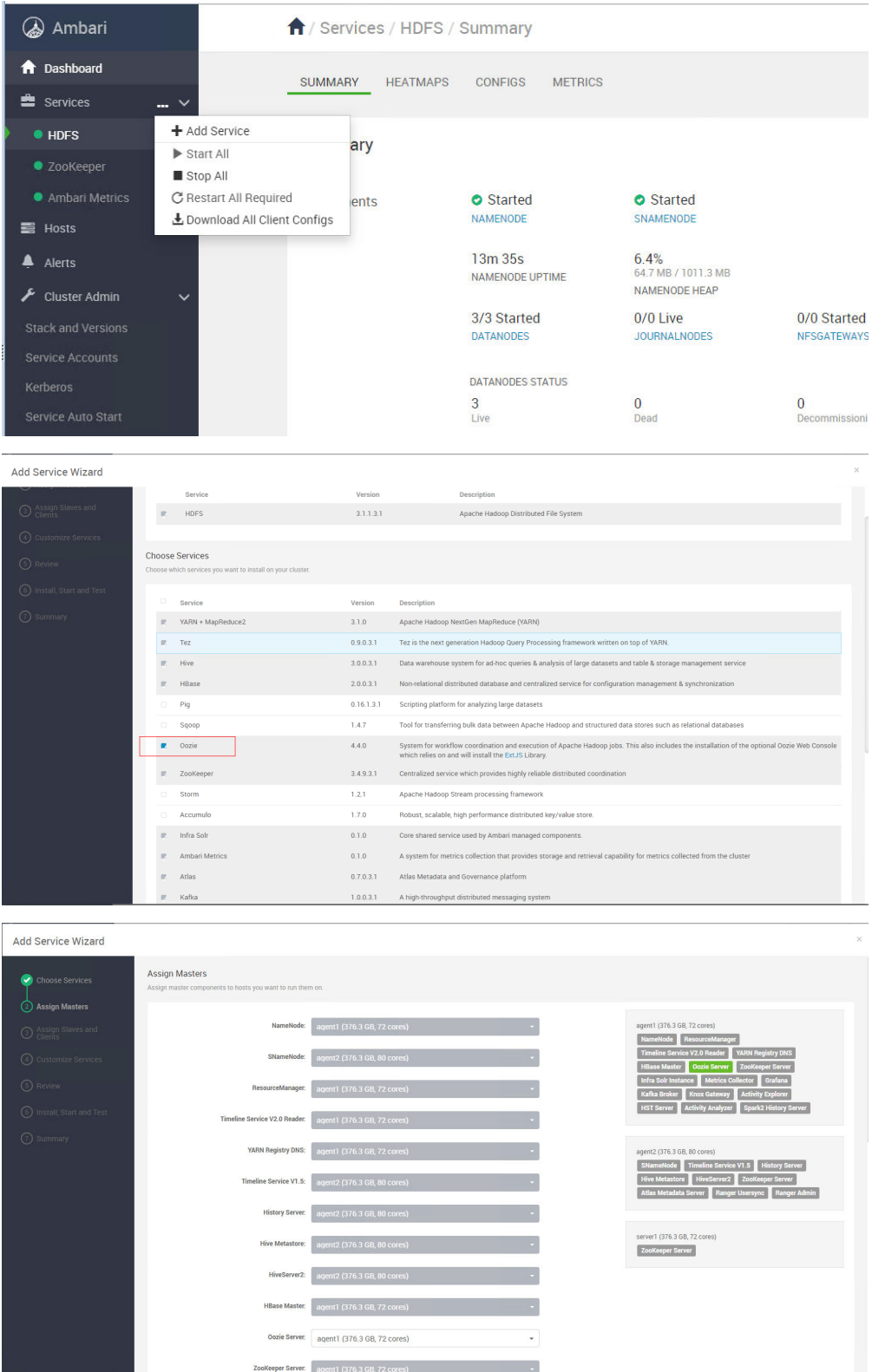
11.1 部署 Oozie 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

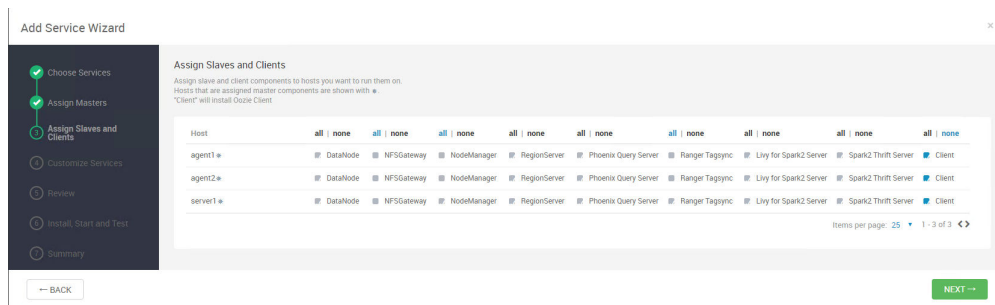


步骤2 添加Oozie服务。

1. 单击Add Service，选择Oozie。



2. 推荐按照下图勾选client。



说明

- 添加Oozie服务时需要再mysql中新建一个database，并在对应的框中填入对应的信息。
- oozie database下拉选择Existing MySQL / Mariadb。
- Username为《Ambari 2.7.3 部署指南（CentOS 7.6&openEuler 20.03）》“[安装数据库](#)”中创建的Ambari，密码为ambari123，Database URL需改成server主机的hostname，这里是server1，根据实际情况填写。

3. 在安装了MariaDB的节点上，登录客户端。

```
[root@server1 yum.repos.d]# mysql -uroot -p
```

Enter password:

Welcome to the MariaDB monitor. Commands end with ; or \g.

Your MariaDB connection id is 4

Server version: 5.5.60-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

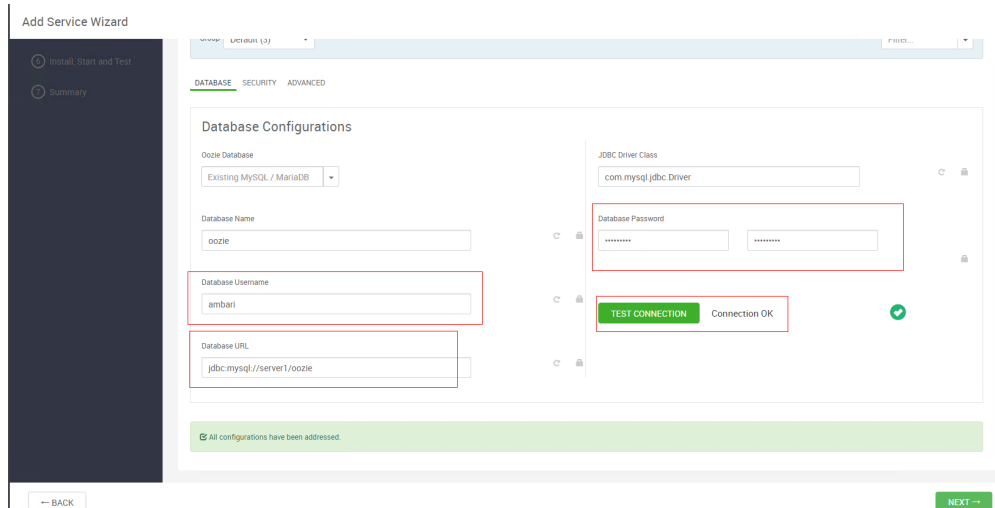
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
MariaDB [(none)]> create database oozie;
```

```
Query OK, 1 row affected (0.00 sec)
```

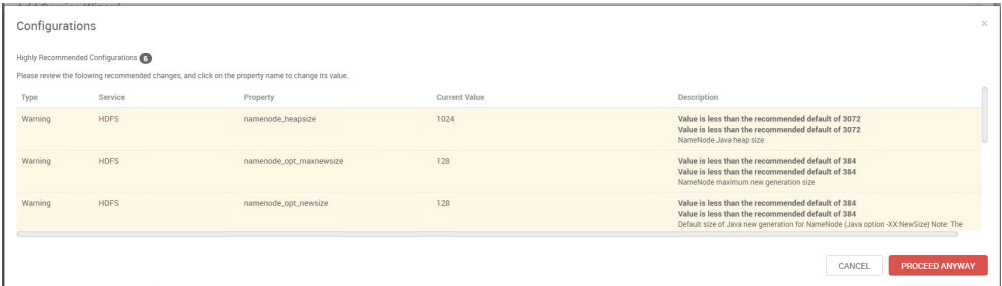
```
MariaDB [(none)]> exit;
```

```
Bye
```

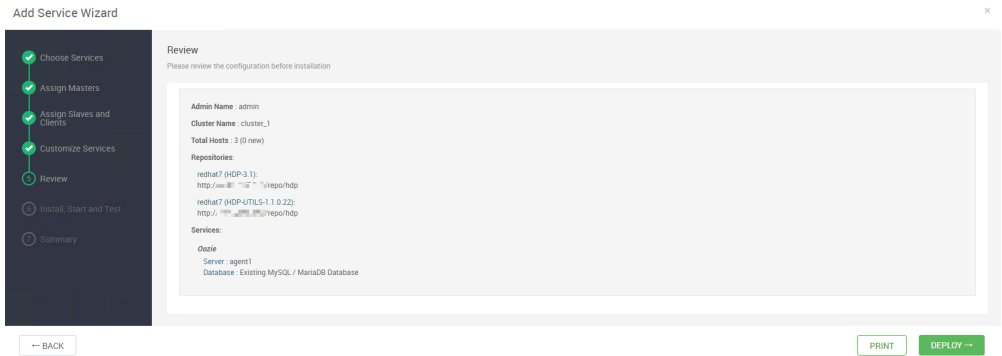


4. Test connection成功后可单击NEXT。

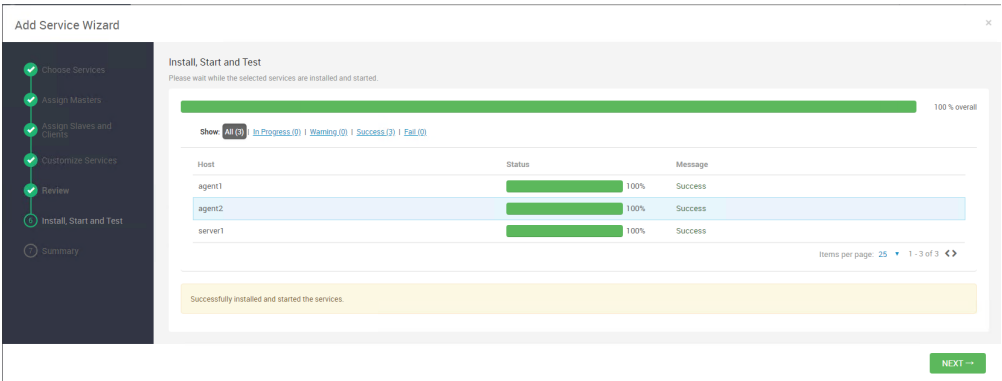
如有弹框，跳过即可，不影响部署，也可按照弹框提示对参数进行修改。



5. 单击DEPLOY。



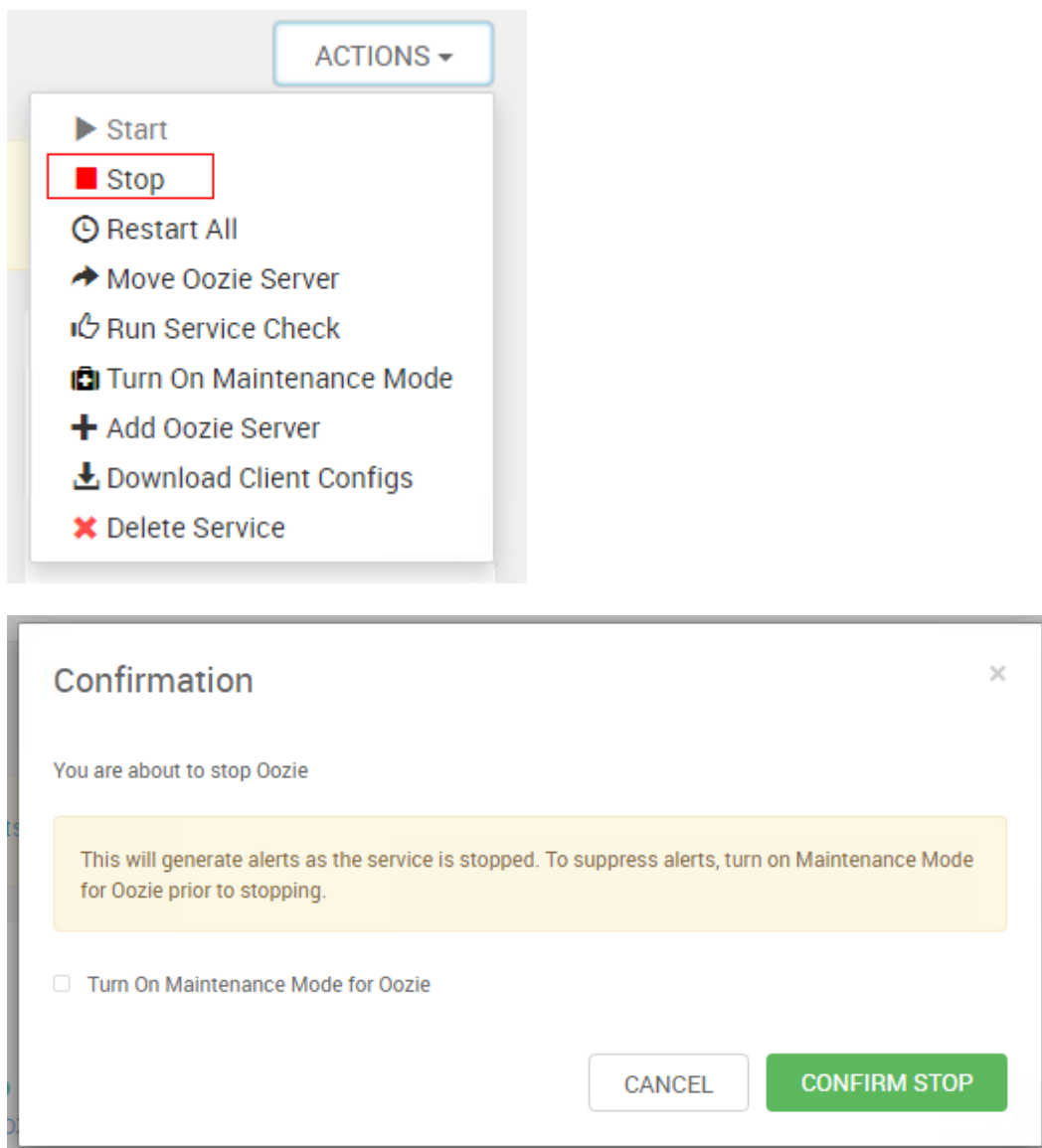
6. 单击NEXT，等待安装完成。



----结束

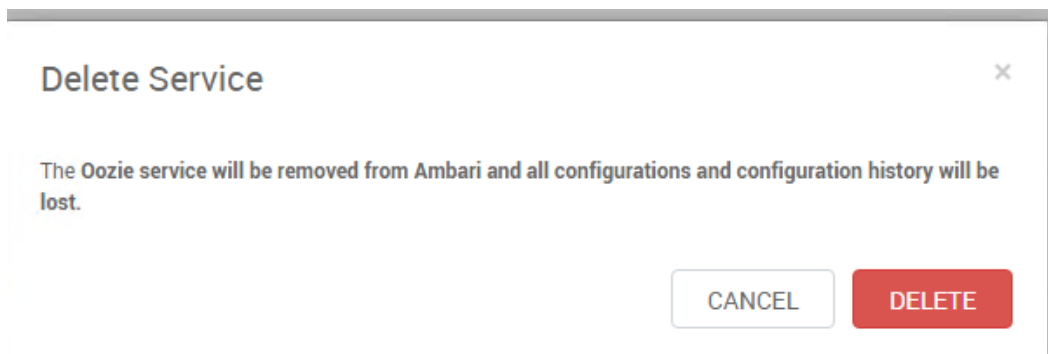
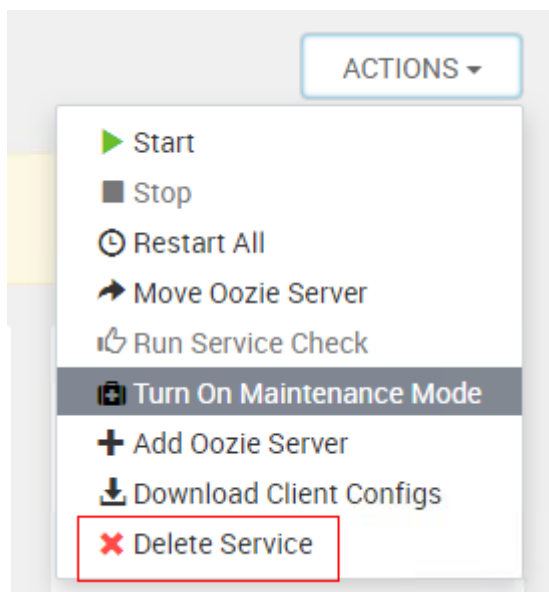
11.2 删除 Oozie 服务

步骤1 在Oozie界面单击ACTIONS，选择Stop，先停止服务。

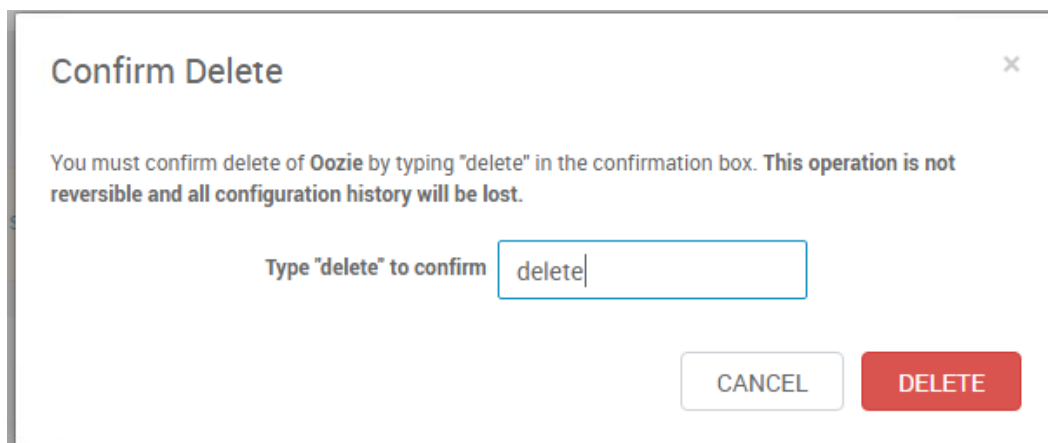


选择CONFIRE STOP。

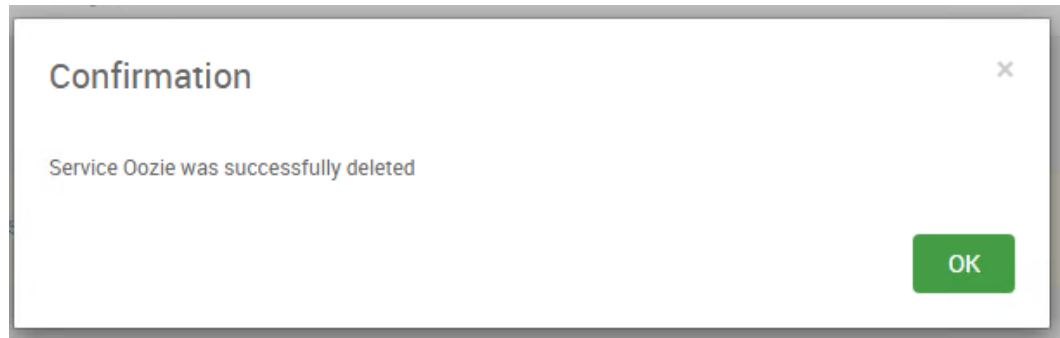
步骤2 停止服务后选择Delete Service，并单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

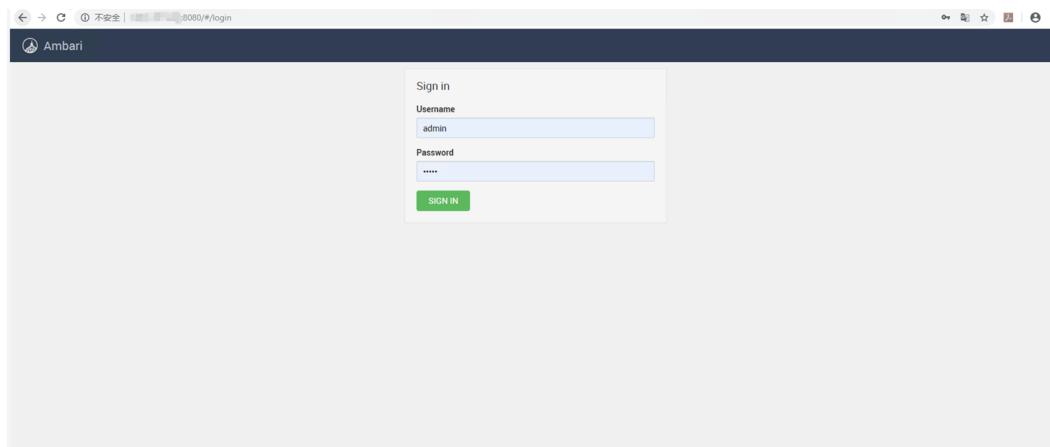
12 部署 Pig

12.1 部署Pig服务

12.2 删除Pig服务

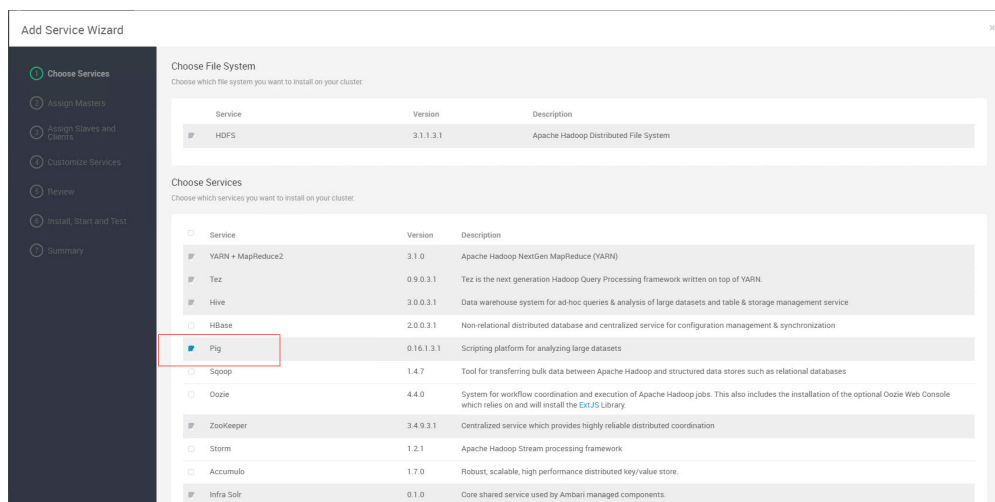
12.1 部署 Pig 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

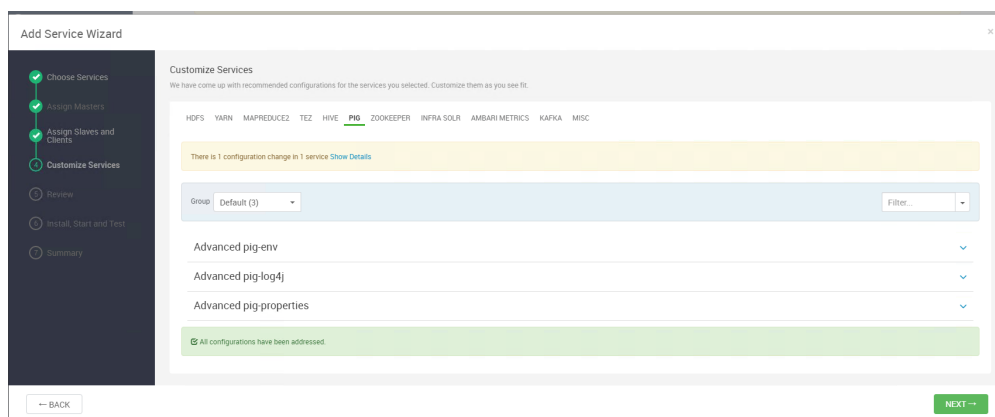
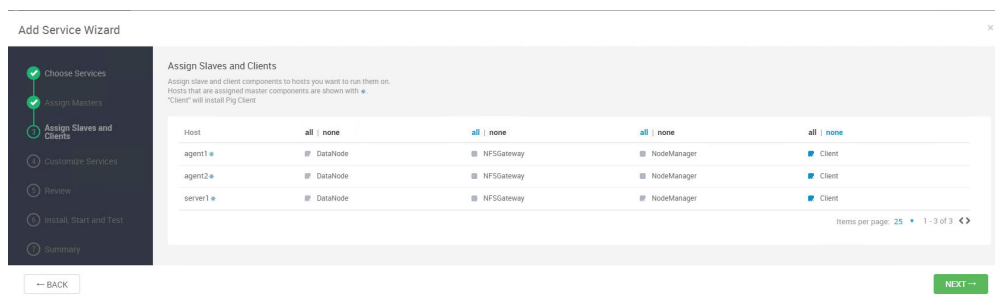


步骤2 添加Pig服务。

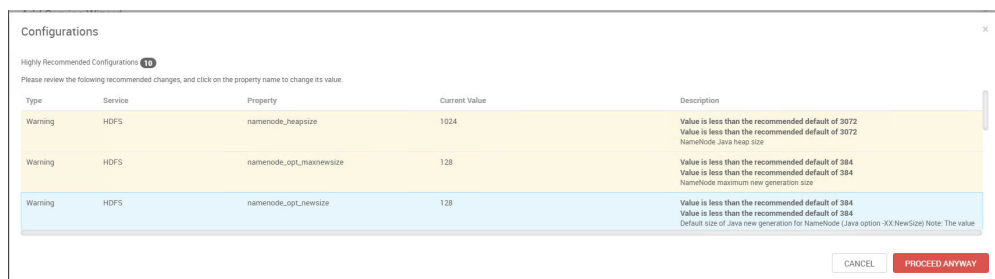
1. 单击Add Service，选择Pig。



2. 推荐按照下图勾选。



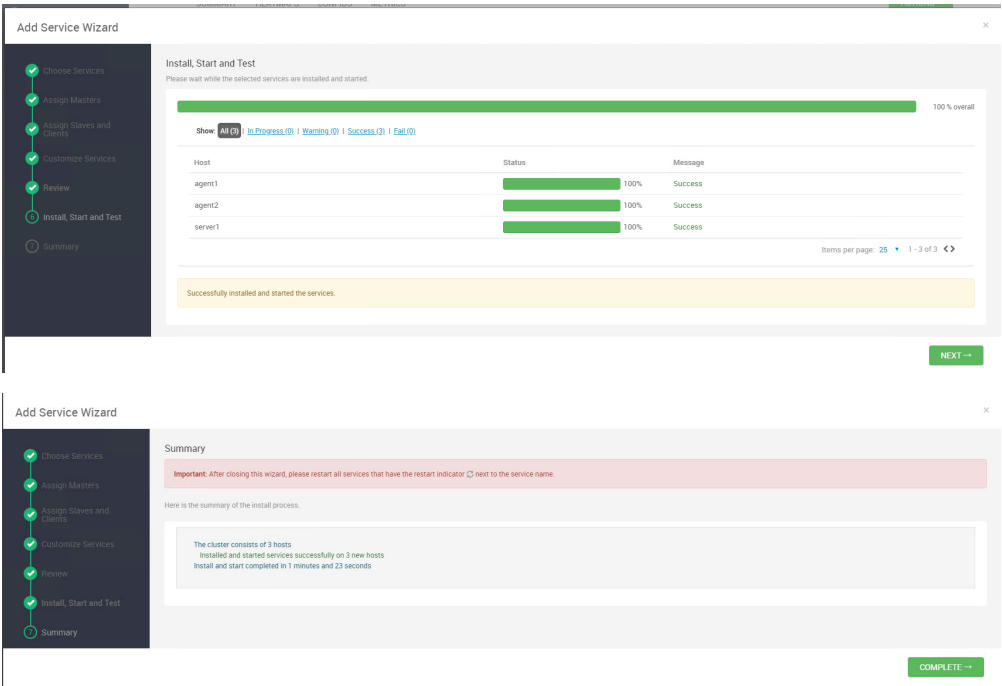
3. 如有弹框，跳过即可，不影响部署，也可按照弹框提示对参数进行修改。



4. 单击DEPLOY。



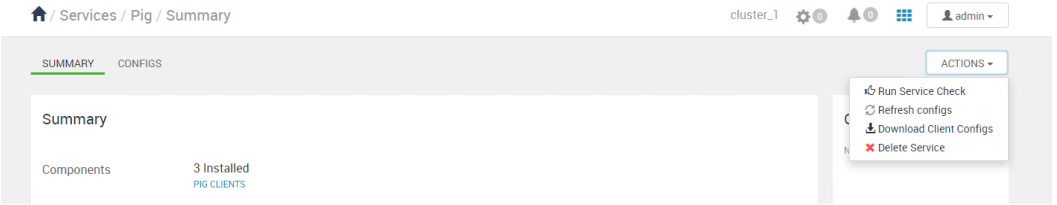
5. 单击NEXT，等待安装完成。



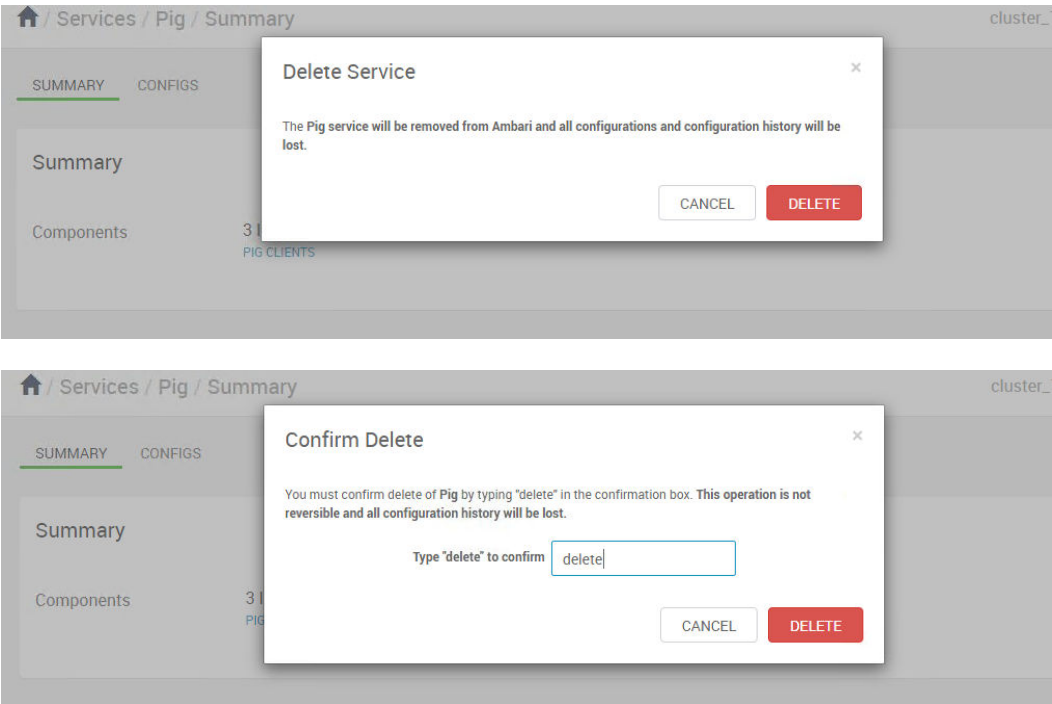
-----结束

12.2 删除 Pig 服务

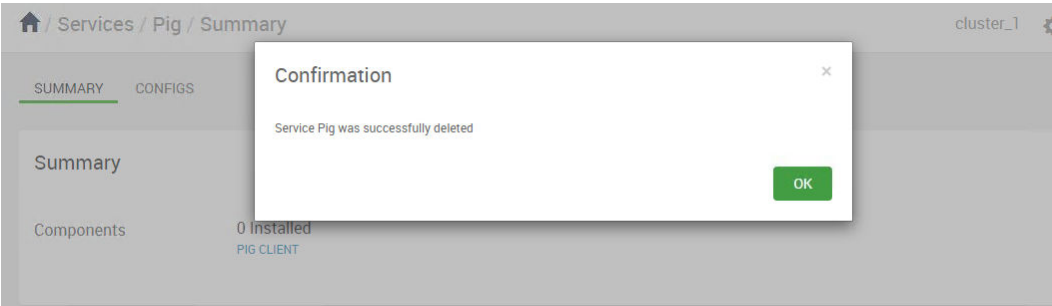
步骤1 选择Delete Service。



步骤2 在对话框中输入delete，并单击DELETE。



步骤3 删除完成后单击OK。



----结束

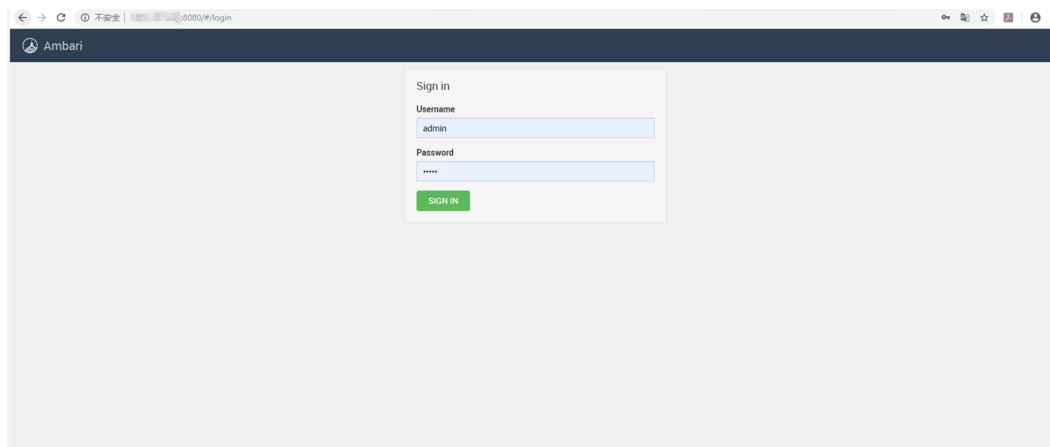
13 部署 Ranger

13.1 部署Ranger服务

13.2 删除Ranger服务

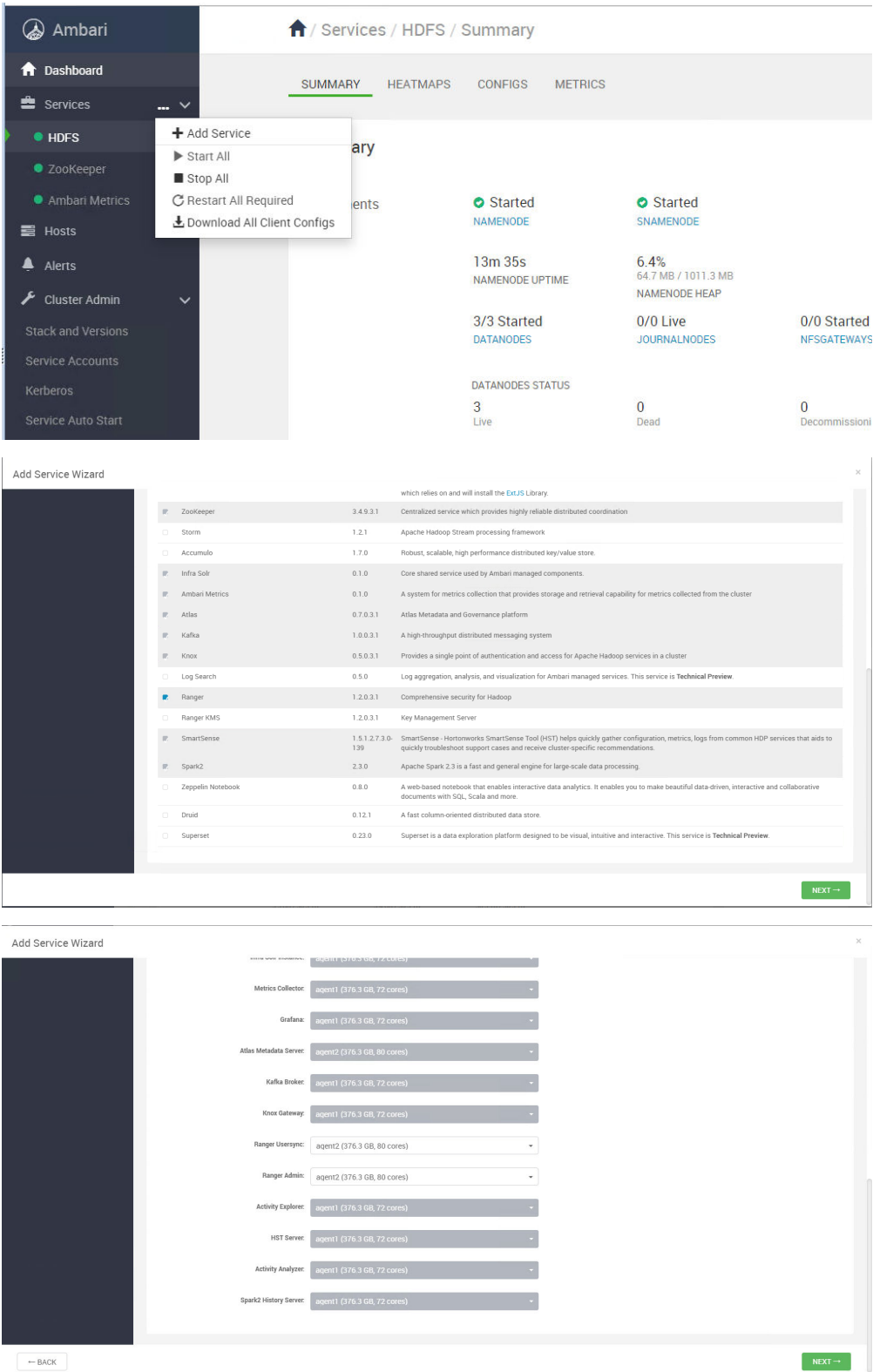
13.1 部署 Ranger 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

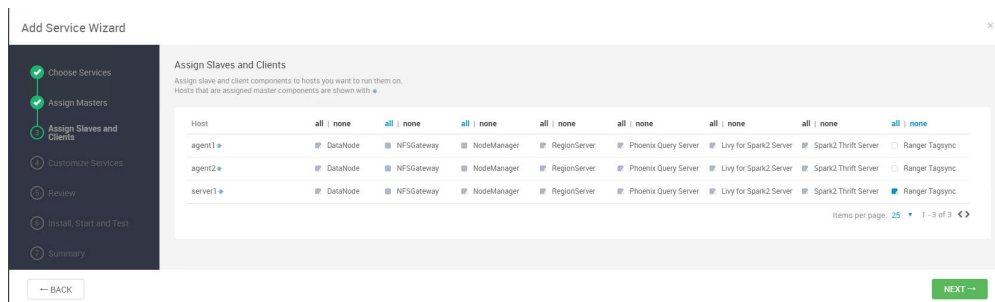


步骤2 添加Ranger服务。

1. 单击Add Service，选择Ranger。



2. 推荐按照下图勾选。



说明

- 添加Ranger服务时需要在mysql中新建一个database，并在对应的框中填入对应的信息。
- DB FLAVOR下拉选择MySQL。
- Username为《Ambari 2.7.3 部署指南（CentOS 7.6&openEuler 20.03）》“[安装数据库](#)”中创建的Ambari，密码为ambari123，Database URL需改成server主机的hostname，这里是server1，根据实际情况填写。

3. 在安装了MariaDB的节点上，登录客户端。

```
[root@server1 yum.repos.d]# mysql -uroot -p
```

Enter password:

Welcome to the MariaDB monitor. Commands end with ; or \g.

Your MariaDB connection id is 4

Server version: 5.5.60-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

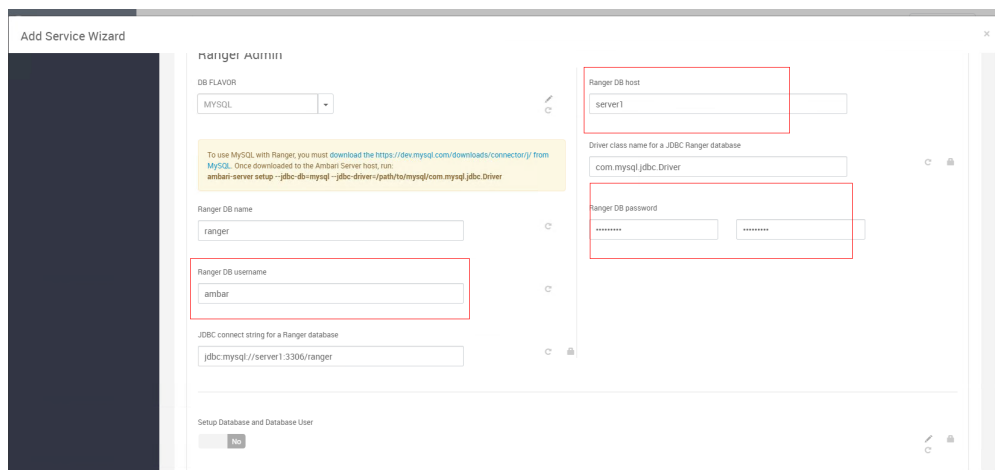
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
MariaDB [(none)]> create database ranger;
```

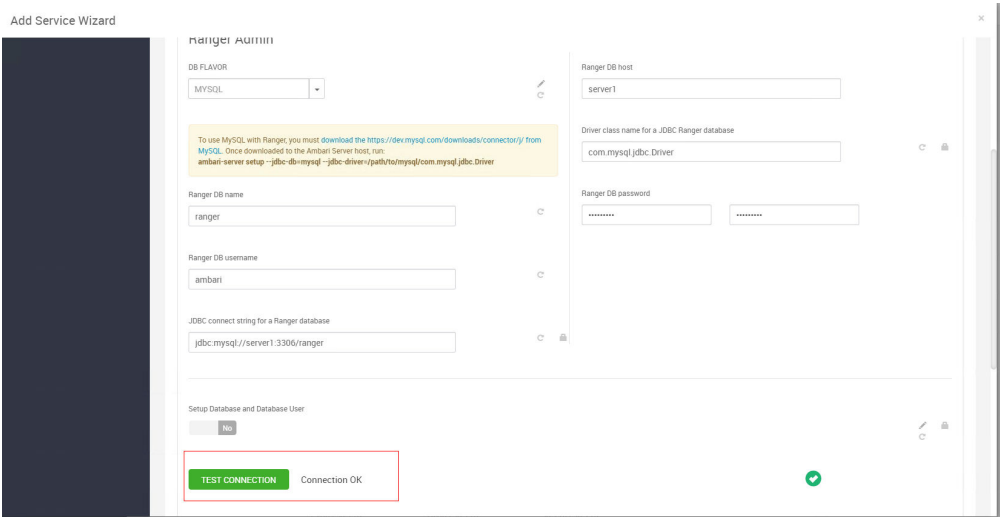
```
Query OK, 1 row affected (0.00 sec)
```

```
MariaDB [(none)]> exit;
```

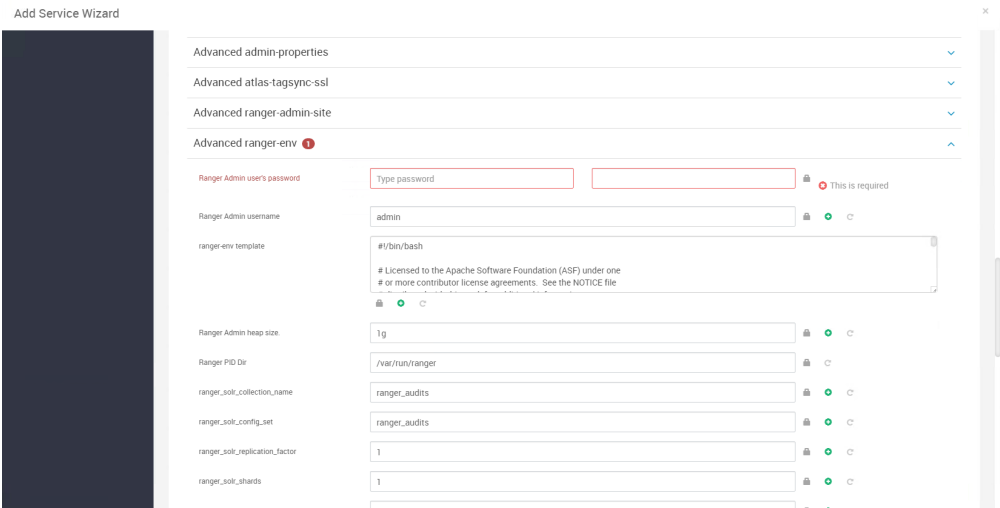
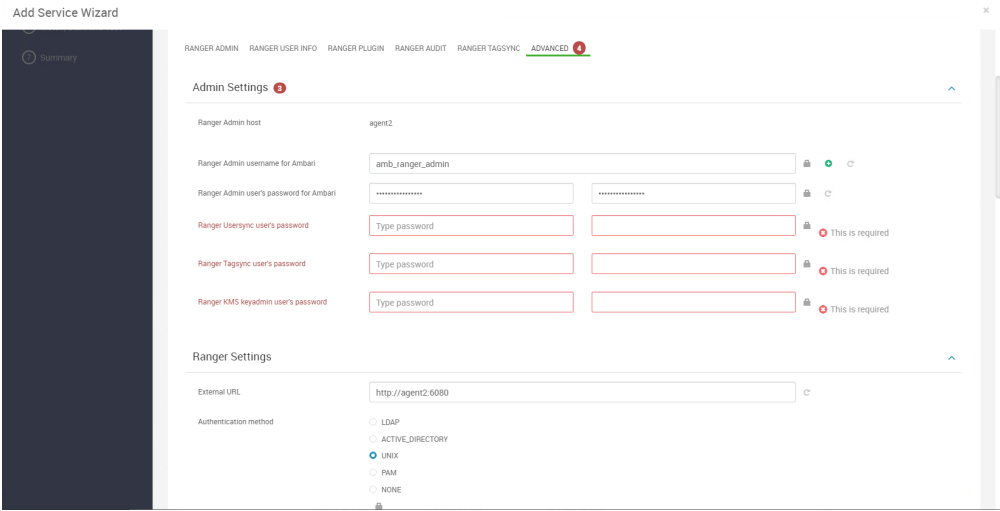
```
Bye
```



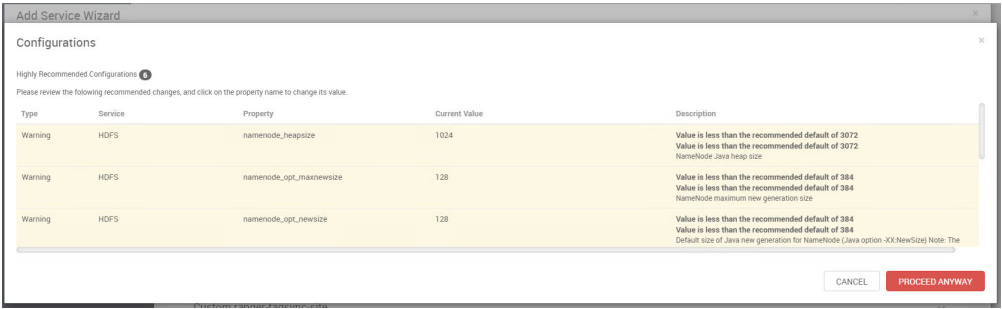
4. Test connection成功后可单击NEXT。



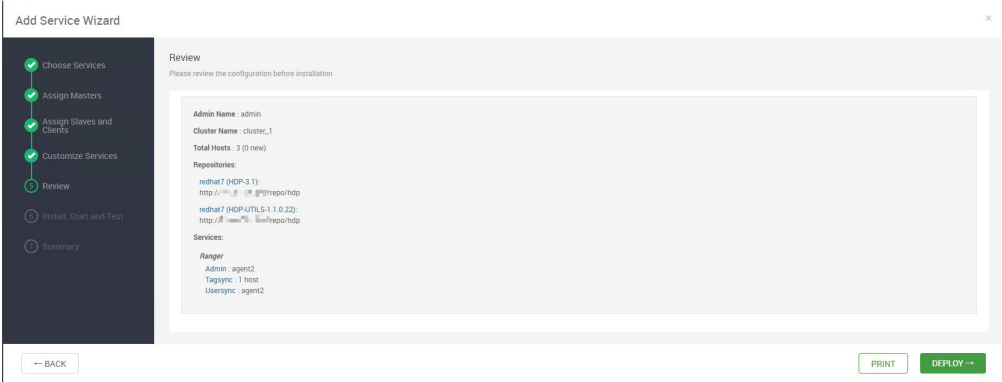
5. 设置密码。



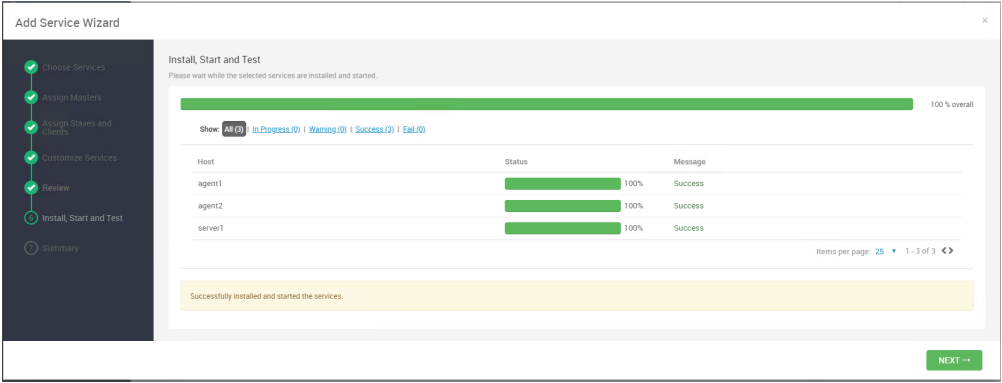
6. 如有弹框，跳过即可，不影响部署，也可按照弹框提示对参数进行修改。



7. 单击DEPLOY。



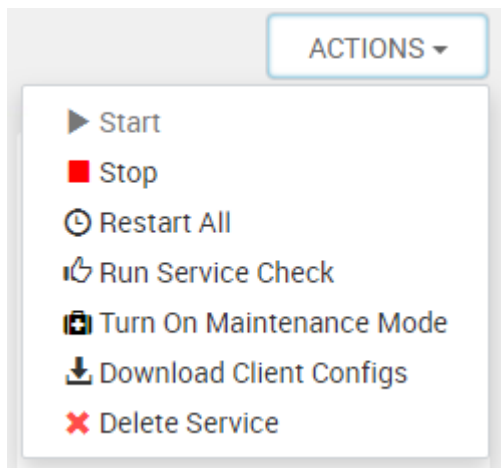
8. 单击NEXT，等待安装完成。



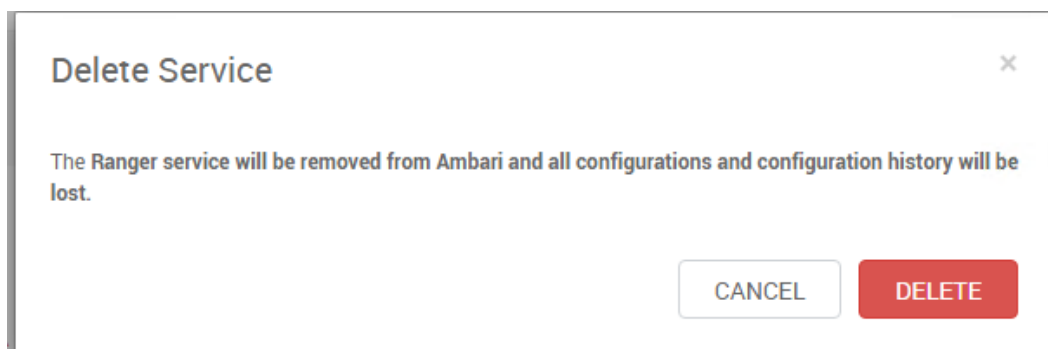
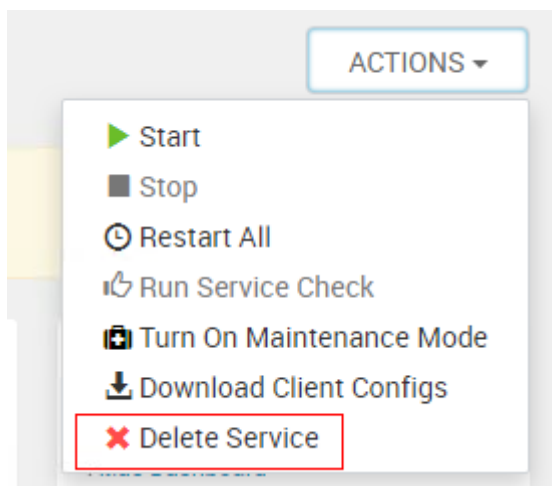
----结束

13.2 删除 Ranger 服务

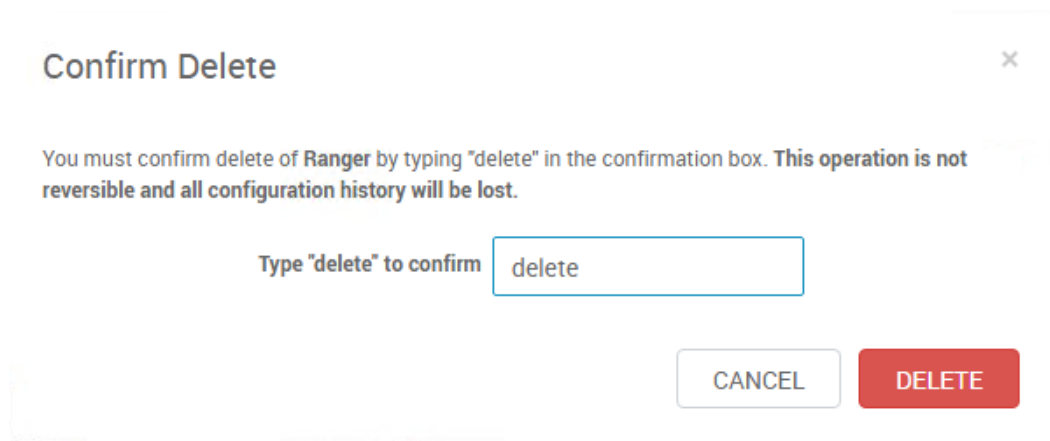
步骤1 在ranger界面单击ACTIONS，选择Stop，先停止服务。



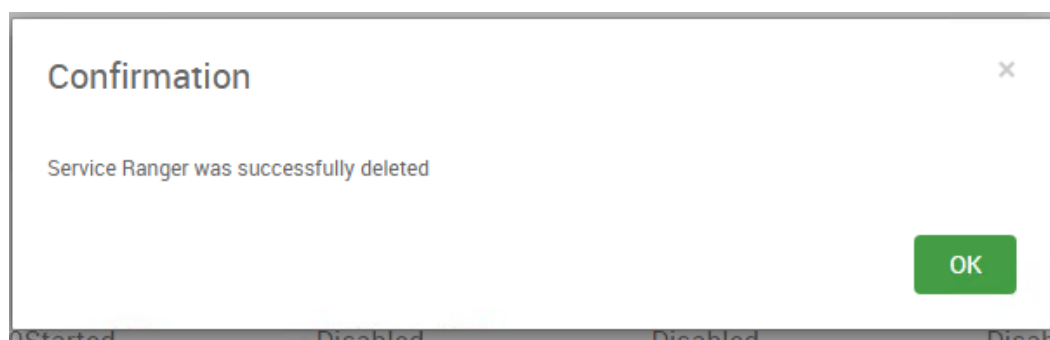
步骤2 停止服务后选择Delete Service，并单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

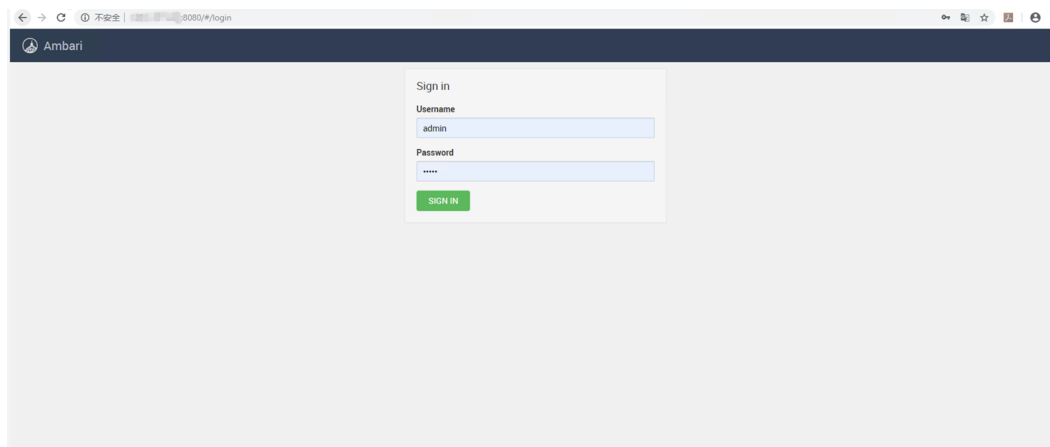
14 部署 Spark&Livy

[14.1 部署Spark&Livy服务](#)

[14.2 删除Spark&Livy服务](#)

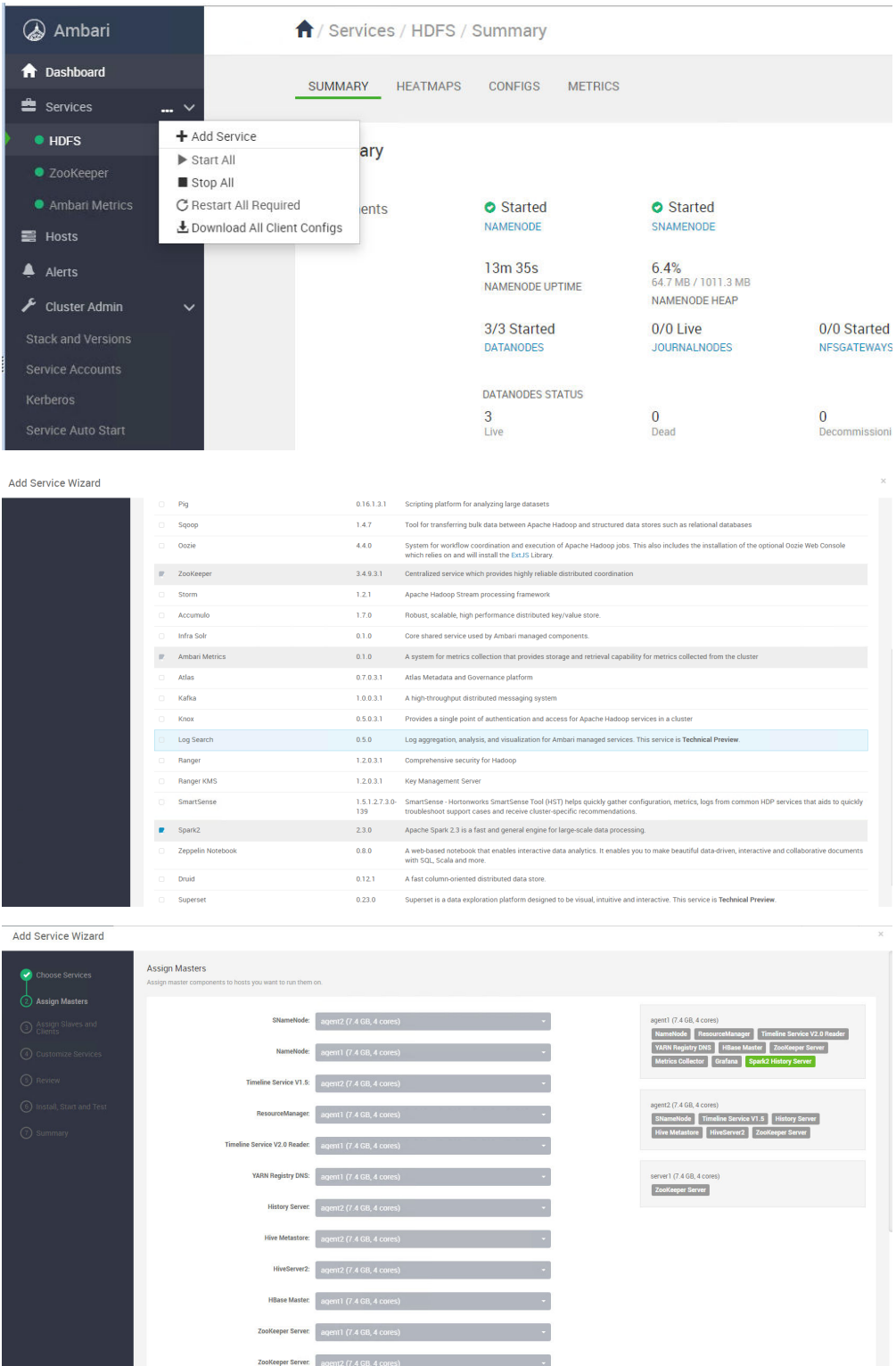
14.1 部署 Spark&Livy 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

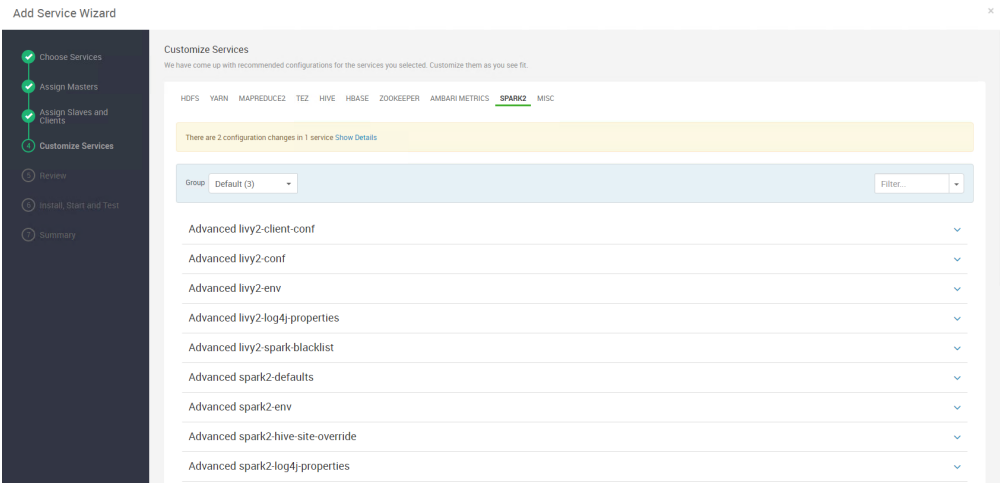
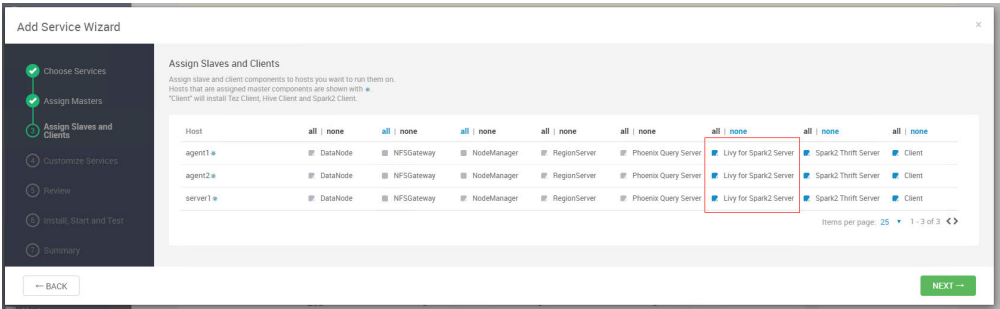


步骤2 添加Spark服务。

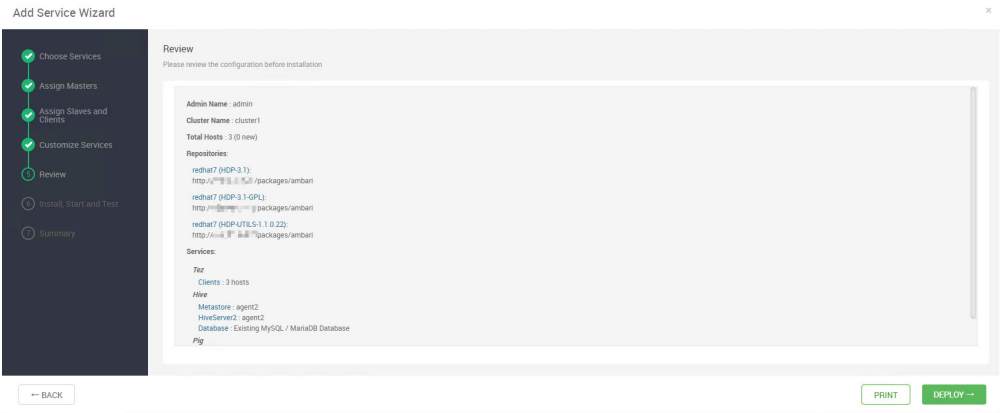
1. 单击Add Service，选择Spark。



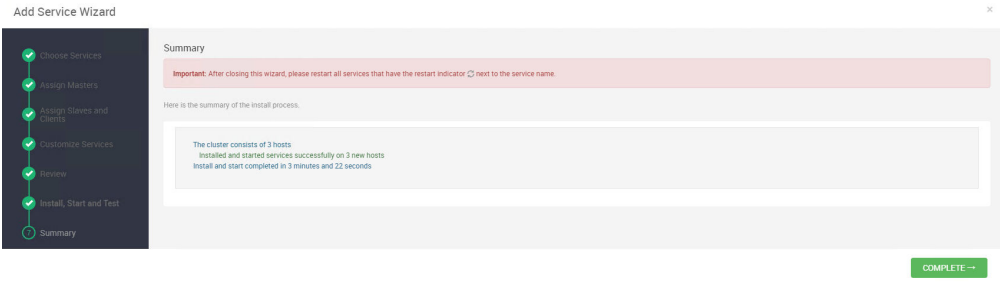
2. 在Client和Spark2 Thrift Server的基础上勾选Livy for Spark2 Server。



3. 单击DEPLOY。



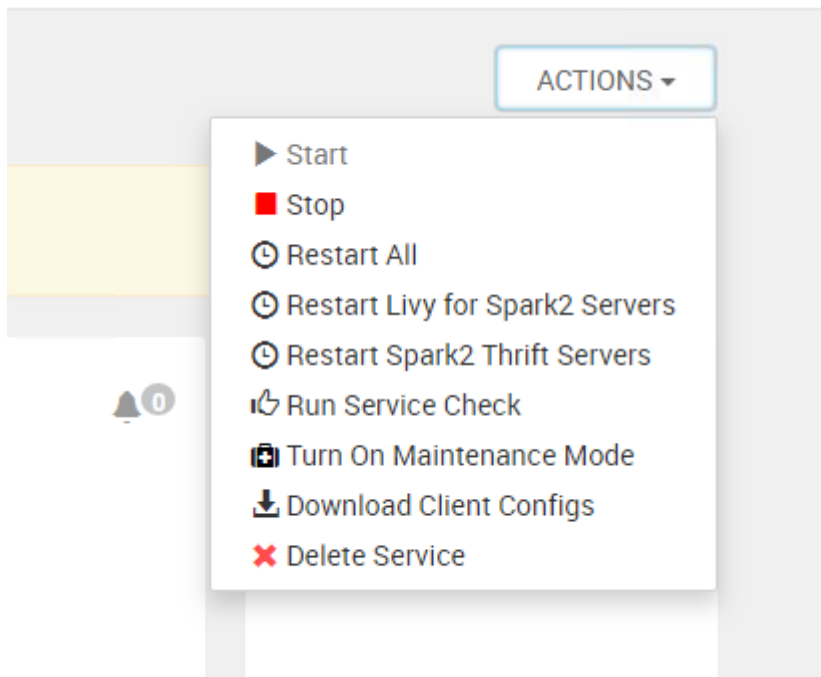
4. 单击NEXT，等待安装完成。



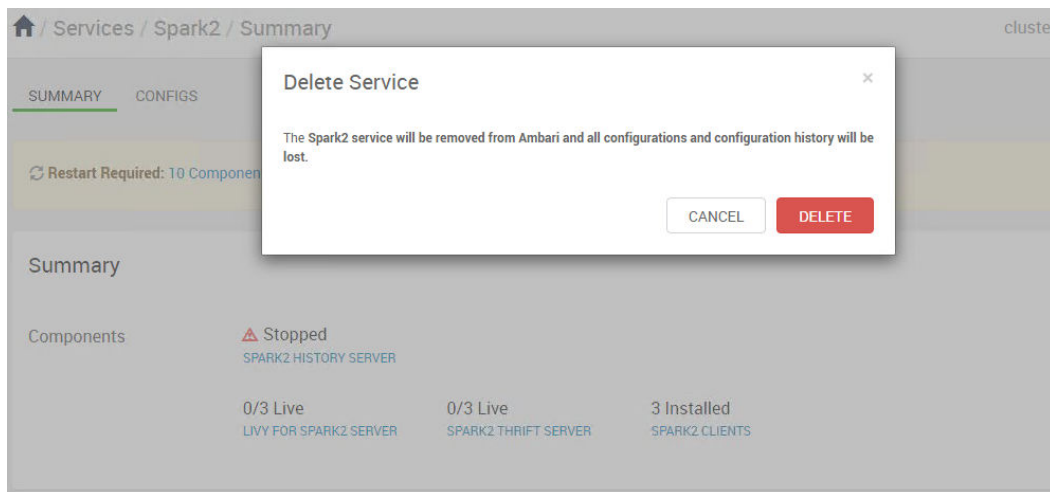
----结束

14.2 删除 Spark&Livy 服务

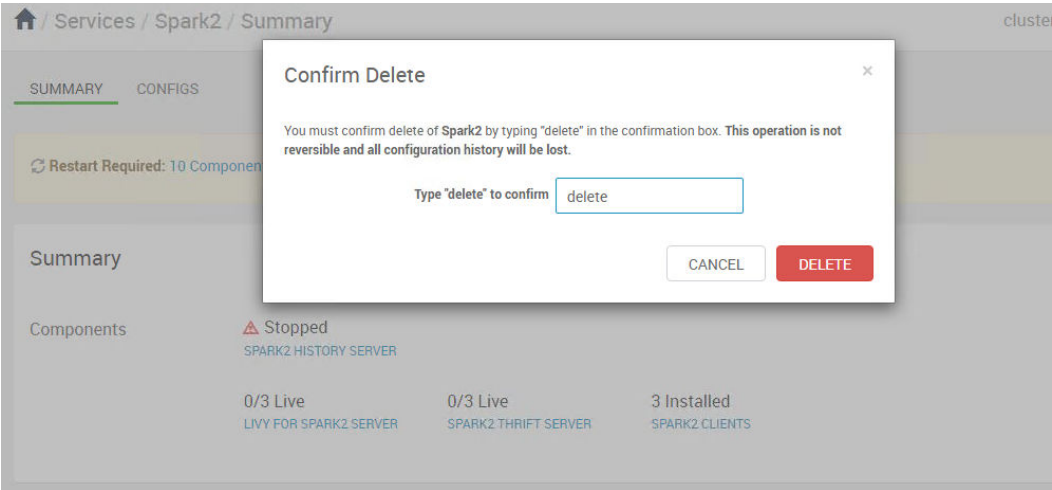
步骤1 在Spark界面单击ACTIONS，选择Stop，先停止服务。



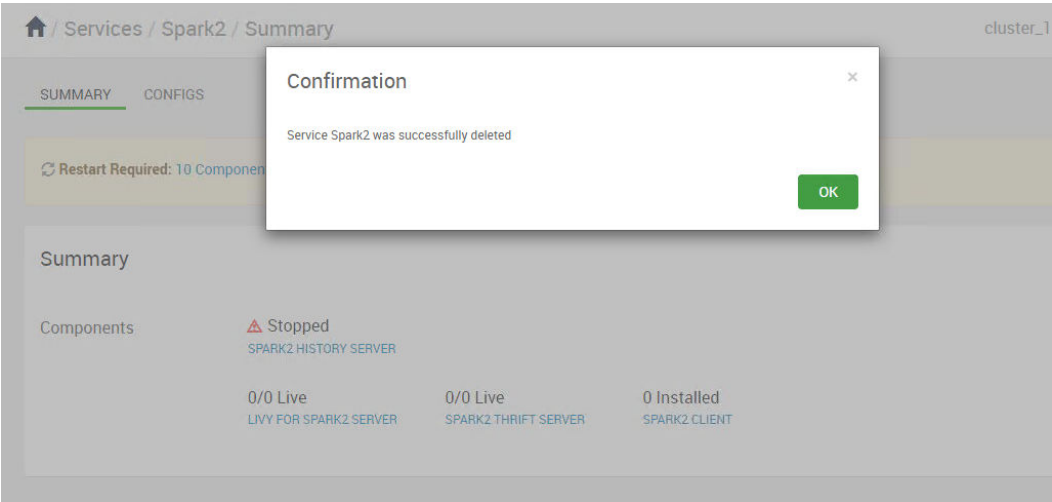
步骤2 停止服务后选择Delete Service，单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

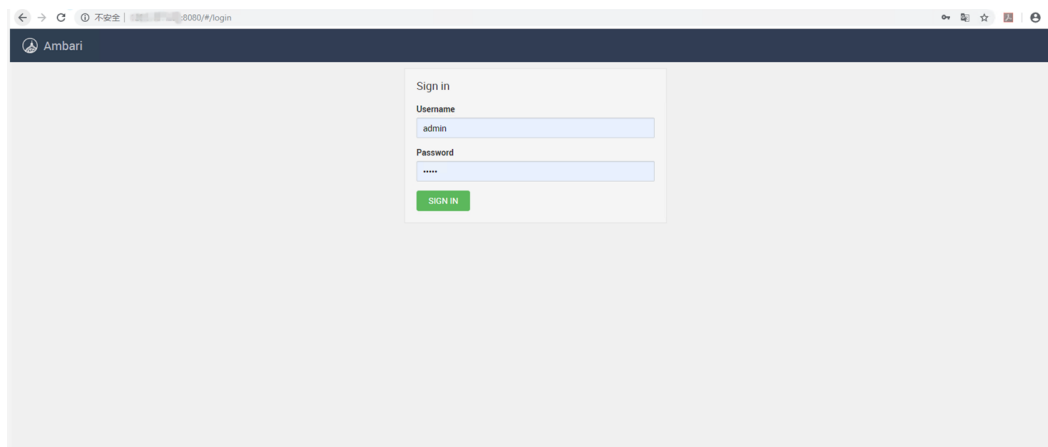
15 部署 Sqoop

[15.1 部署Sqoop服务](#)

[15.2 删除Sqoop服务](#)

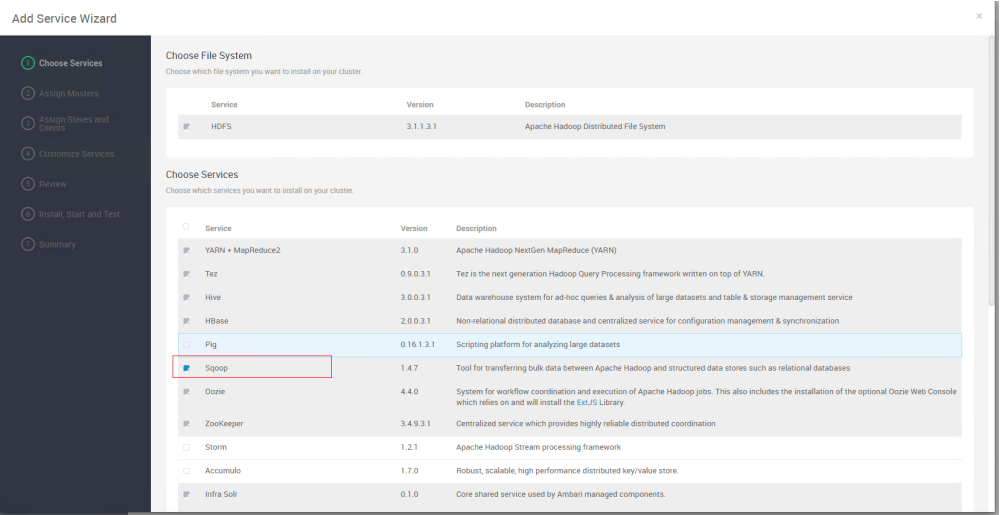
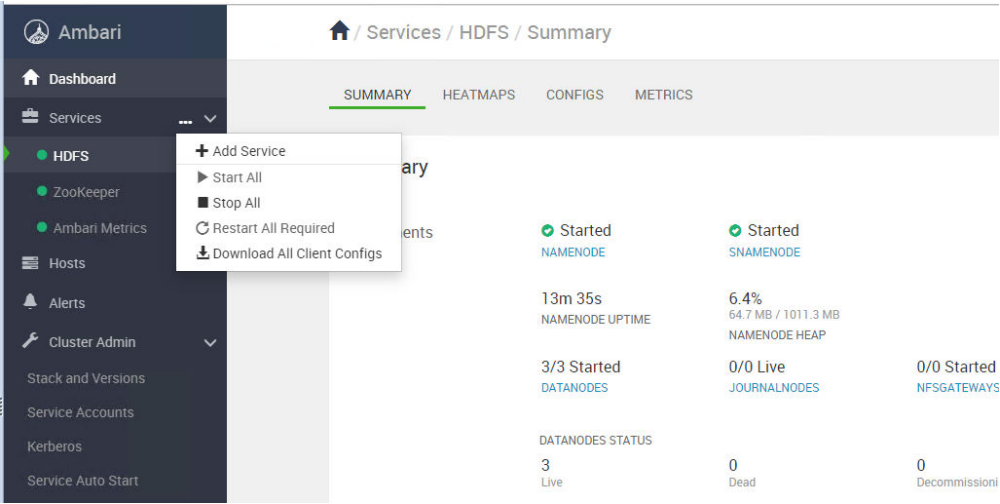
15.1 部署 Sqoop 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

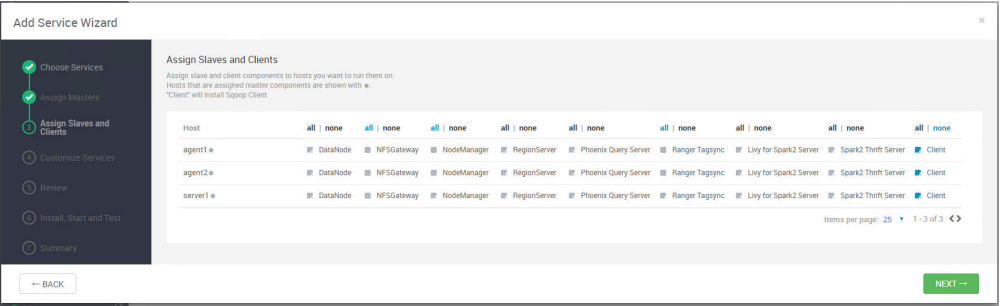


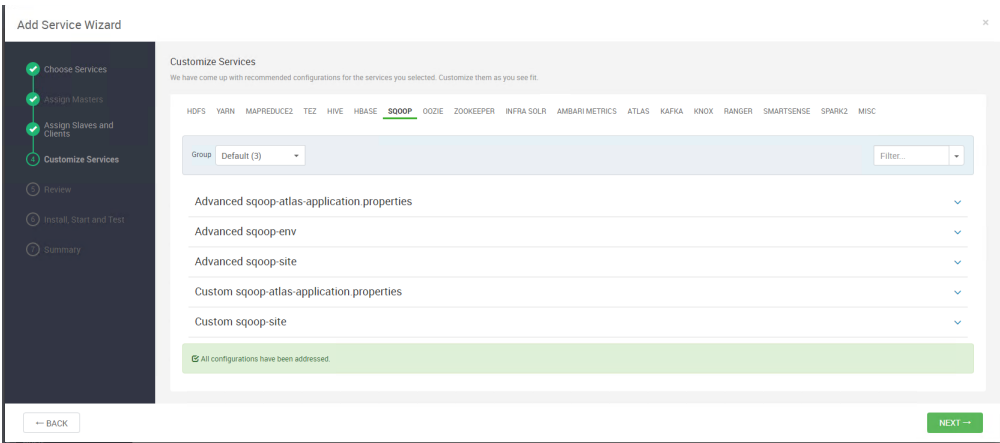
步骤2 添加Sqoop服务。

1. 单击Add Service，选择Sqoop。

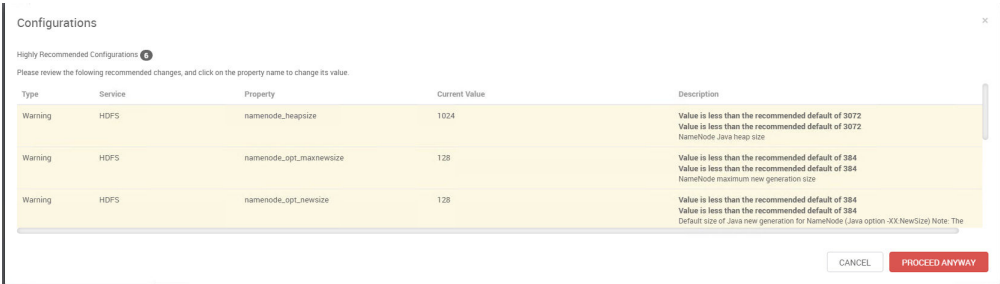


2. 推荐按照下图勾选client。

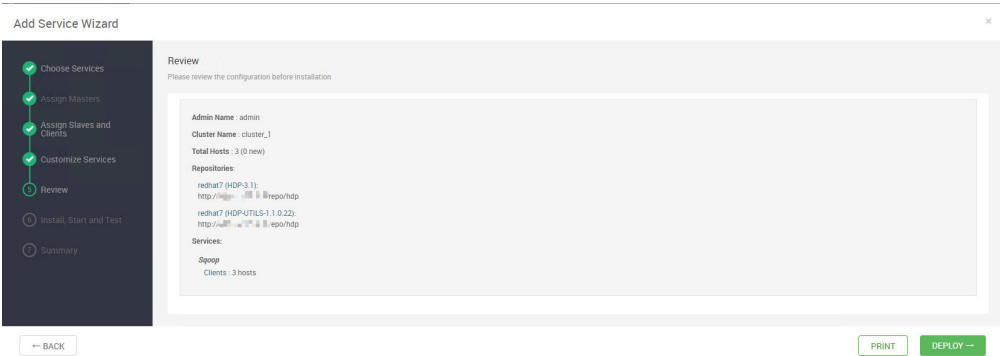




3. 如有弹框，跳过即可，不影响部署，也可按照弹框提示对参数进行修改。



4. 单击DEPLOY。

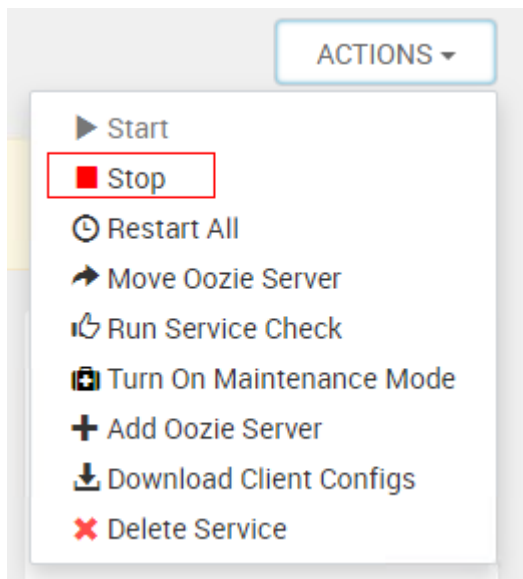


5. 单击NEXT，等待安装完成。

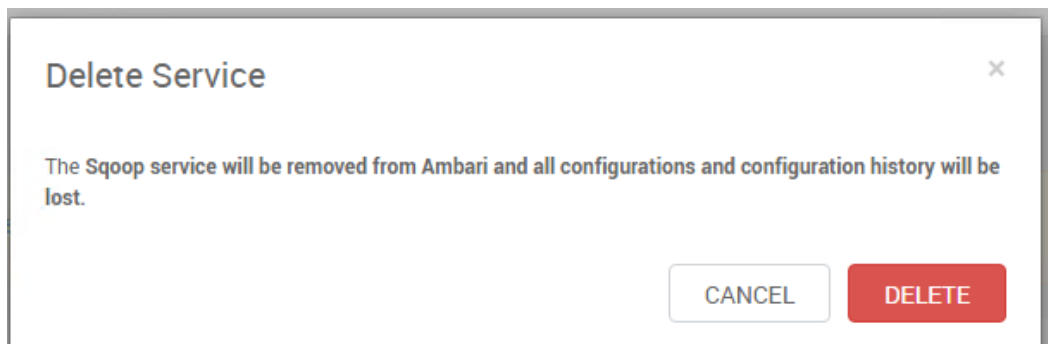
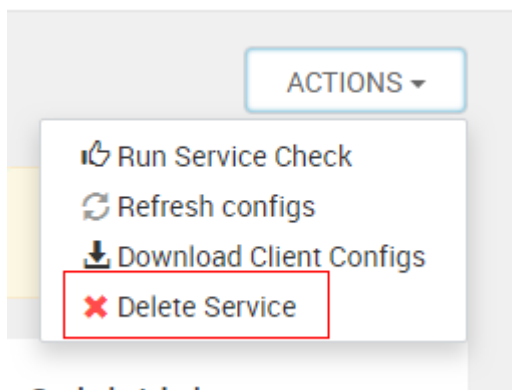
-----结束

15.2 删除 Sqoop 服务

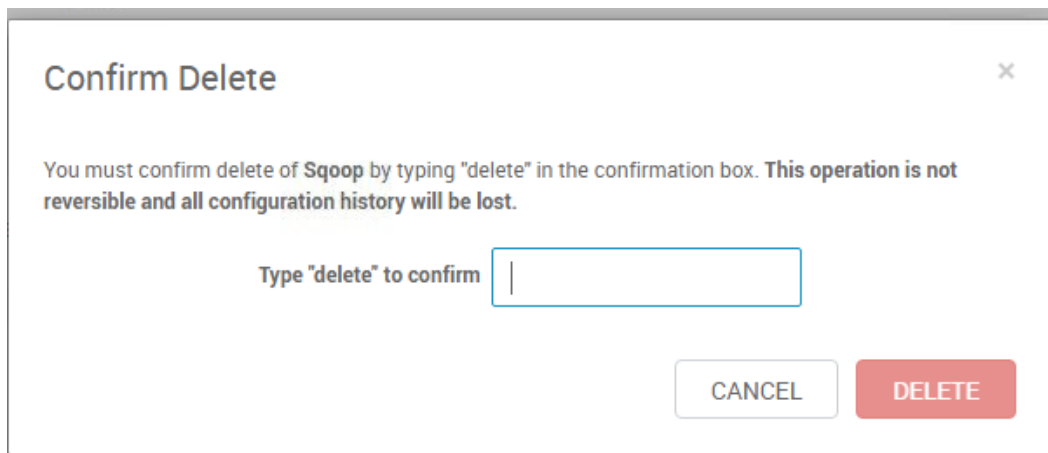
步骤1 在Sqoop界面单击ACTIONS，选择Stop，先停止服务。



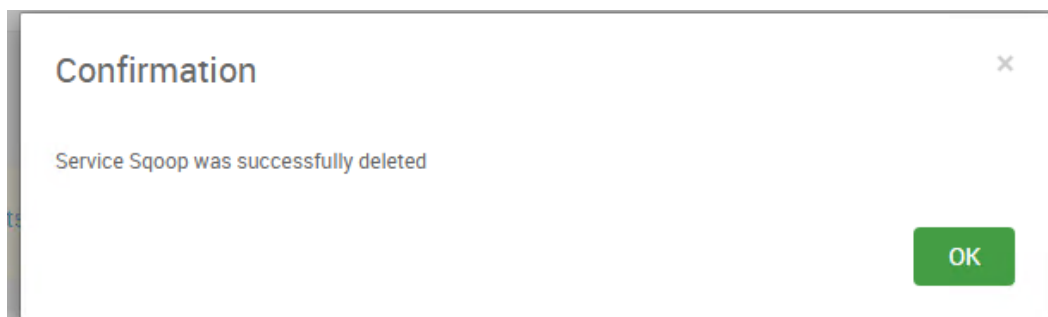
步骤2 停止服务后选择Delete Service，并单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

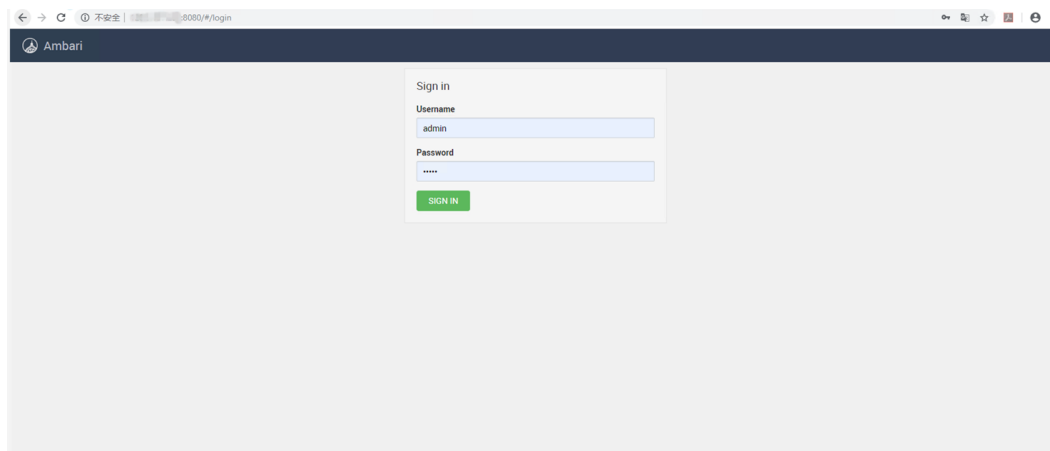
16 部署 Storm

16.1 部署Storm服务

16.2 删除Storm服务

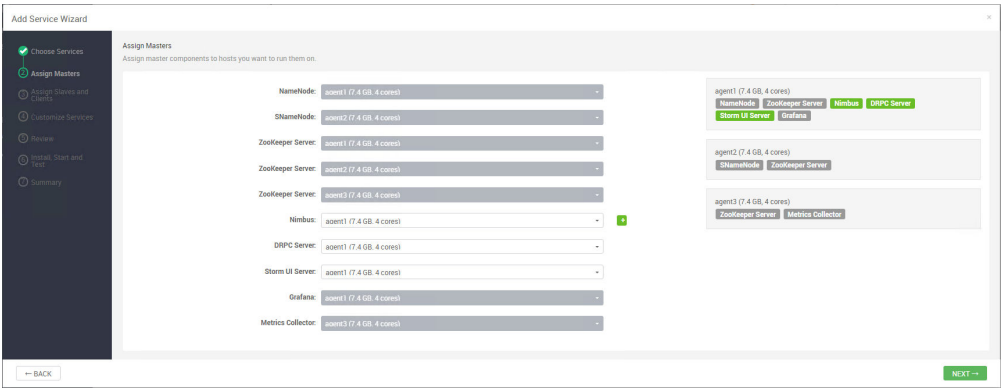
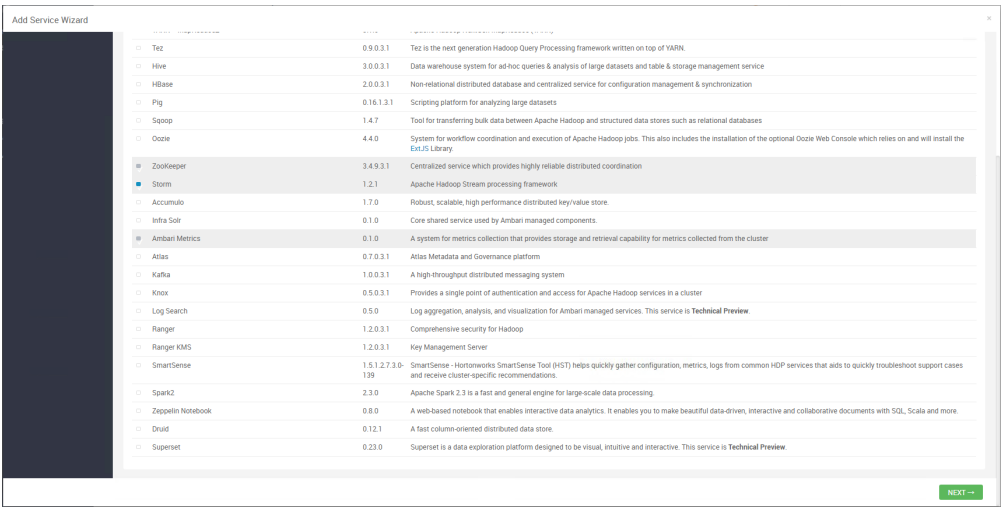
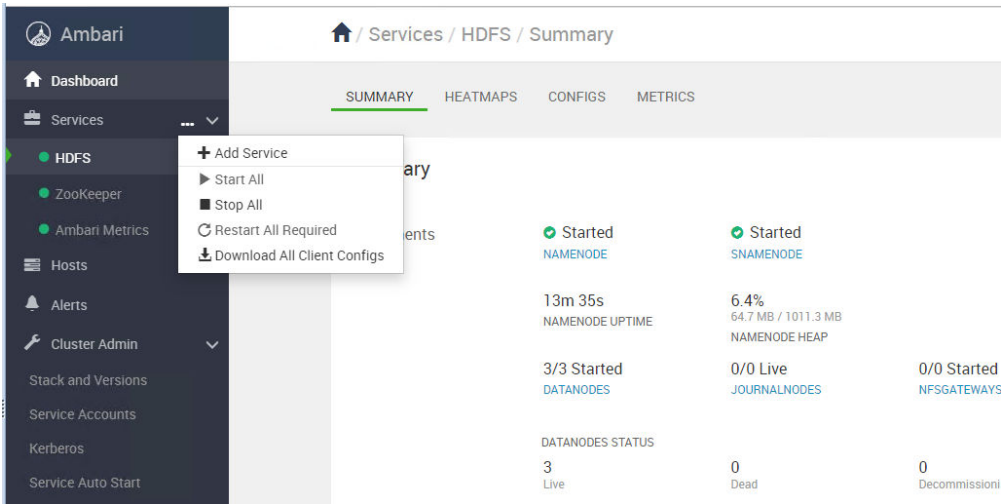
16.1 部署 Storm 服务

步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。

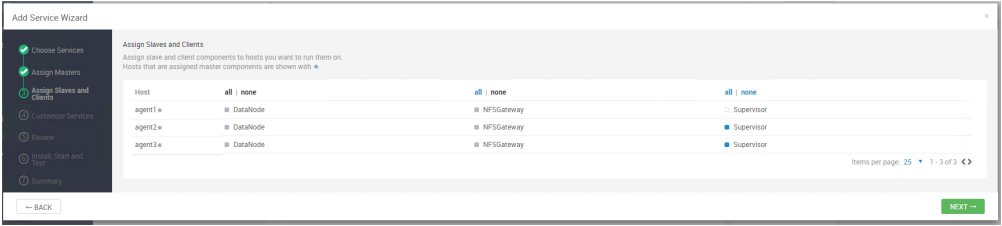


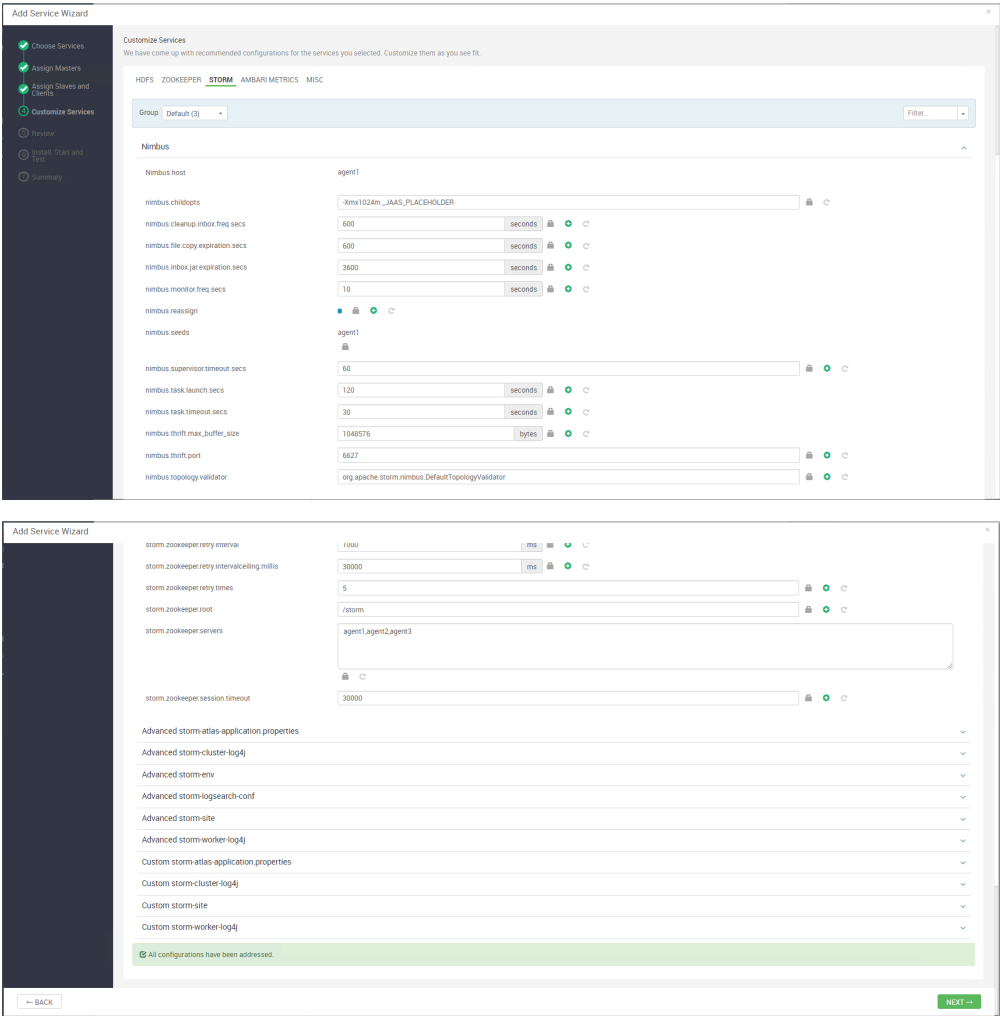
步骤2 添加Storm服务。

1. 单击Add Service，选择Storm。

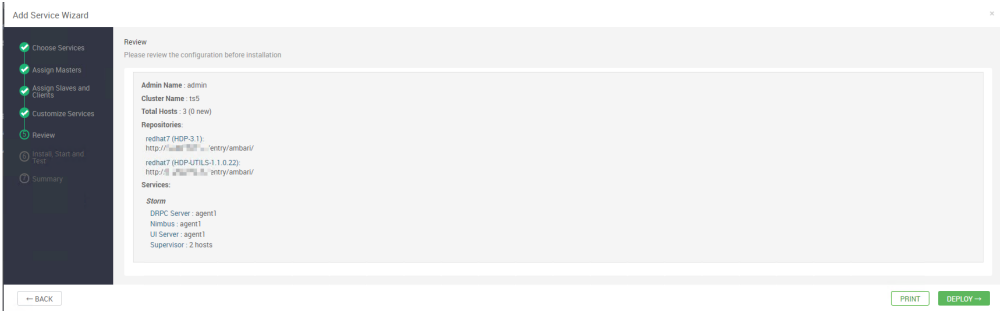


2. 为数据节点勾选Supervisor。

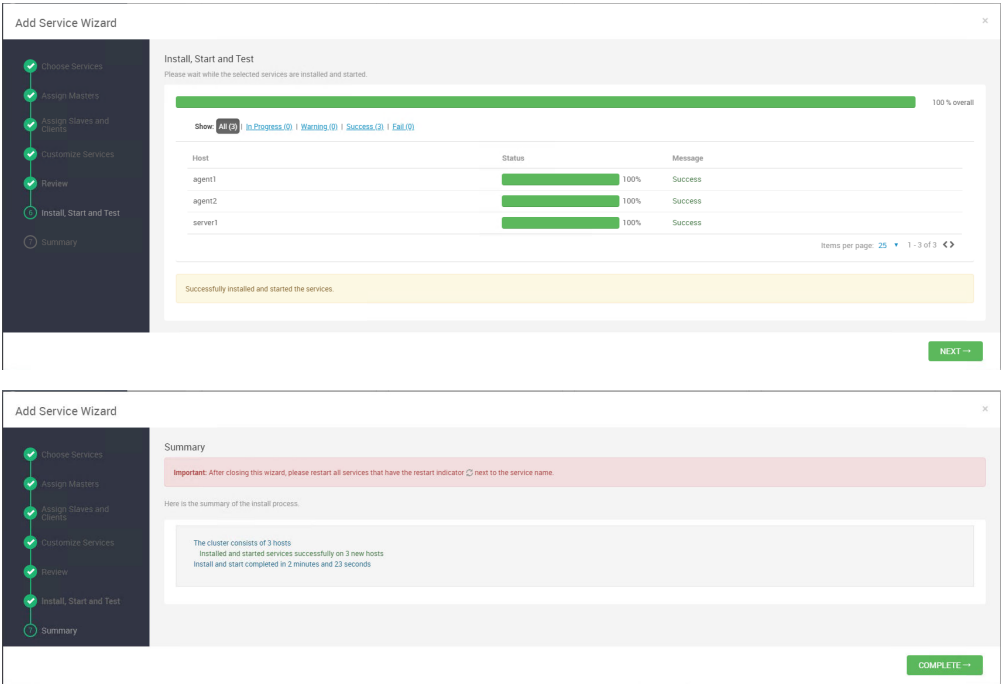




3. 单击DEPLOY。



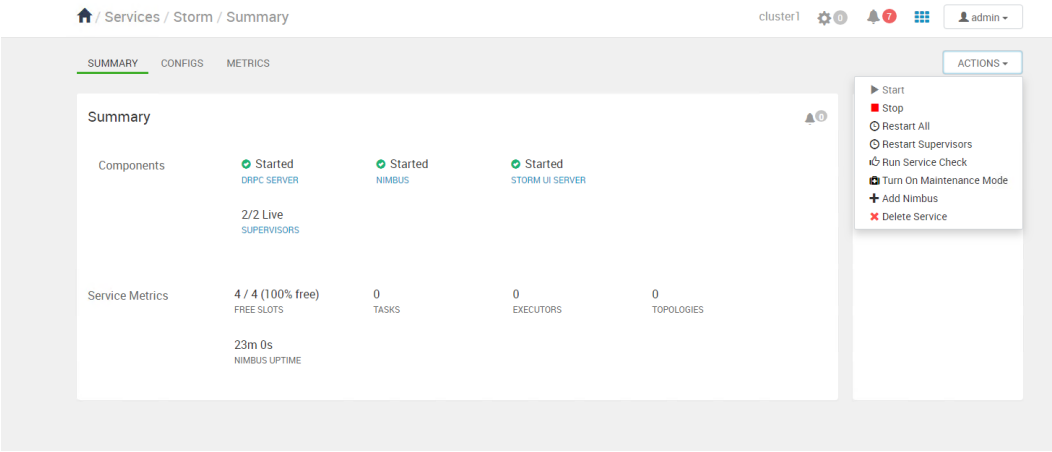
4. 单击NEXT，等待安装完成。



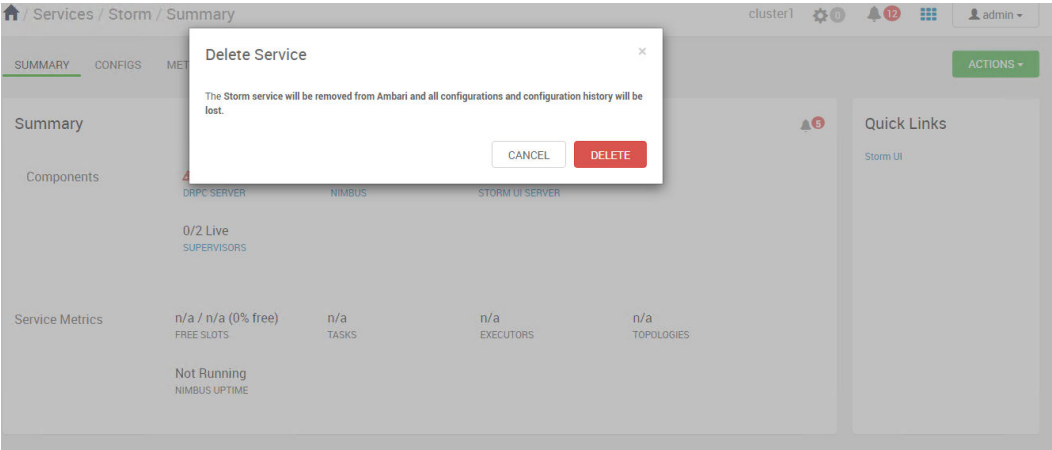
----结束

16.2 删除 Storm 服务

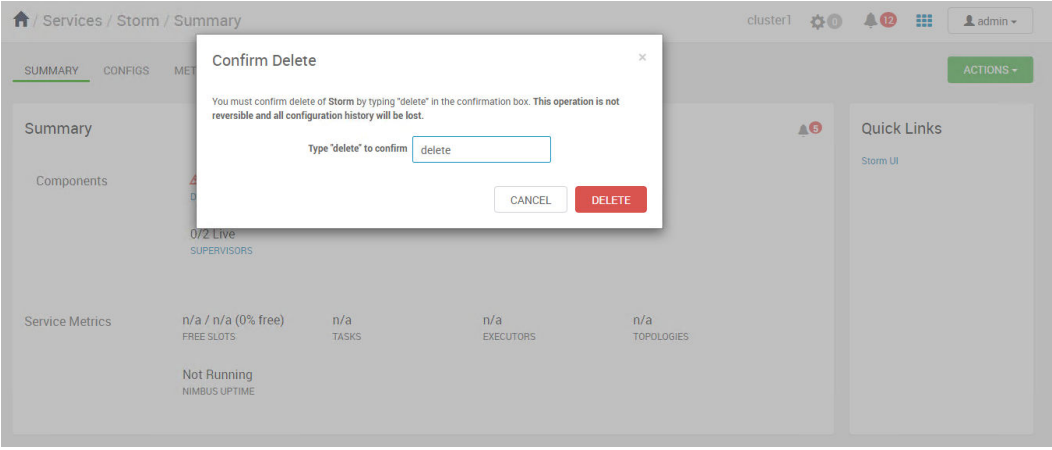
步骤1 在Storm界面单击ACTIONS，选择Stop，先停止服务。



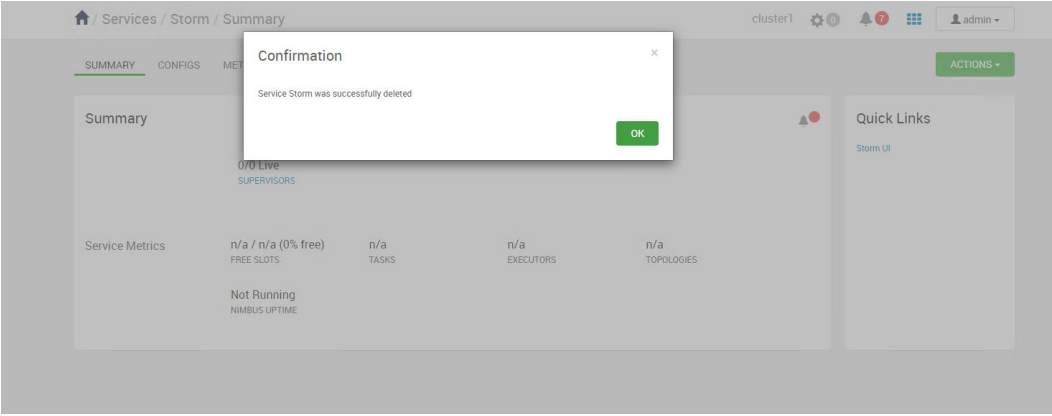
步骤2 停止服务后选择Delete Service，单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

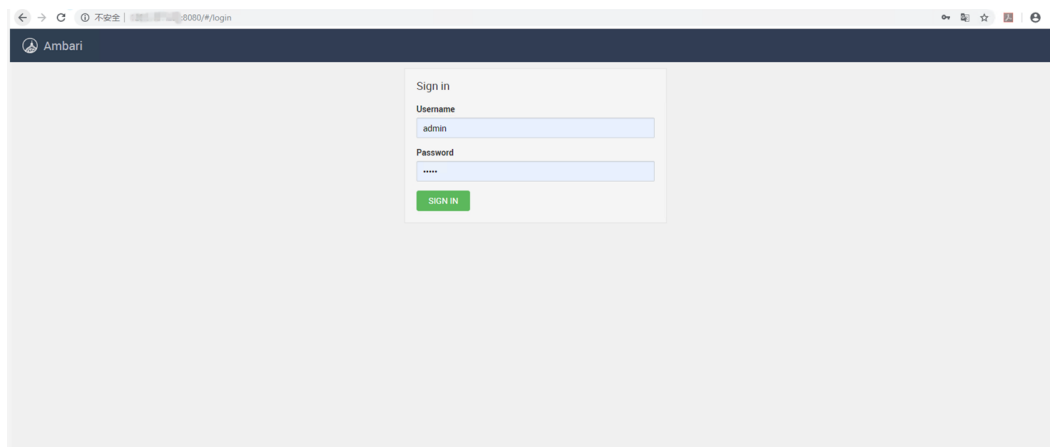
17 部署 Zeppelin

[17.1 部署Zeppelin服务](#)

[17.2 删除Zeppelin服务](#)

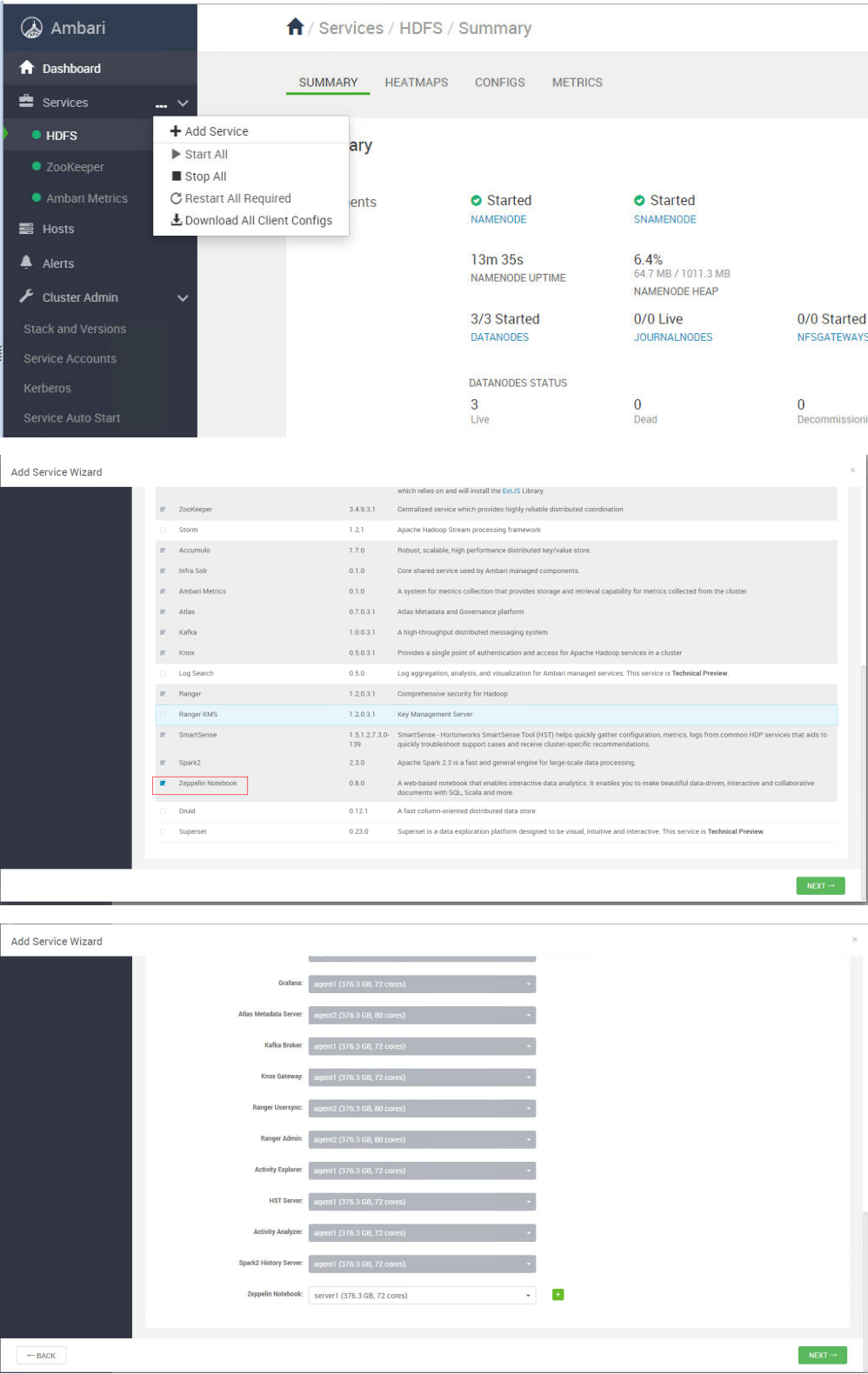
17.1 部署 Zeppelin 服务

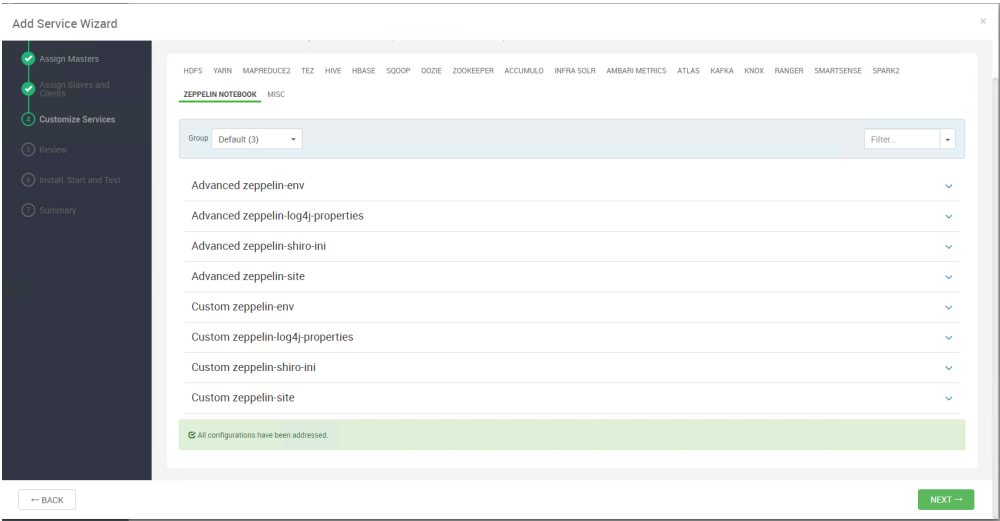
步骤1 启动ambari-server后，通过客户端浏览器登录Ambari的界面。



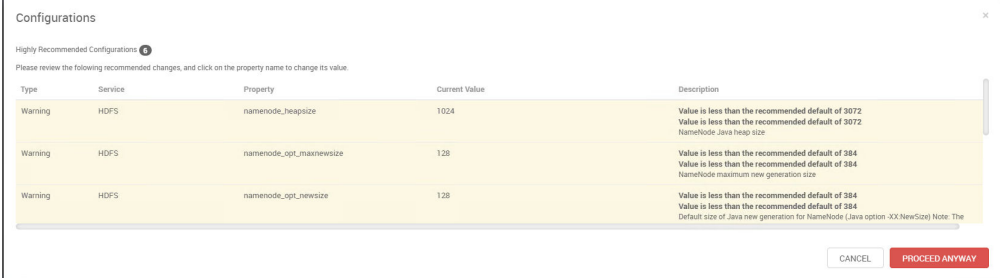
步骤2 添加Zeppelin服务。

1. 单击Add Service，选择Zeppelin。

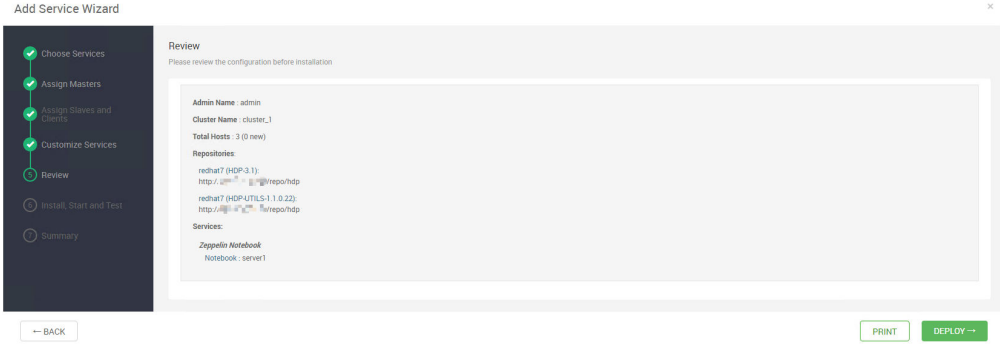




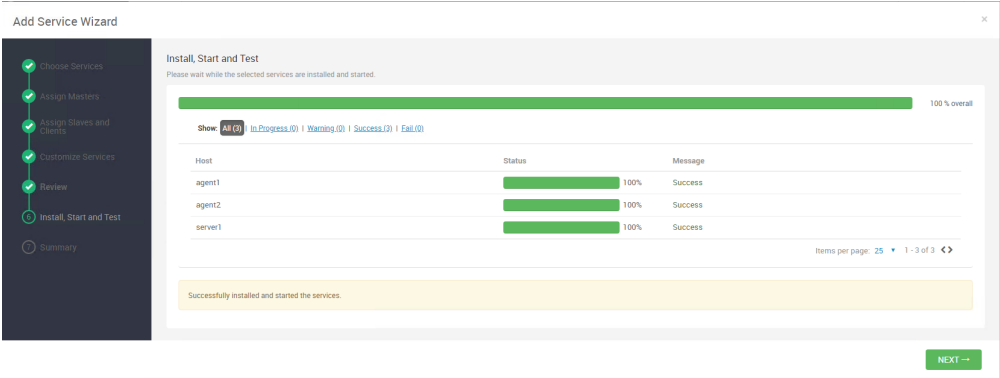
2. 如有弹框，跳过即可，不影响部署，也可按照弹框提示对参数进行修改。



3. 单击DEPLOY。



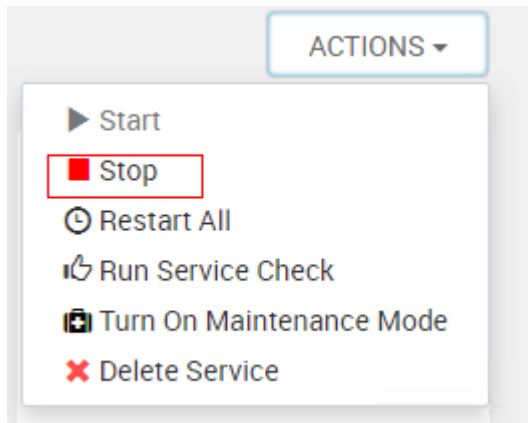
4. 单击NEXT，等待安装完成。



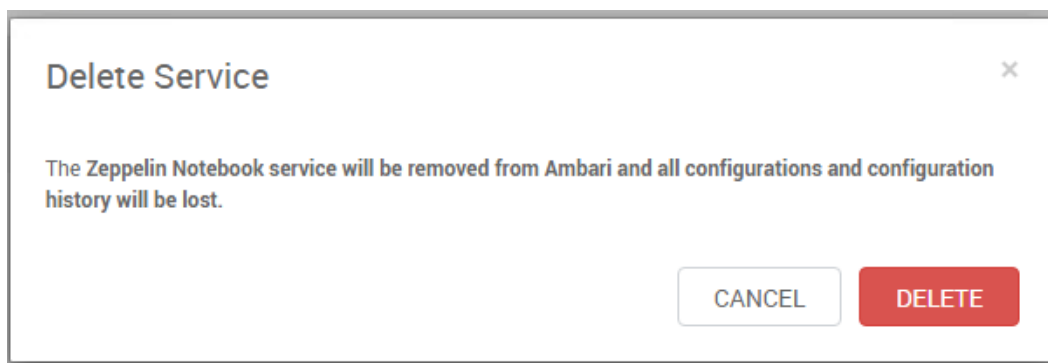
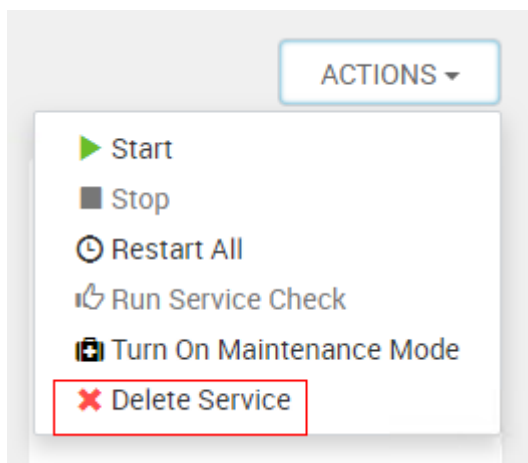
----结束

17.2 删除 Zeppelin 服务

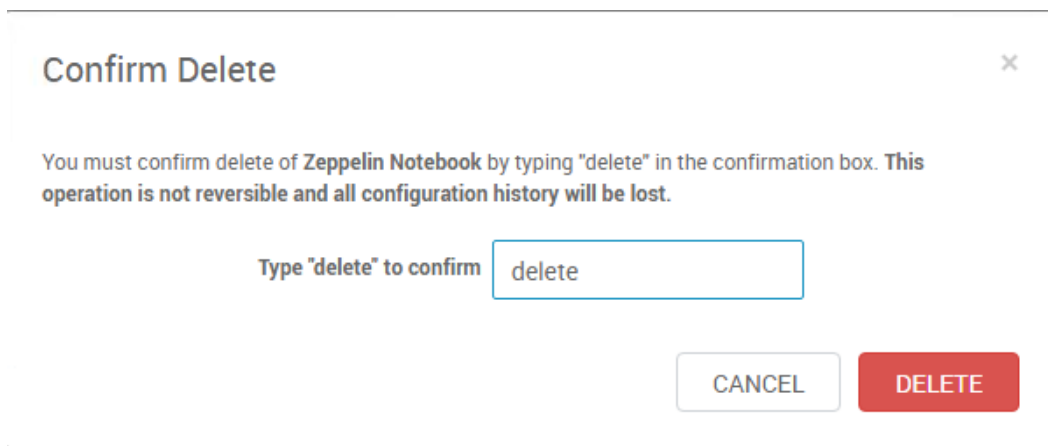
步骤1 在Zeppelin界面单击ACTIONS，选择Stop，先停止服务。



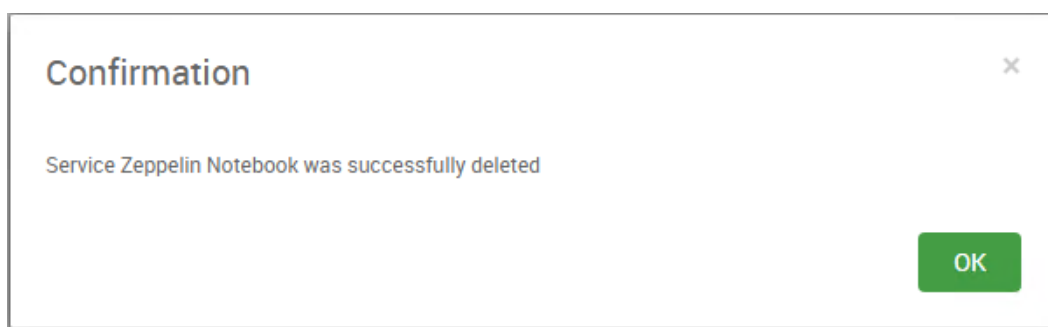
步骤2 停止服务后选择Delete Service，并单击DELETE。



步骤3 在对话框中输入delete，并单击DELETE。



步骤4 删除完成后单击OK。



----结束

18 卸载 Ambari 集群

步骤1 在集群中所有服务器上执行以下脚本。

脚本下载地址：

https://mirrors.huaweicloud.com/kunpeng/archive/kunpeng_solution/bigdata/Scripts/uninstall.zip

解压并安装：

```
unzip uninstall.zip
cd uninstall
sh uninstall/uninstallHDP.sh
```

步骤2 删除创建的数据库。

1、登录mysql，提示输入密码时，输入**4.1 安装数据库**中设置的密码即可。

```
mysql -u root -p
```

2、删除ambari和hive数据库。

```
MariaDB [(none)]> drop database ambari;
Query OK, 0 rows affected (0.00 sec)
```

```
MariaDB [(none)]> drop database hive;
Query OK, 0 rows affected (0.00 sec)
```

----结束

须知

卸载脚本会将mysql-connector-java.noarch也一同卸载，若需再次安装，则需重新安装mysql-connector-java.noarch，具体步骤请参考**步骤1**

19 故障排除

- [19.1 无权访问httpd文件服务器下级目录](#)
- [19.2 提示未绑定变量](#)
- [19.3 安装cluster失败](#)
- [19.4 Ambari部署时进度部署到5%时立刻失败](#)
- [19.5 在主机注册时失败](#)
- [19.6 提示Connection failed: \[Errno 111\] Connection refused to XXX:16030](#)
- [19.7 ambari-metrics启动失败: cannot import name _common](#)
- [19.8 ambari-server启动失败: listening on http port 8080 after 90 seconds](#)
- [19.9 提示需要安装openeuler-lsb和python2-kerberos](#)

19.1 无权访问 httpd 文件服务器下级目录

现象描述

无法访问httpd文件服务器的下一级目录，提示无权限访问。

```
You don't have permission to access /entry/ on this server.
```

问题分析

1. 查看系统中的日志（“/etc/httpd/logs/error_log或var/log/httpd/error_log”），发现selinux权限处于打开状态，没有权限访问“/var/www/html/entry”目录；报错日志内容：

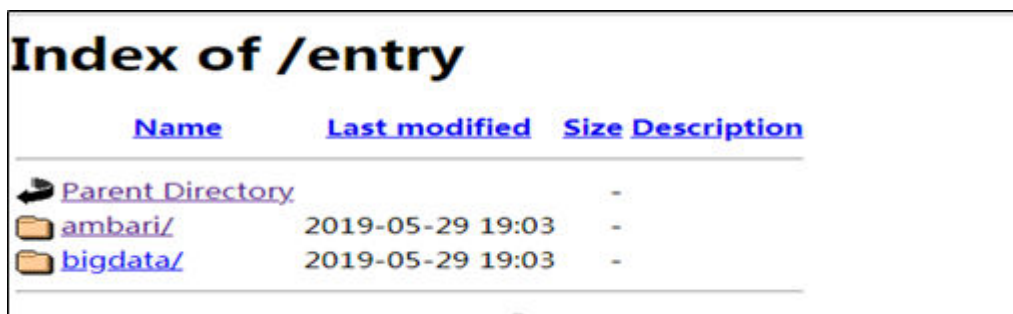
```
SELinux policy enabled; httpd running as context system_u:system_r:httpd_t:s0  
Permission denied: [client 192.168.100.157:50777] AH01275: Can't open directory for  
index: /var/www/html/entry/
```
2. 采用getenforce查看selinux权限状态显示Enforcing，说明selinux权限处于打开状态。

解决方法

步骤1 修改“/etc/sysconfig/selinux”文件如下，将其置为disabled状态。

```
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled - No SELinux policy is loaded.
SELINUX=disabled
# SELINUXTYPE= can take one of three values:
#   targeted - Targeted processes are protected,
#   minimum - Modification of targeted policy. Only selected processes are protected.
#   mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

步骤2 重启httpd服务，再次访问，能够正常访问了。



----结束

19.2 提示未绑定变量

现象描述

在执行ambari-server setup时报错。

```
/usr/sbin/ambari-server: line 34: buildNumber: unbound variable
```

问题分析

```
vim /usr/sbin/ambari-server
# /etc/init.d/ambari-server
set -u # fail on unset variables
VERSION="2.7.3.0-0"
HASH="${buildNumber}"
```

buildNumber是一个变量，是没有具体的值导致该问题。

解决方法

步骤1 在“/etc/profile”文件末尾处增加下面的语句，解决找不到此变量的问题。

```
export buildNumber=2.7.3.0.0
```

步骤2 使环境变量生效。

```
source /etc/profile
```

----结束

19.3 安装 cluster 失败

现象描述

安装cluster失败，提示“Failed to connect to https://localhost:8440/ca”。

问题分析

1. 单击网页上的Failed，查看日志，发现不是IP地址，是localhost，可能是域名的问题。
2. 前面有报错buildNumber找不到，当时在ambari-server启动的时候也报了这个错误（请参见[19.2 提示未绑定变量](#)），也有可能是引起后面的根因。

```
Command start time 2019-06-03 17:35:40
/var/lib/ambari-agent/tmp/ambari_commons/subprocess32.py:156: RuntimeWarning: The
_posixsubprocess module is not being used. Could not import it from ambari_commons.libs.aarch64
Child process reliability may suffer if your program uses threads.
"program uses threads.", RuntimeWarning)
/var/lib/ambari-agent/bin/ambari-agent: line 23: buildNumber: unbound variable
('WARNING 2019-06-03 17:34:37,729 NetUtil.py:115 - Server at https://localhost:8440 is not
reachable, sleeping for 10 seconds...
INFO 2019-06-03 17:34:47,729 NetUtil.py:61 - Connecting to https://localhost:8440/ca
WARNING 2019-06-03 17:34:47,730 NetUtil.py:92 - Failed to connect to https://localhost:8440/ca due
to [Errno 111] Connection refused
WARNING 2019-06-03 17:34:47,730 NetUtil.py:115 - Server at https://localhost:8440 is not reachable,
sleeping for 10 seconds...
INFO 2019-06-03 17:34:57,731 main.py:481 - Connecting to Ambari server at https://localhost:8440
(127.0.0.1)
INFO 2019-06-03 17:34:57,731 NetUtil.py:61 - Connecting to https://localhost:8440/ca
Connection to 172-19-43-19 closed.
```

解决方法

在所有ambari-agent节点以及ambari-server节点上，增加环境变量，在“/etc/profile”里面增加如下代码：

```
export buildNumber="2.7.3.0.0"
```

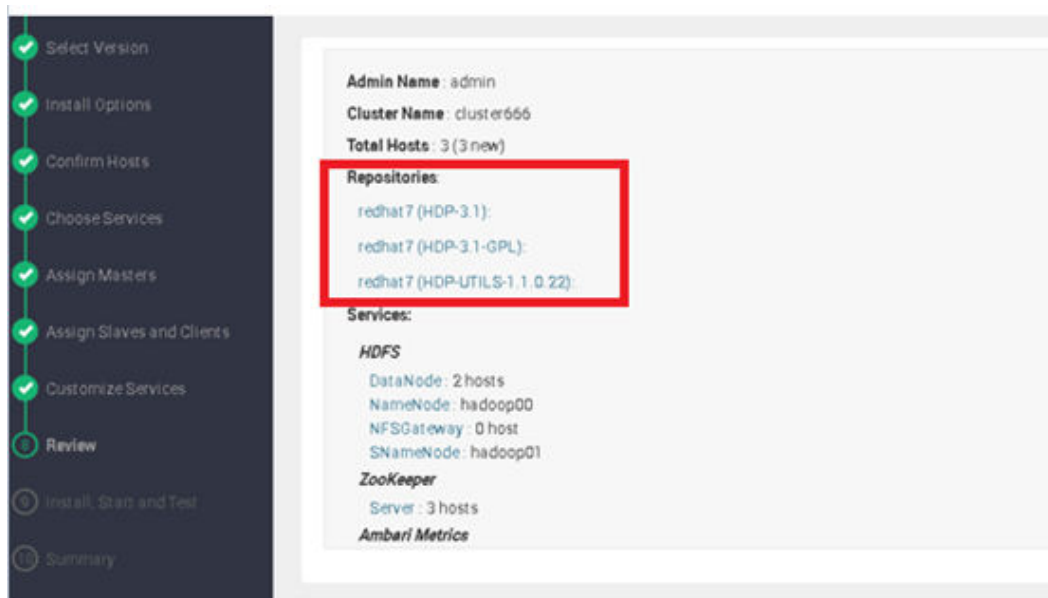
19.4 Ambari 部署时进度部署到 5%时立刻失败

问题现象

Ambari部署时进度部署到5%时立刻失败

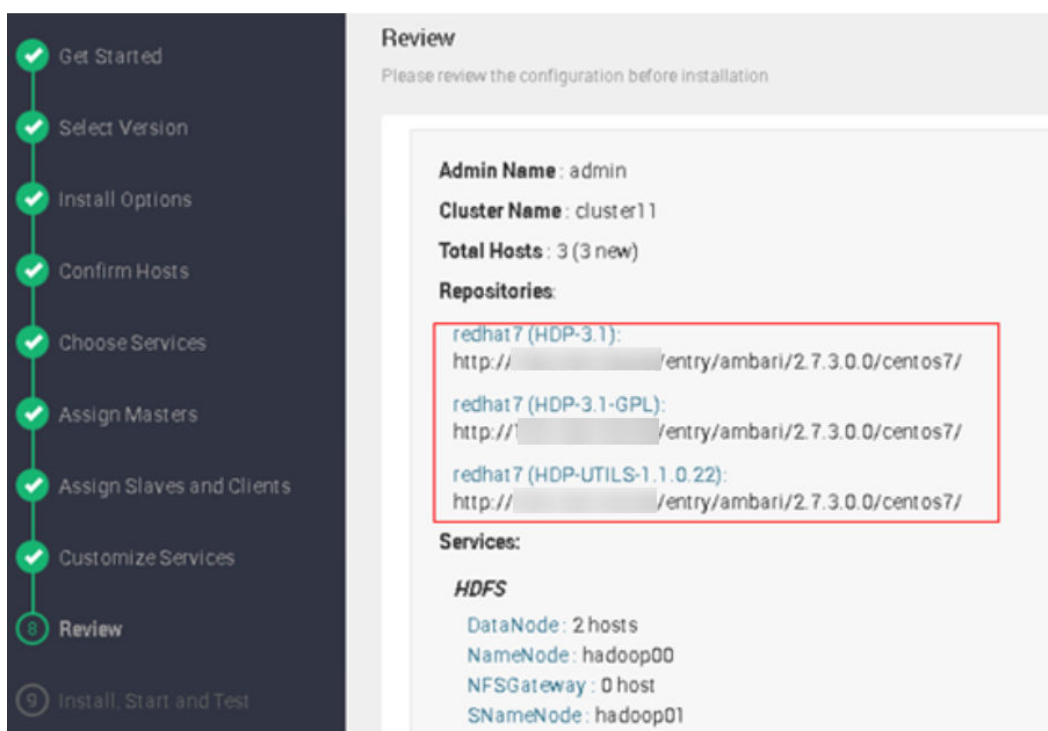
问题分析

问题主要根因是部署时在Review步骤，该界面下的Repositories中URL没有显示出来。



解决方法

需要在此页面按F5刷新页面等URL显示出来才能进入下一步骤，不然部署时由于找不到Yum源而在进度5%处迅速失败。



19.5 在主机注册时失败

问题现象

在主机注册时失败。

问题分析

一般是/etc/yum.repos.d中的yum源有问题，或者有多余的*.repo文件未删除。

```
[root@hadoop00 yum.repos.d]# ll
total 16
-rw-r--r--. 1 root root 413 Dec 14 15:06 ambari-hdp-1.repo
-rw-r--r--. 1 root root 110 Dec 14 14:43 ambari.repo
drwxr-xr-x. 2 root root 4096 Dec 13 20:38 bak
-rw-r--r--. 1 root root 68 Dec 13 20:39 local.repo
```

解决方法

步骤1 将yum.repos.d中的repo文件除ambari.repo和local.repo的文件之外的ambari-hdp-1.repo文件删除。

步骤2 重新生效源地址。

```
yum clean all
yum makecache
```

----结束

19.6 提示 Connection failed: [Errno 111] Connection refused to XXX:16030

问题现象

提示Connection failed: [Errno 111] Connection refused to XXX:16030。

解决方法

在hs_err_pid开头的文件中，有提示的办法，在主机上执行如下命令：

```
ulimit -c unlimited
```

在ambari页面上重新启动regionserver，启动成功，告警消失了。

19.7 ambari-metrics 启动失败：cannot import name _common

问题现象

ambari-metrics启动时失败，提示错误python报错：ImportError: cannot import name _common.

```
File "/usr/lib/python2.6/site-packages/resource_monitoring/main.py", line 28, in <module>
    from core.controller import Controller
File "/usr/lib/python2.6/site-packages/resource_monitoring/core/controller.py", line 27, in <module>
    from metric_collector import MetricsCollector
File "/usr/lib/python2.6/site-packages/resource_monitoring/core/metric_collector.py", line 23, in <module>
    from host_info import HostInfo
File "/usr/lib/python2.6/site-packages/resource_monitoring/core/host_info.py", line 25, in <module>
    import psutil
File "/usr/lib/python2.6/site-packages/resource_monitoring/psutil/build/lib.linux-aarch64-2.7/psutil/__init__.py", line 89, in <module>
    import psutil._pslinux as _psplatform
File "/usr/lib/python2.6/site-packages/resource_monitoring/psutil/build/lib.linux-aarch64-2.7/psutil/_pslinux.py", line 20, in <module>
    from psutil import _common
ImportError: cannot import name _common
```

解决方法

安装python 2.6依赖。

```
cd /usr/lib/python2.6/site-packages/resource_monitoring/psutil
python setup.py build
python setup.py install
```

19.8 ambari-server 启动失败：listening on http port 8080 after 90 seconds

问题现象

启动ambari-server时报错：Server not yet listening on http port 8080 after 90 seconds. Exiting。

解决方法

修改ambari的web超时时间。在ambari.properties文件中添加超时时间。

```
echo server.startup.web.timeout=120 >> /etc/ambari-server/conf/ambari.properties
```

19.9 提示需要安装 openeuler-lsb 和 python2-kerberos

问题现象

安装过程中提示openeuler-lsb和python2-kerberos需要安装。

解决方法

通过yum安装openeuler-lsb和python2-kerberos。

```
yum install openeuler-lsb -y
yum install python2-kerberos -y
```

A 修订记录

发布日期	修订记录
2021-06-30	第四次正式发布 更新HDP相关组件的部署指南内容及结构。
2020-09-30	第三次正式发布 基于openEuler操作系统提供Ambari的部署指南。
2020-05-23	第二次正式发布 优化文中图形。
2020-03-20	第一次正式发布